

Lecture 17

Flow Networks, Ford-Fulkerson Method

Source: Introduction to Algorithms, CLRS and Kleinberg & Tardos

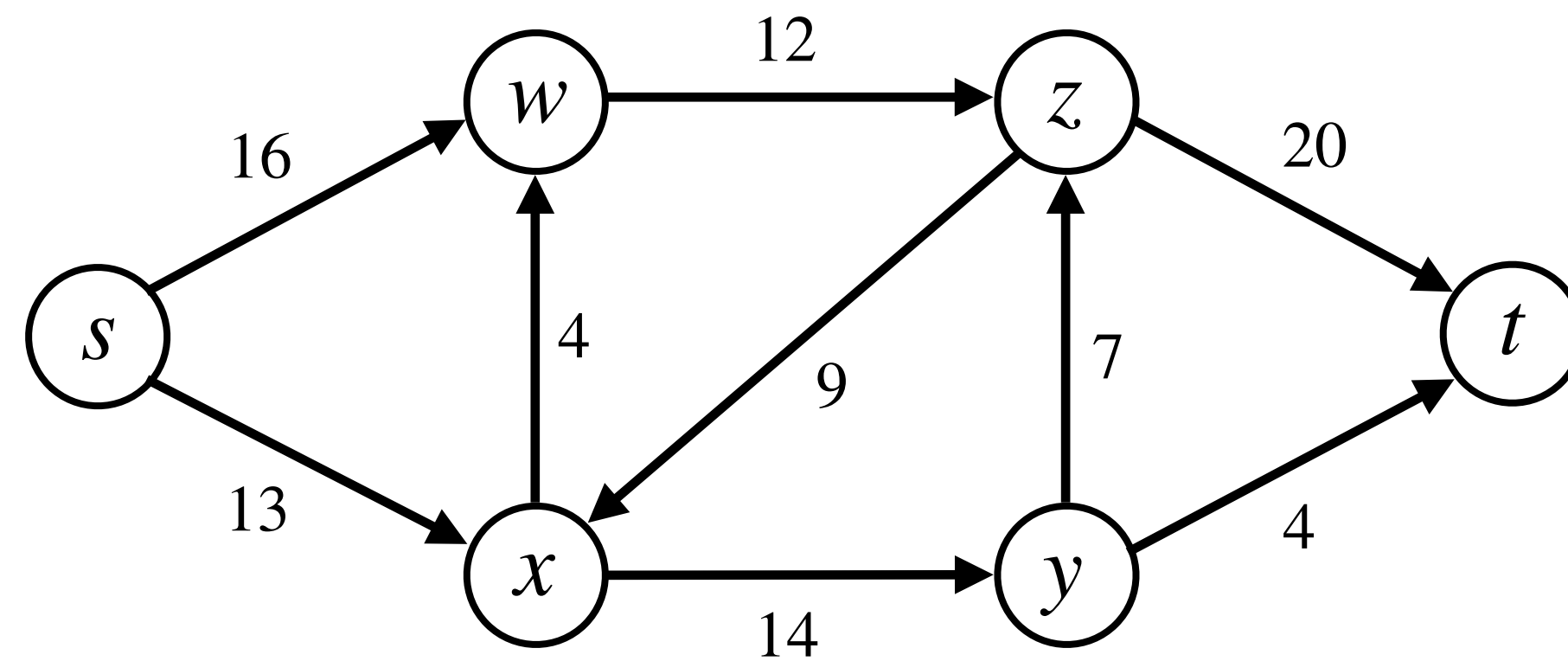
Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

Example:

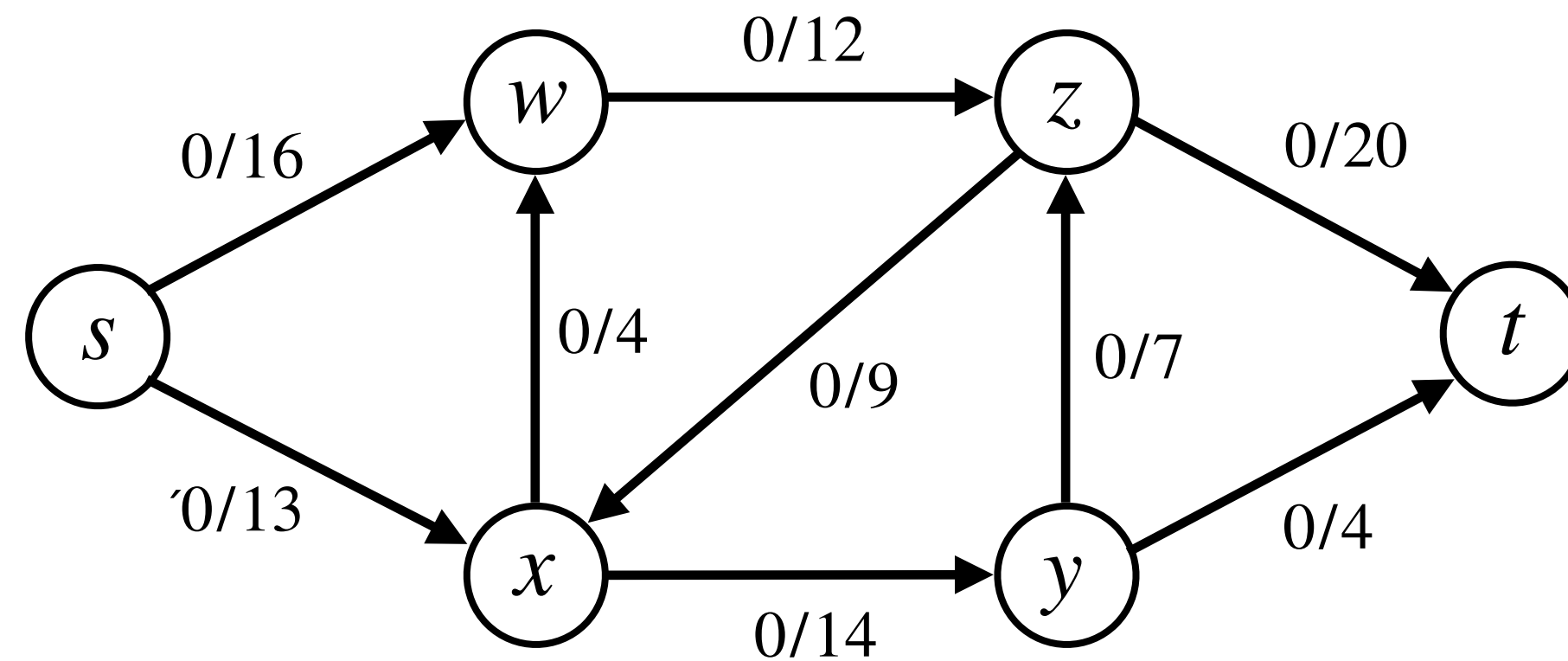


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge

Example:

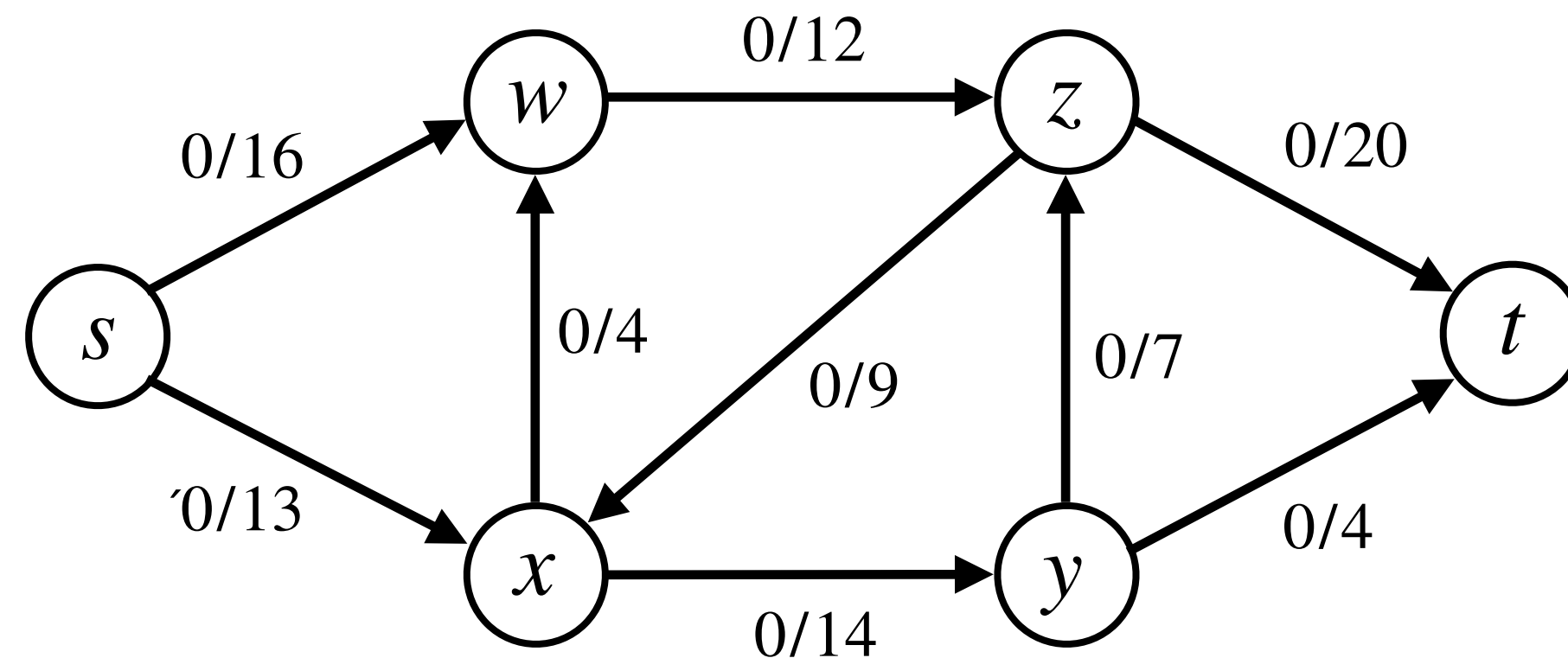


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$

Example:

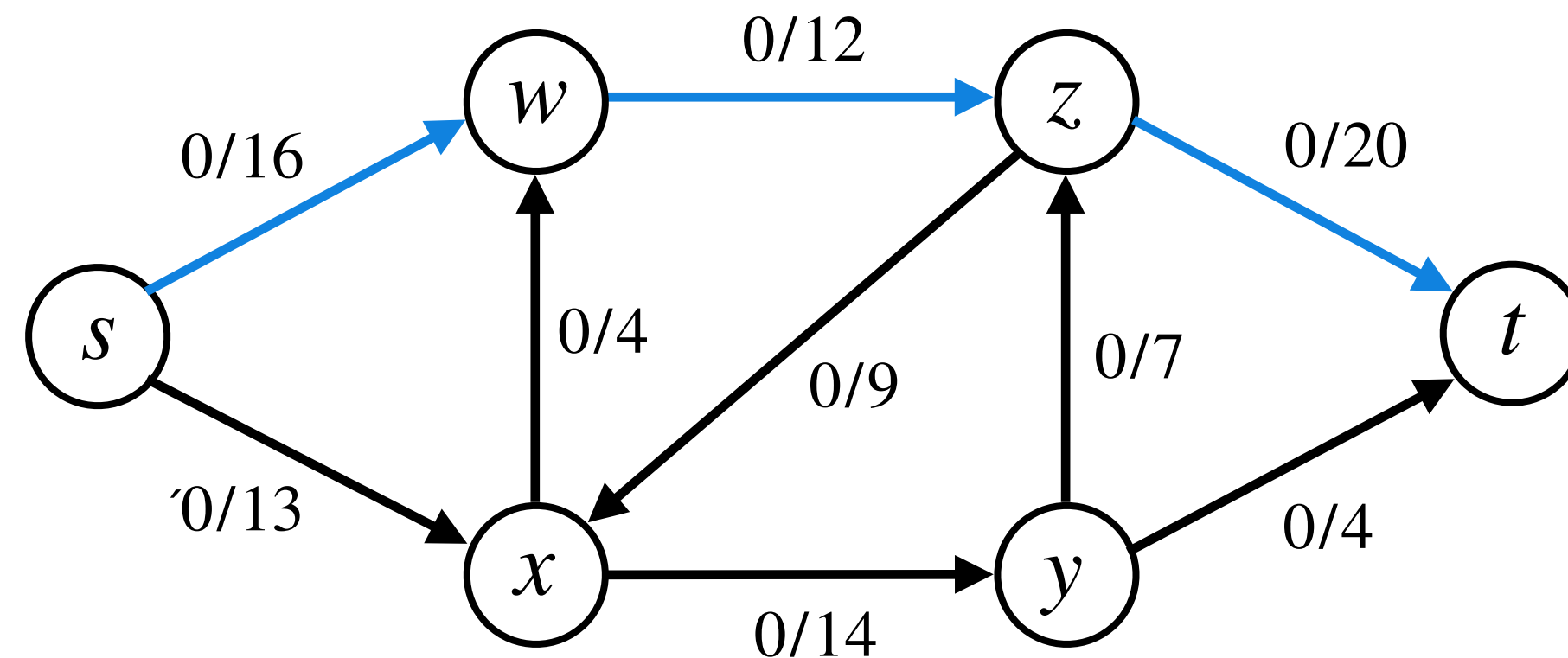


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$

Example:

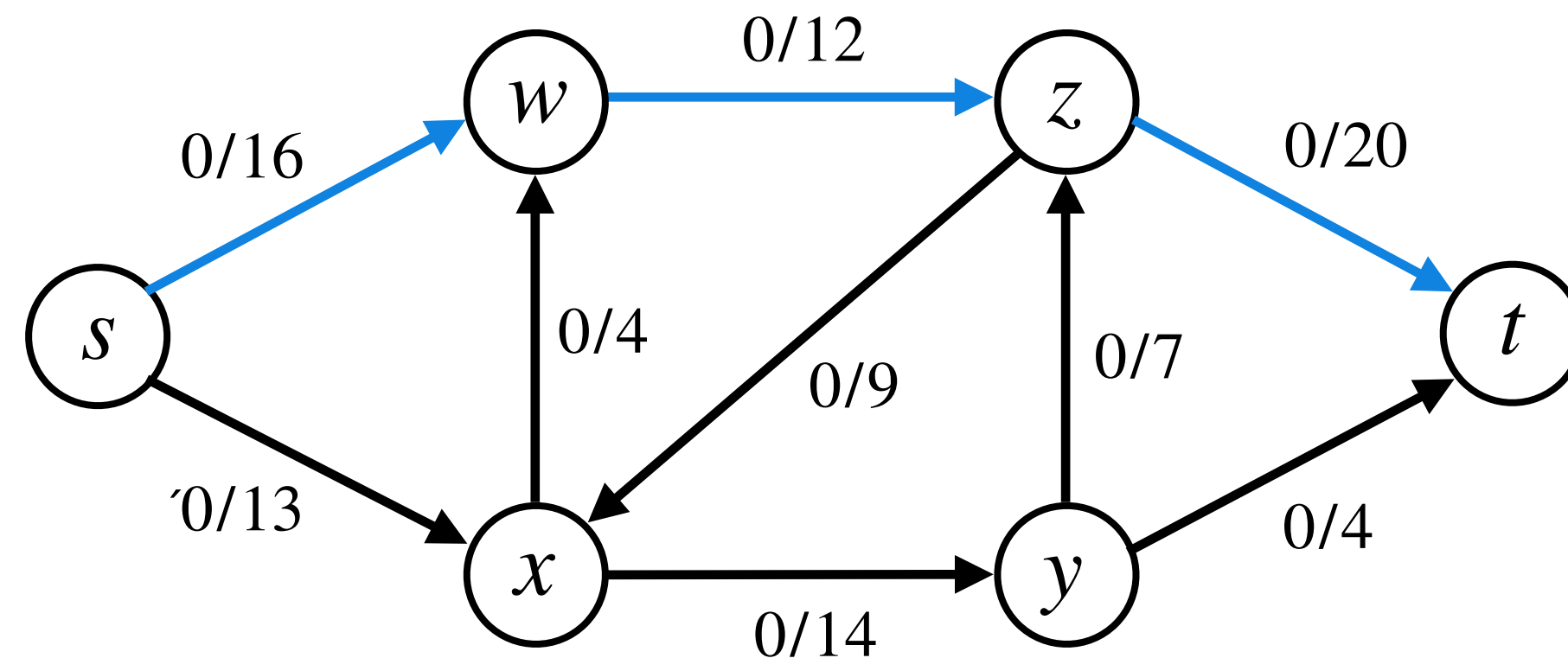


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P

Example:

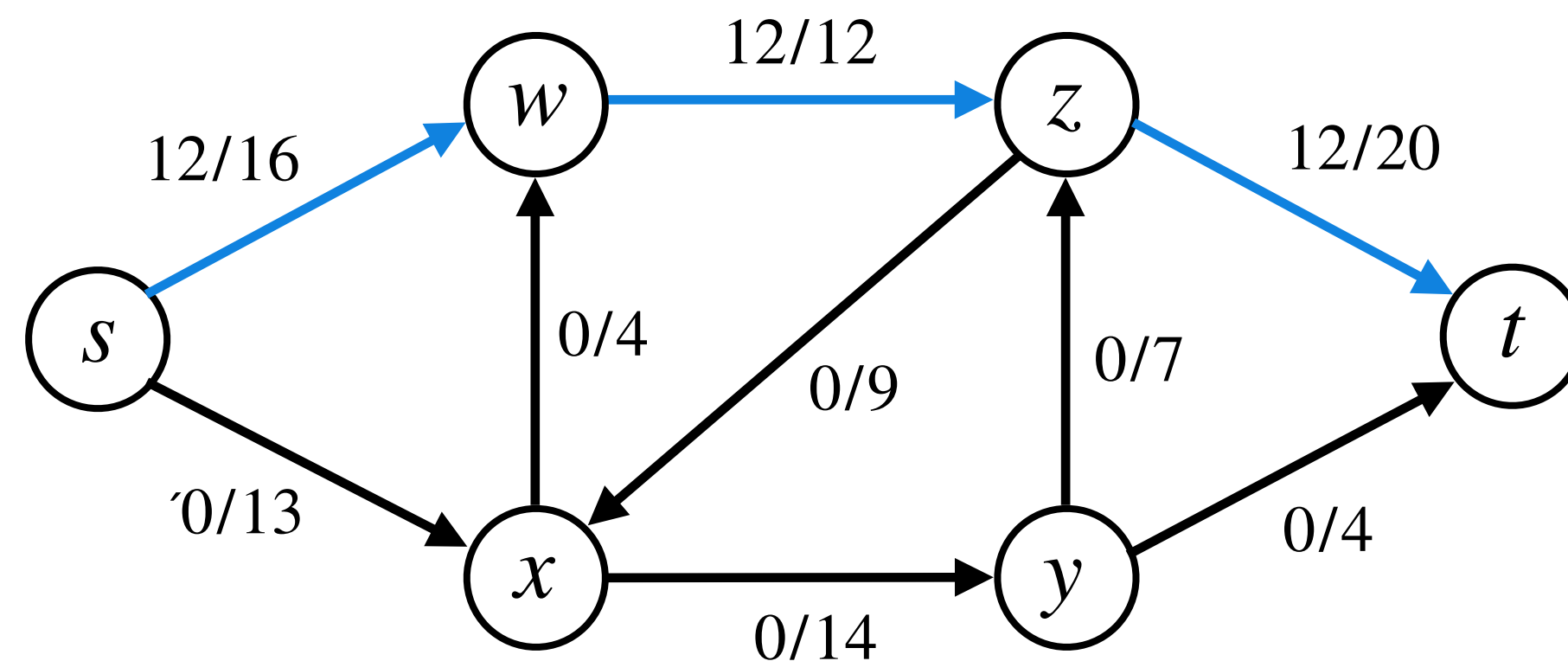


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P

Example:

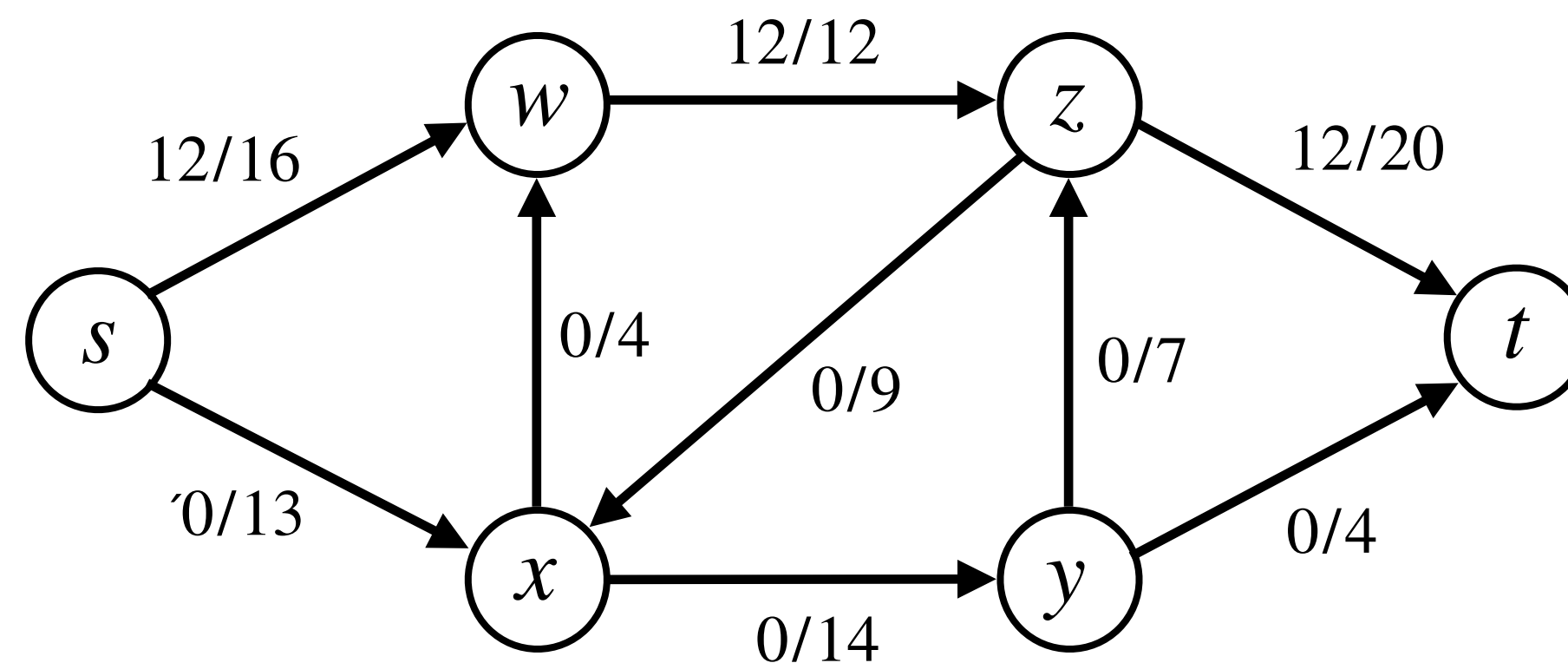


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Example:

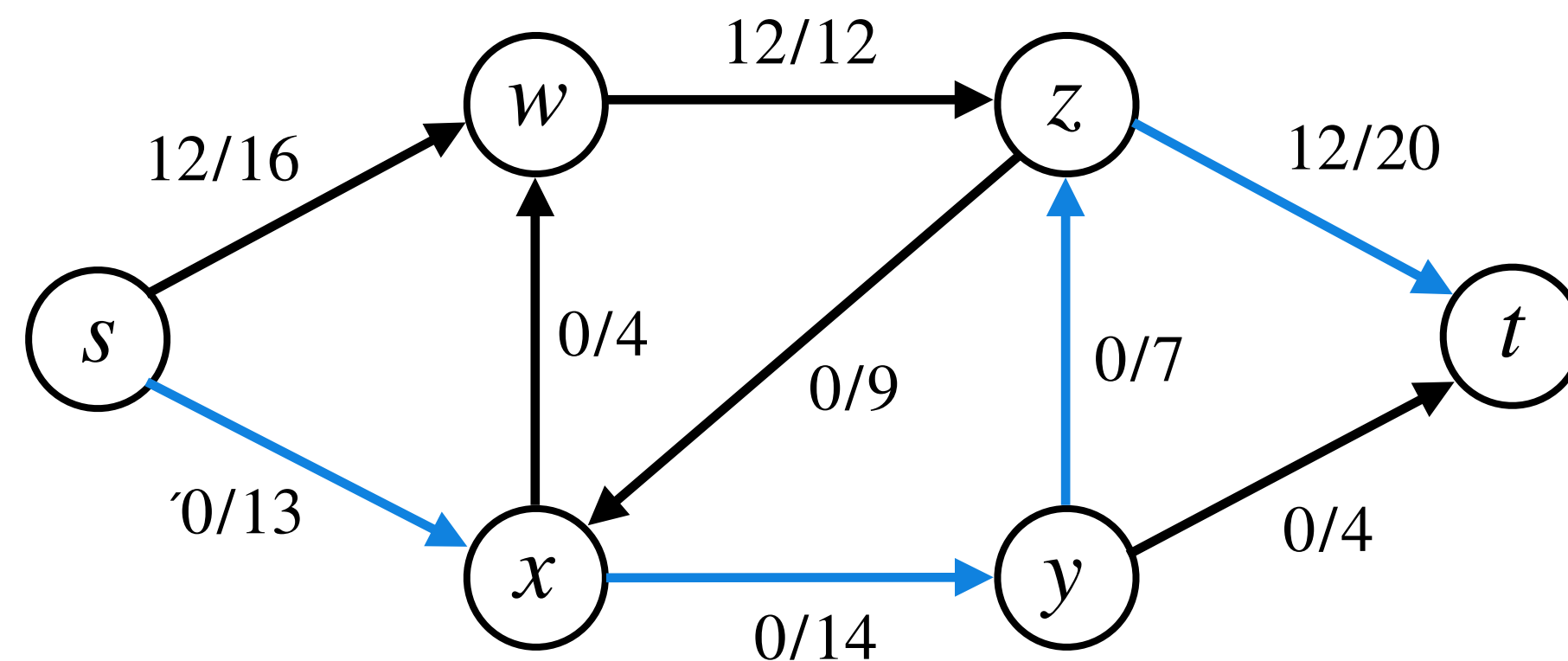


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Example:

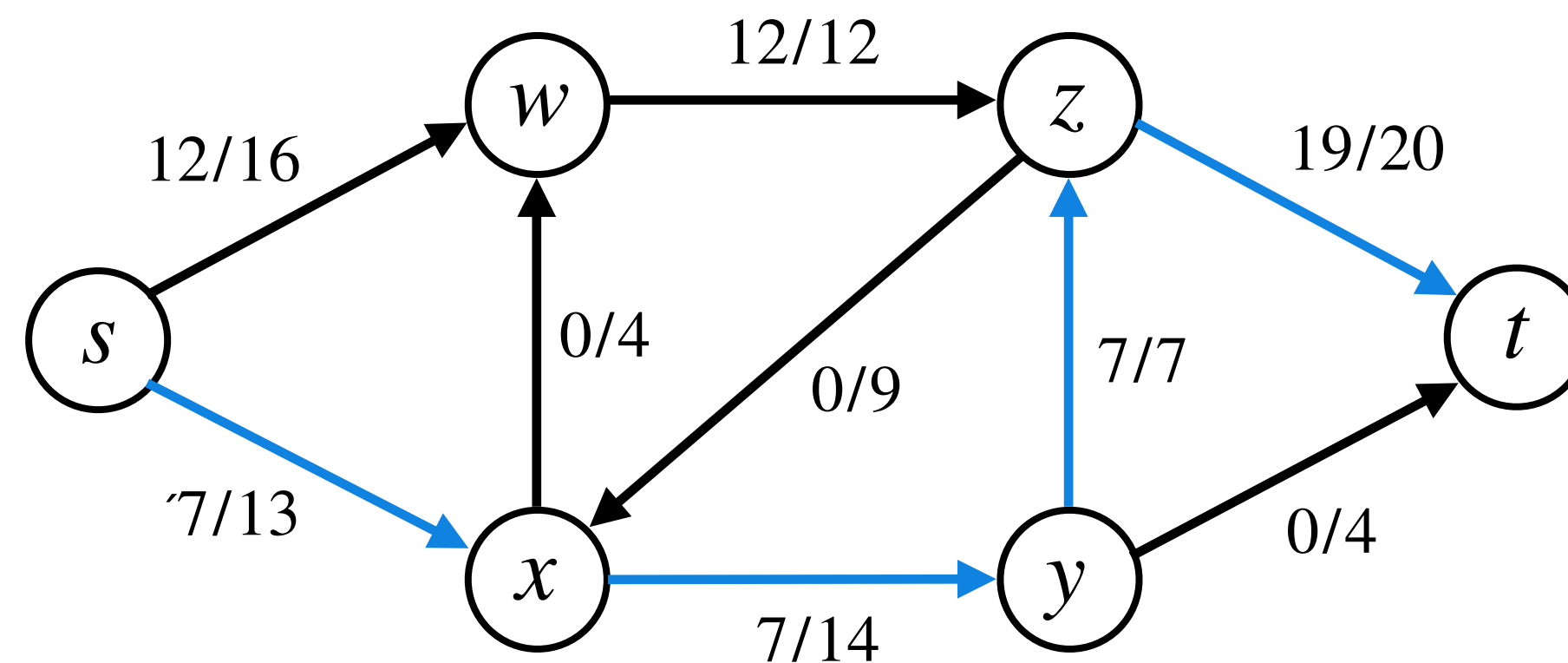


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Example:

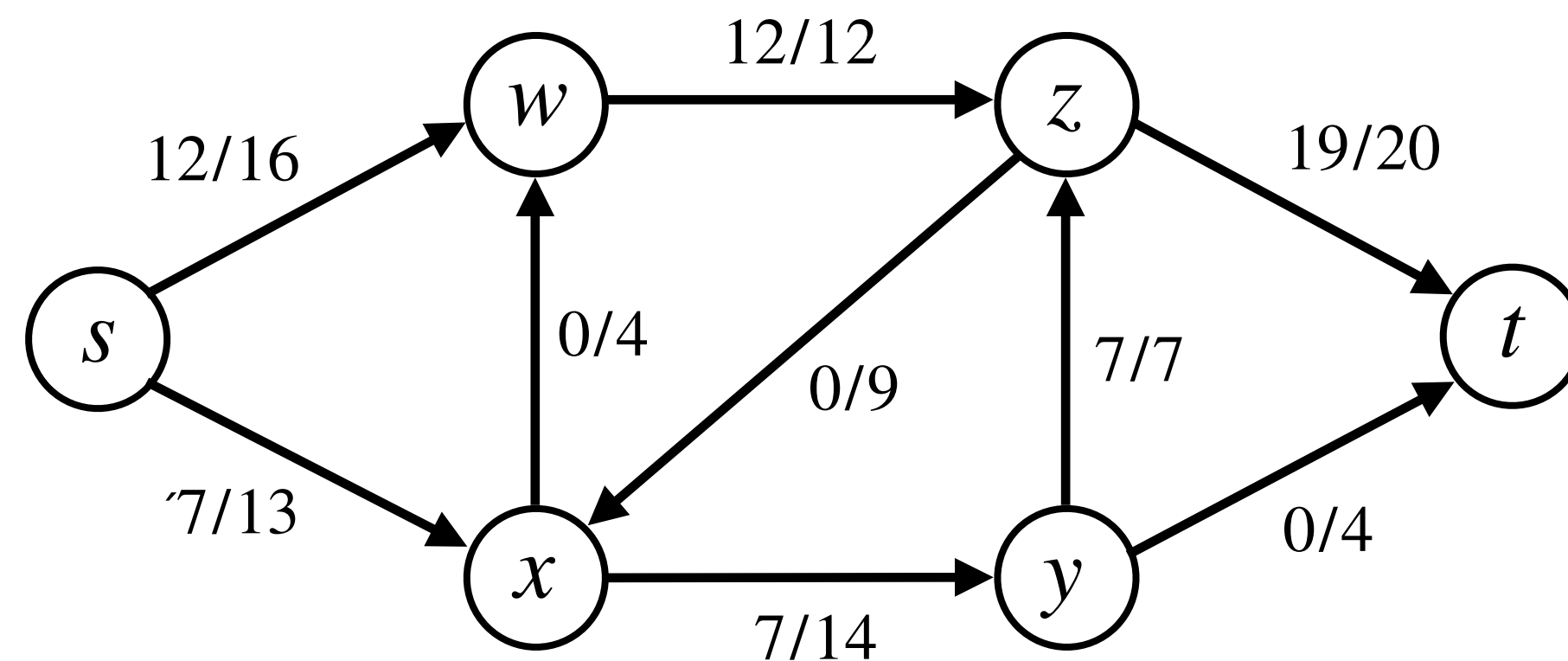


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Example:

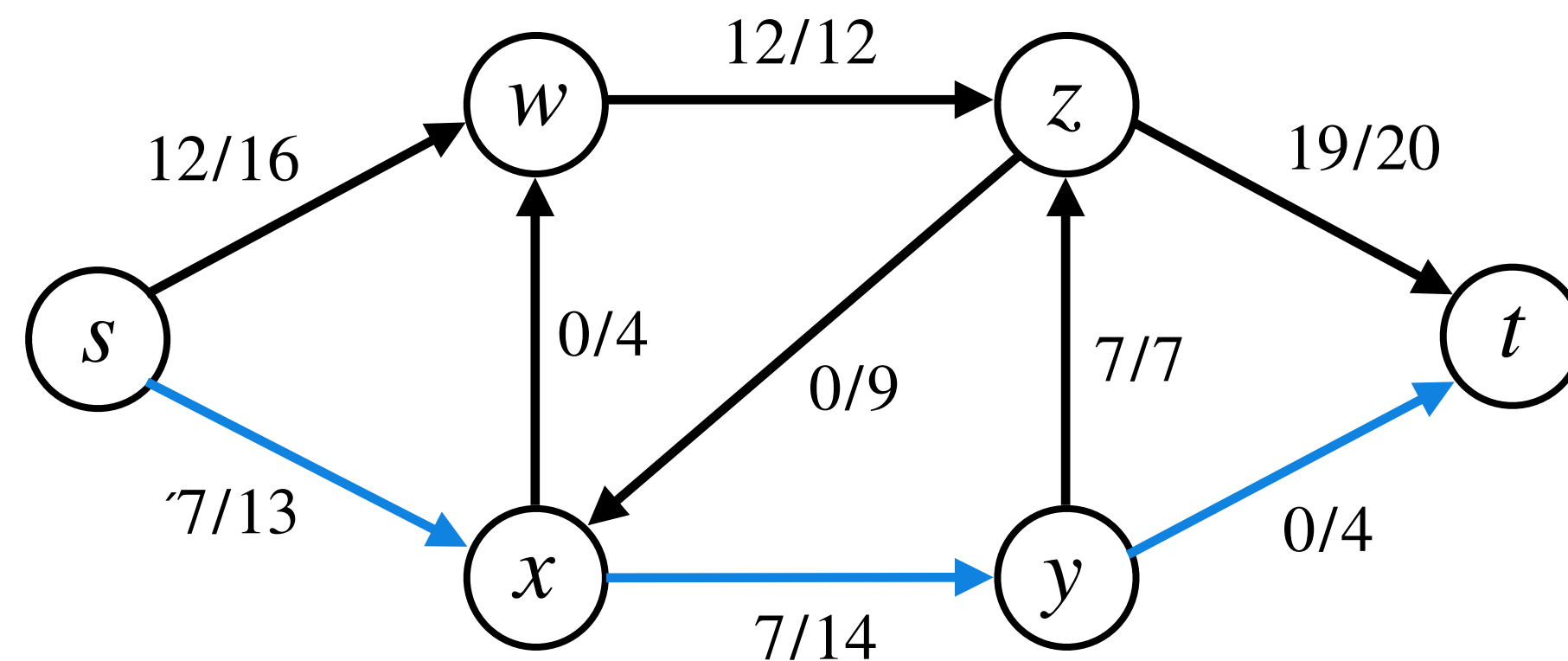


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Example:

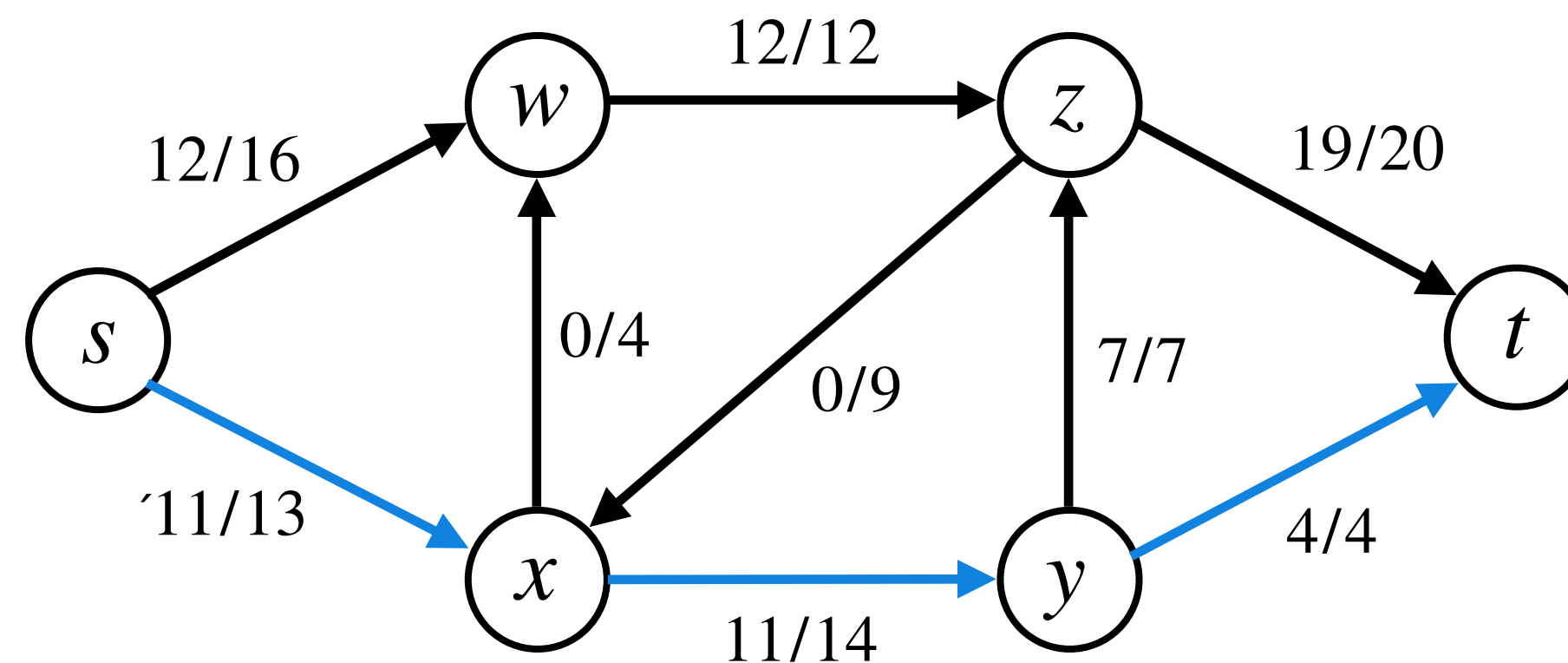


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Example:

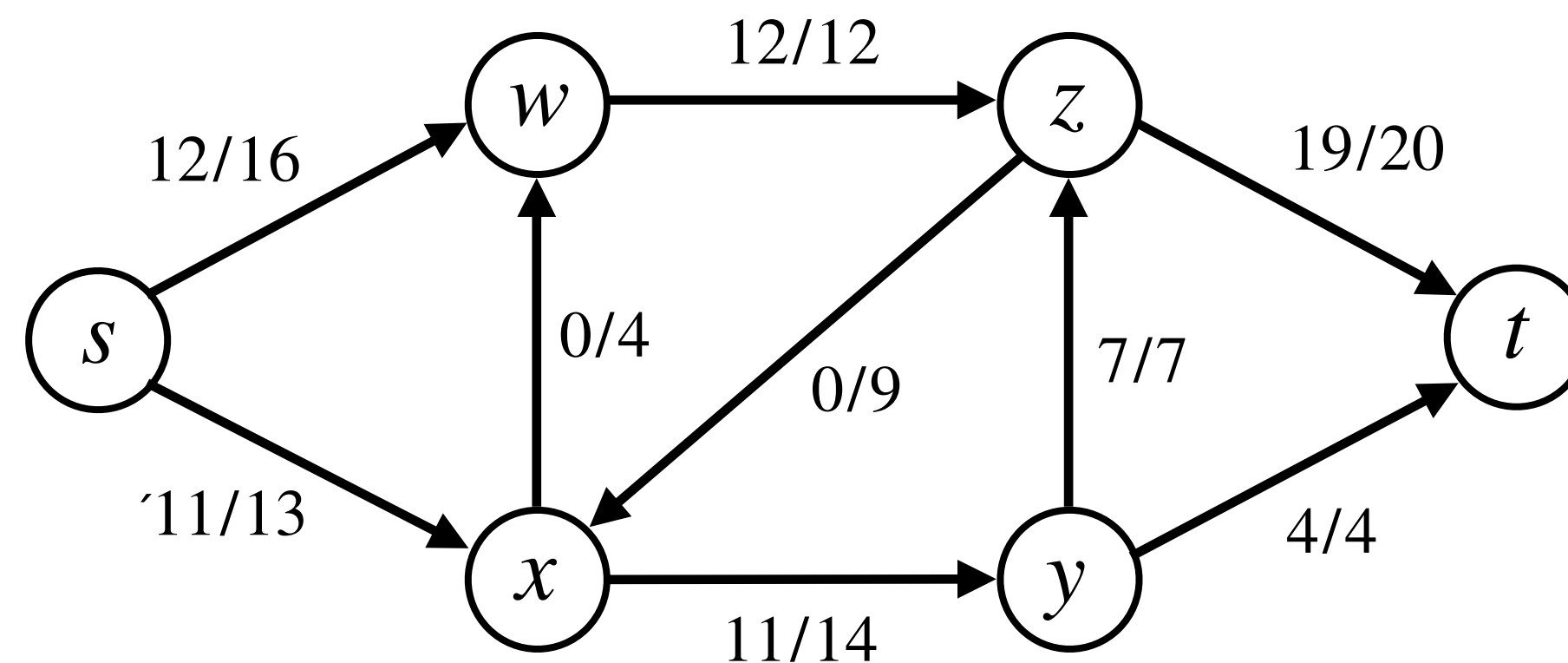


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Example:

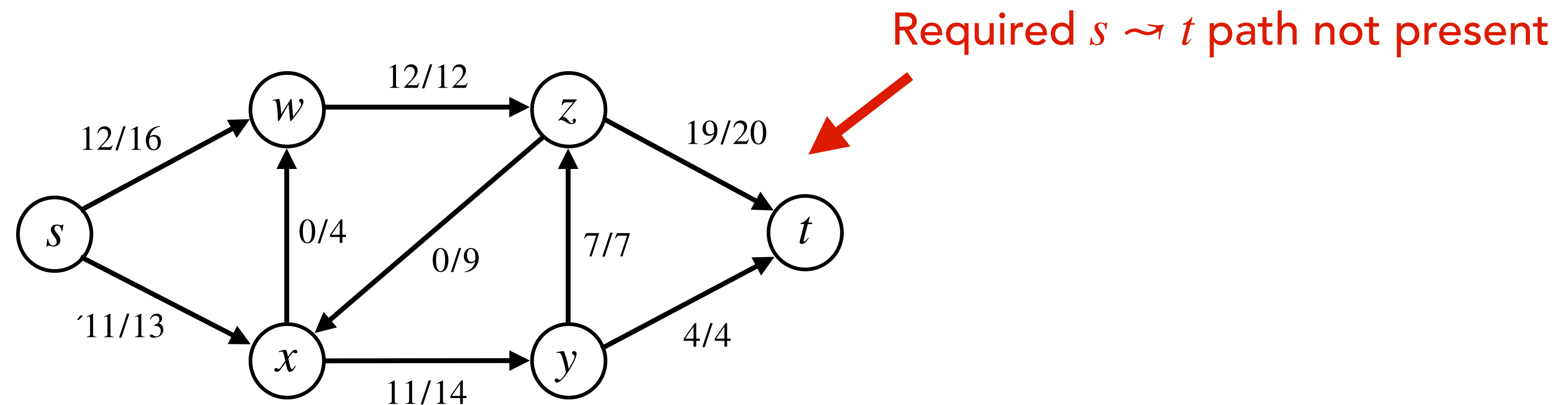


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Example:

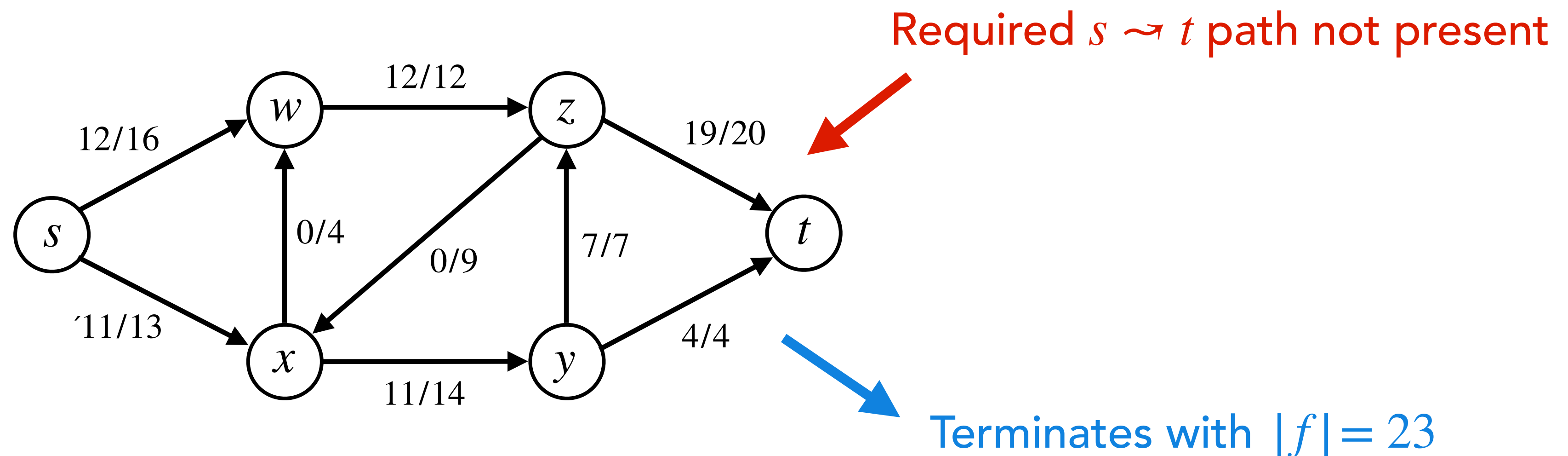


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Example:



Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

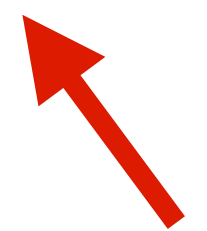


Is the algorithm really correct?

Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f



Is the algorithm really correct? No.

Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

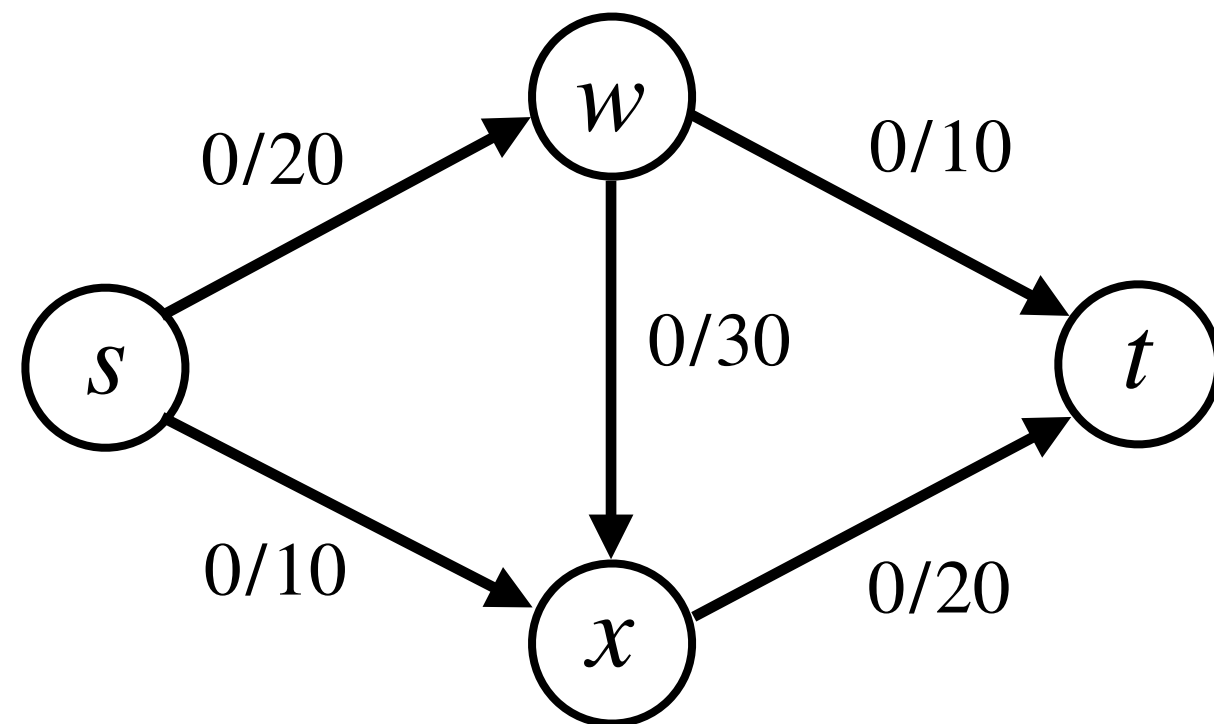
1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Counter-Example:

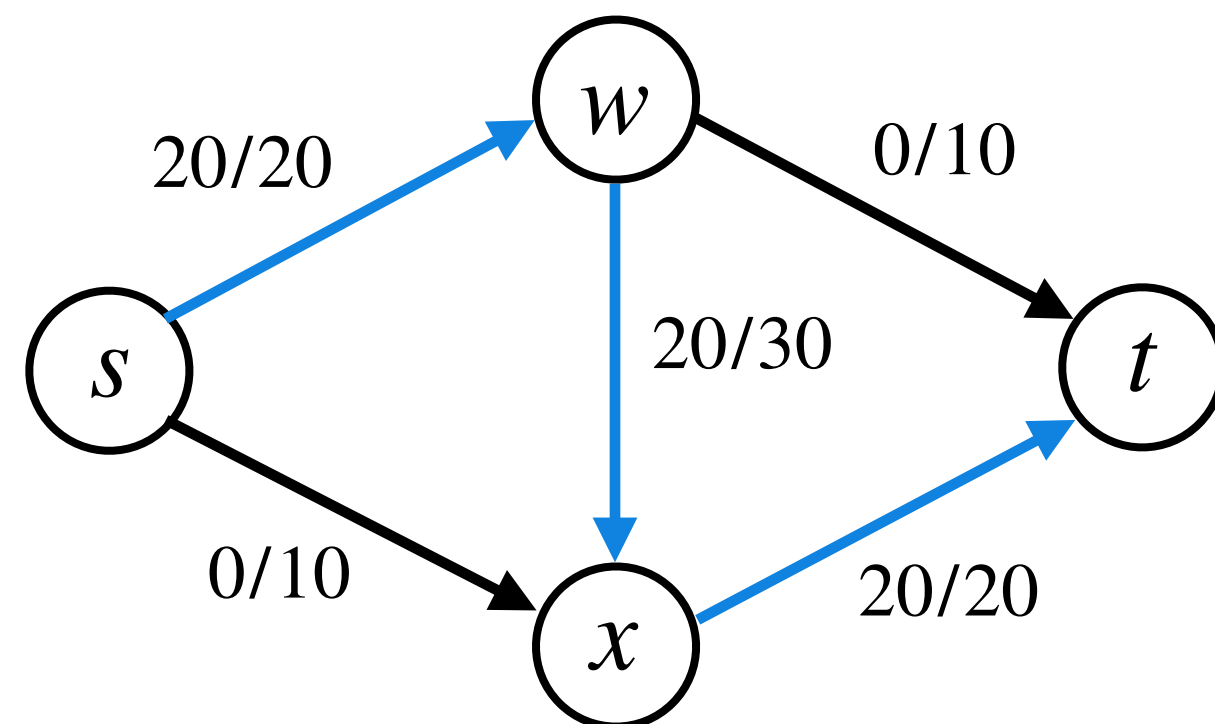


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Counter-Example:

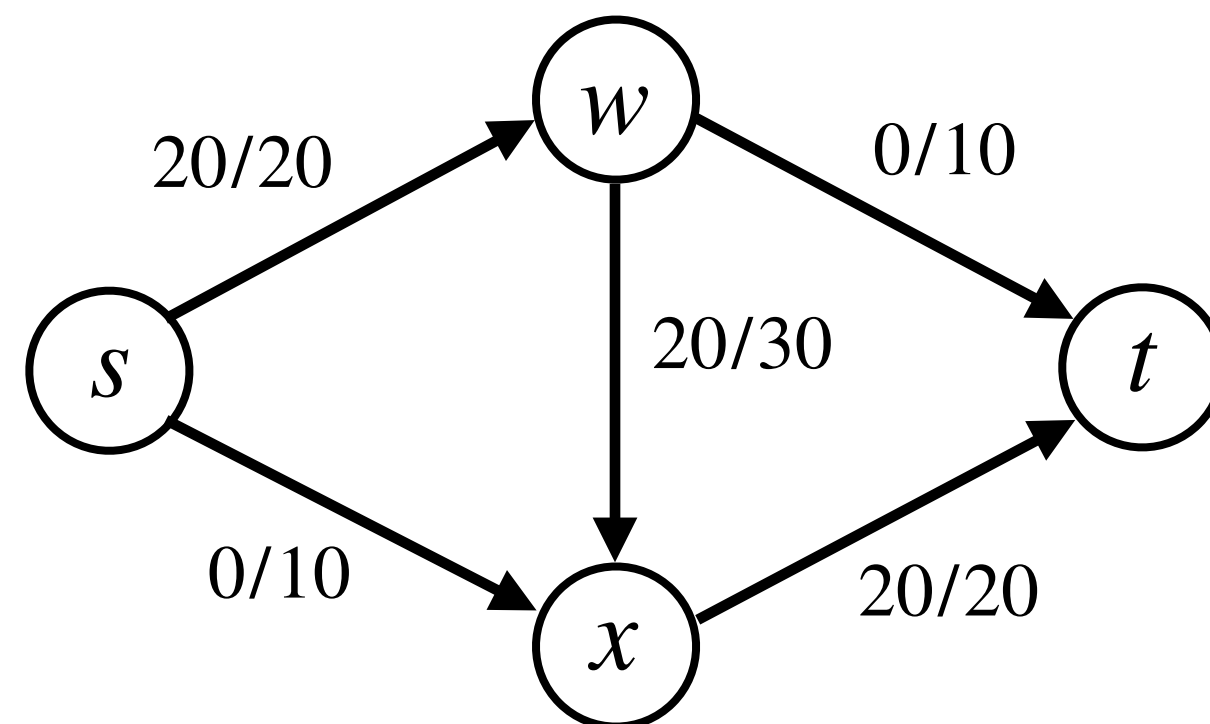


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Counter-Example:

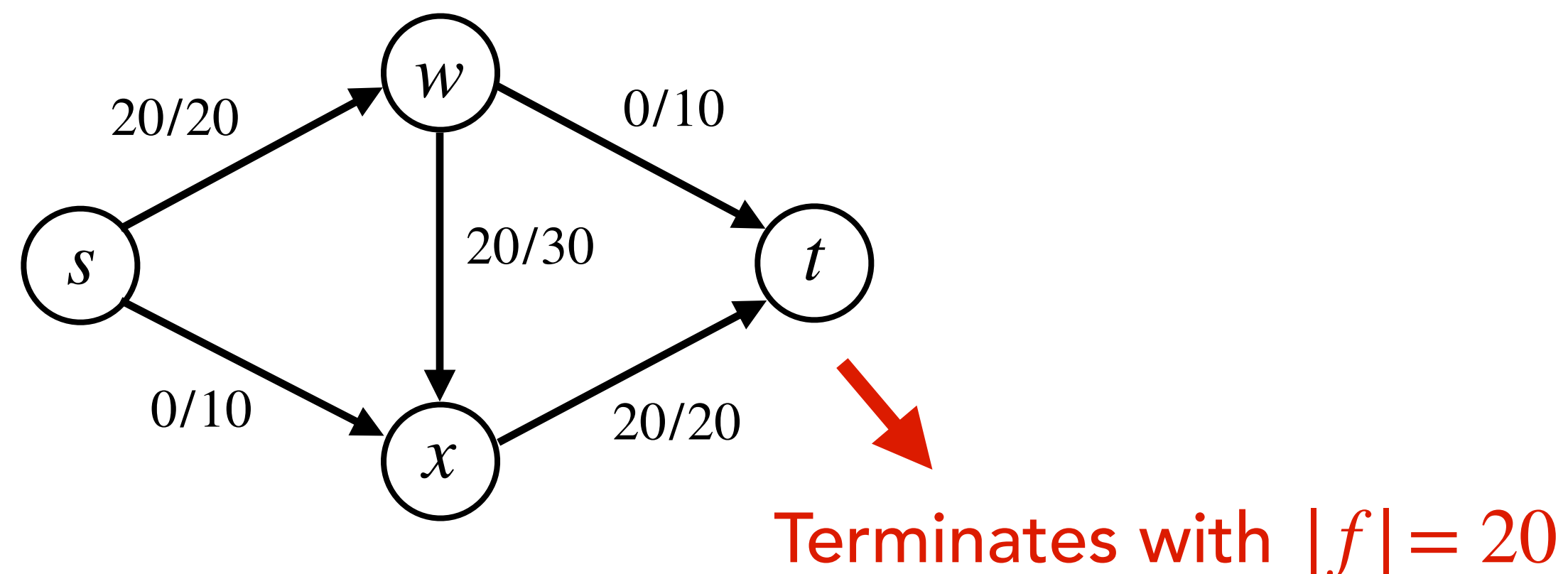


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Counter-Example:

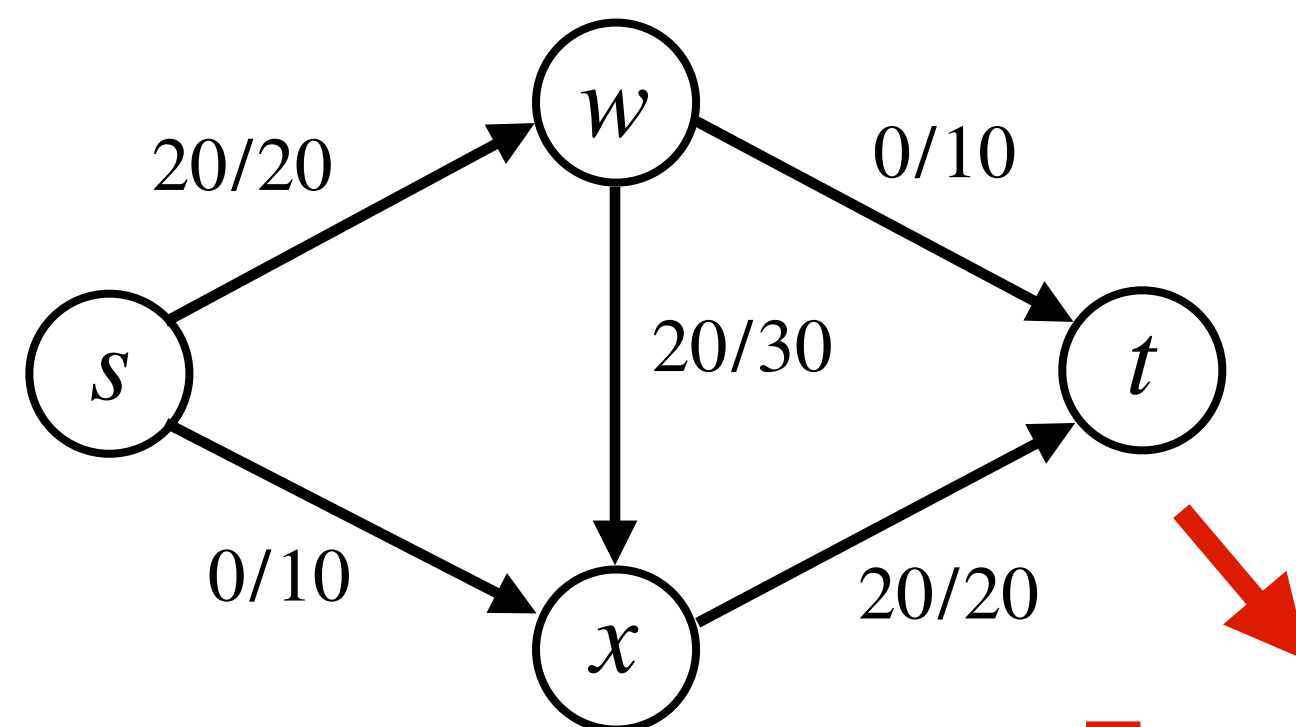


Finding Max Flow: An attempt

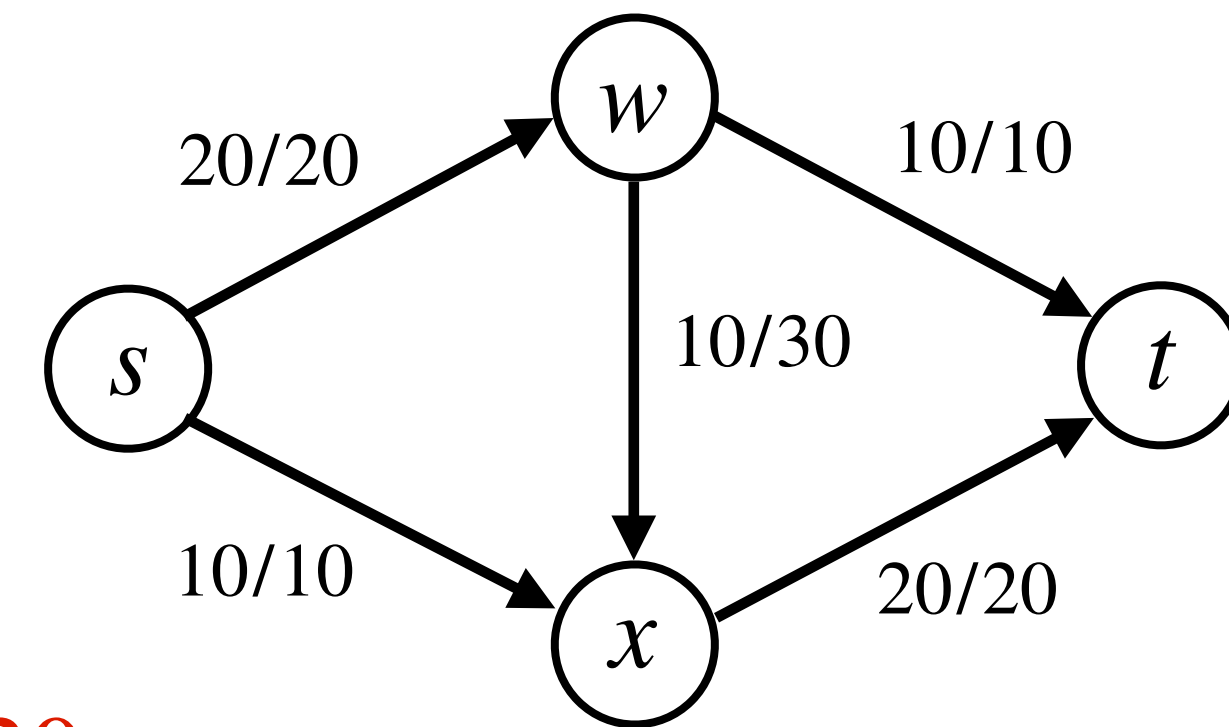
Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Counter-Example:



Terminates with $|f| = 20$

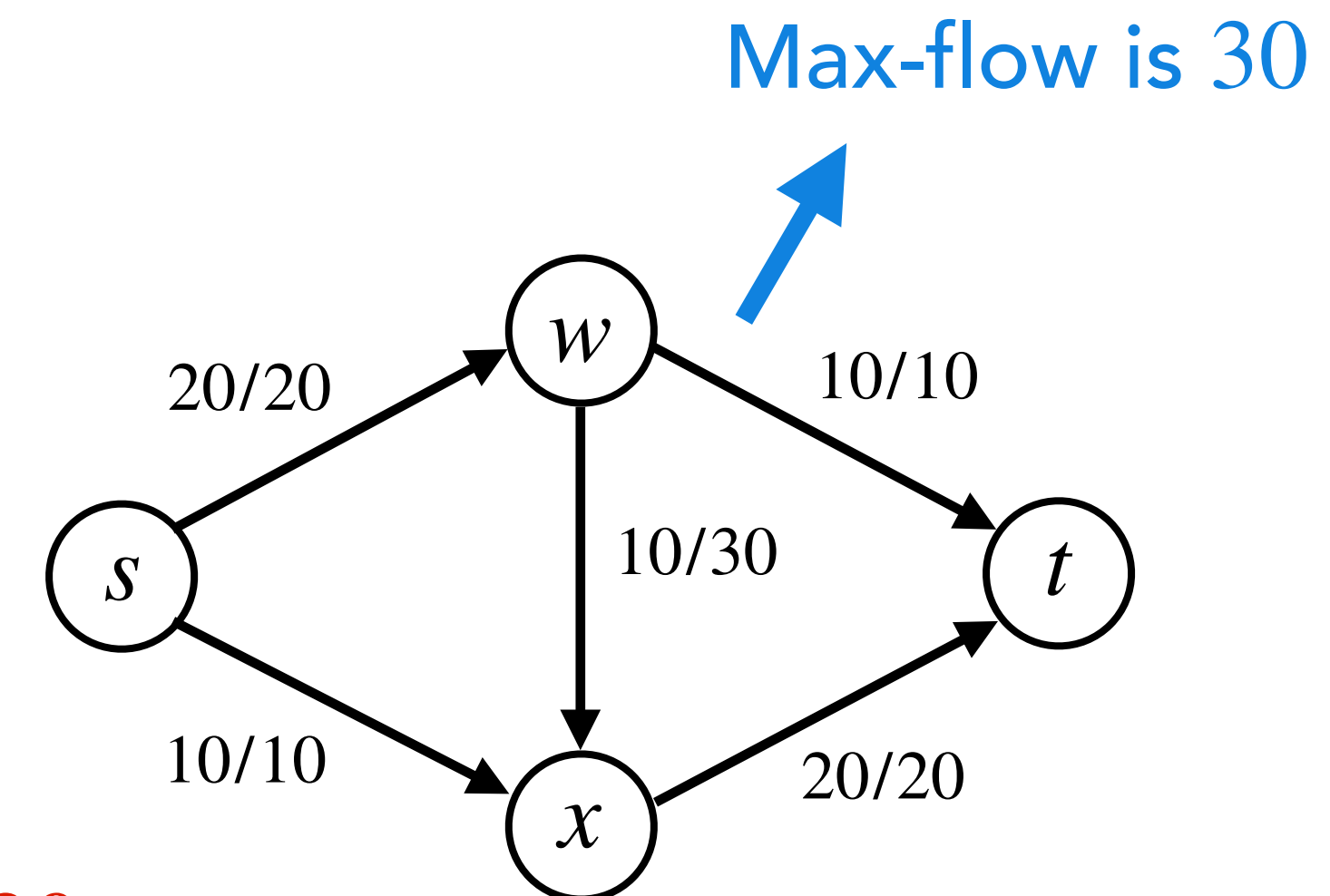
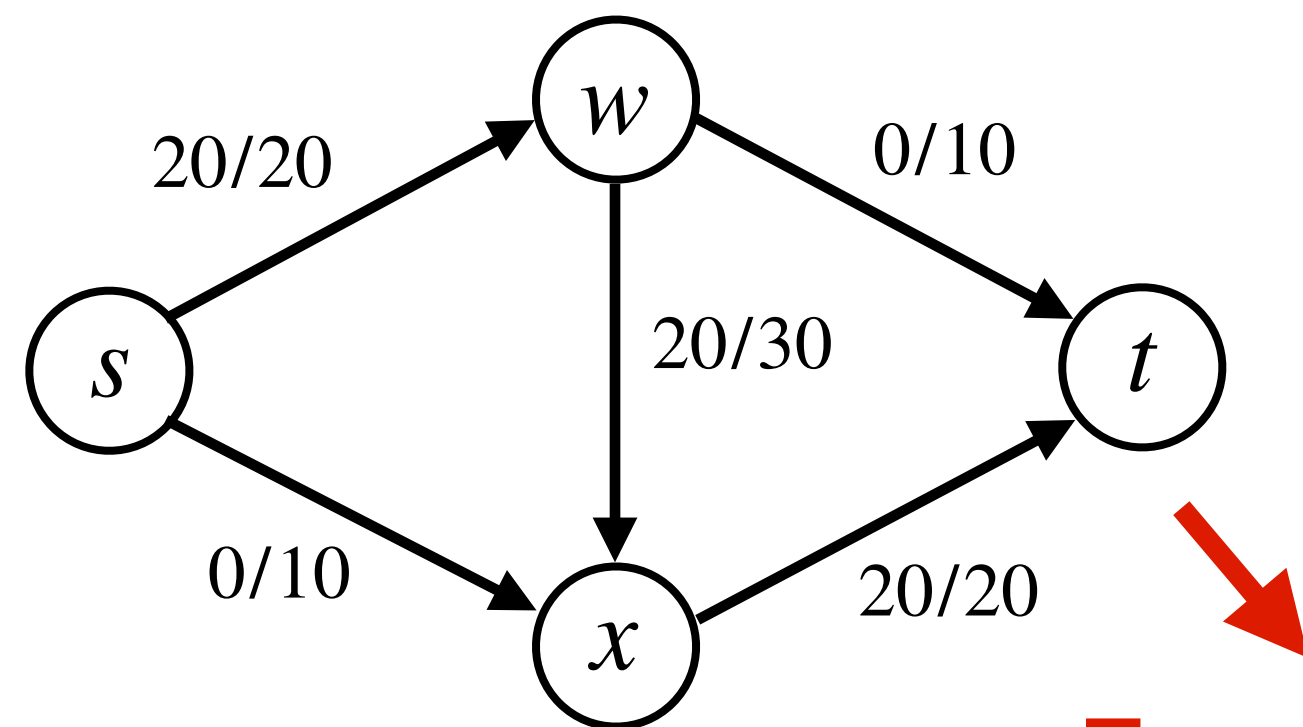


Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Counter-Example:



Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Observation: The above algorithm **never decreases** flow along any edge.

Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Observation: The above algorithm **never decreases** flow along any edge.

Possible Fix: May be we should allow **decreasing/redistributing** flows.

Finding Max Flow: An attempt

Naive-Max-Flow(G, s, t):

1. Start with flow $f = 0$ for every edge
2. Find an $s \rightsquigarrow t$ path P where every edge has $f < c$
3. Augment flow f with the least $c - f$ on P along P
4. Keep repeating line 2 & 3 until you get stuck
5. Return f

Observation: The above algorithm **never decreases** flow along any edge.

Possible Fix: May be we should allow **decreasing/redistributing** flows.



Need to learn a new structure for that!

Residual Networks

Residual Networks

Defn: For a given flow network $G = (V, E)$ and flow f , residual network $G_f = (V, E_f)$ is a flow

Residual Networks

Defn: For a given flow network $G = (V, E)$ and flow f , residual network $G_f = (V, E_f)$ is a flow network, where E_f contains:

Residual Networks

Defn: For a given flow network $G = (V, E)$ and flow f , residual network $G_f = (V, E_f)$ is a flow network, where E_f contains:

- **Forward edges:** Edge (u, v) with capacity $c_f(u, v) = c(u, v) - f(u, v) > 0$

Residual Networks

Defn: For a given flow network $G = (V, E)$ and flow f , residual network $G_f = (V, E_f)$ is a flow network, where E_f contains:

- **Forward edges:** Edge (u, v) with capacity $c_f(u, v) = c(u, v) - f(u, v) > 0$
- **Backward edges:** Edge (v, u) with capacity $c_f(v, u) = f(u, v) > 0$

Residual Networks

Defn: For a given flow network $G = (V, E)$ and flow f , residual network $G_f = (V, E_f)$ is a flow network, where E_f contains:

- **Forward edges:** Edge (u, v) with capacity $c_f(u, v) = c(u, v) - f(u, v) > 0$
- **Backward edges:** Edge (v, u) with capacity $c_f(v, u) = f(u, v) > 0$

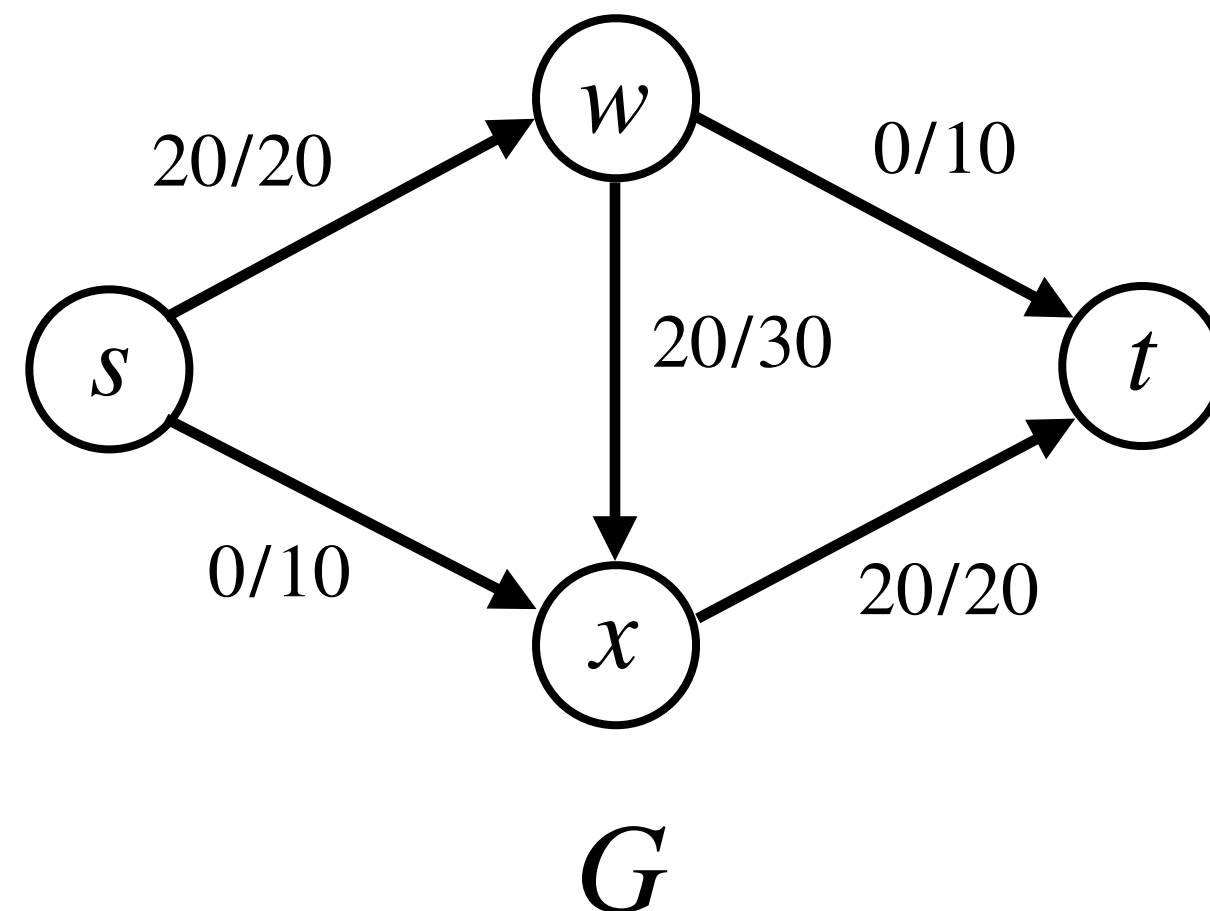
Example:

Residual Networks

Defn: For a given flow network $G = (V, E)$ and flow f , residual network $G_f = (V, E_f)$ is a flow network, where E_f contains:

- **Forward edges:** Edge (u, v) with capacity $c_f(u, v) = c(u, v) - f(u, v) > 0$
- **Backward edges:** Edge (v, u) with capacity $c_f(v, u) = f(u, v) > 0$

Example:

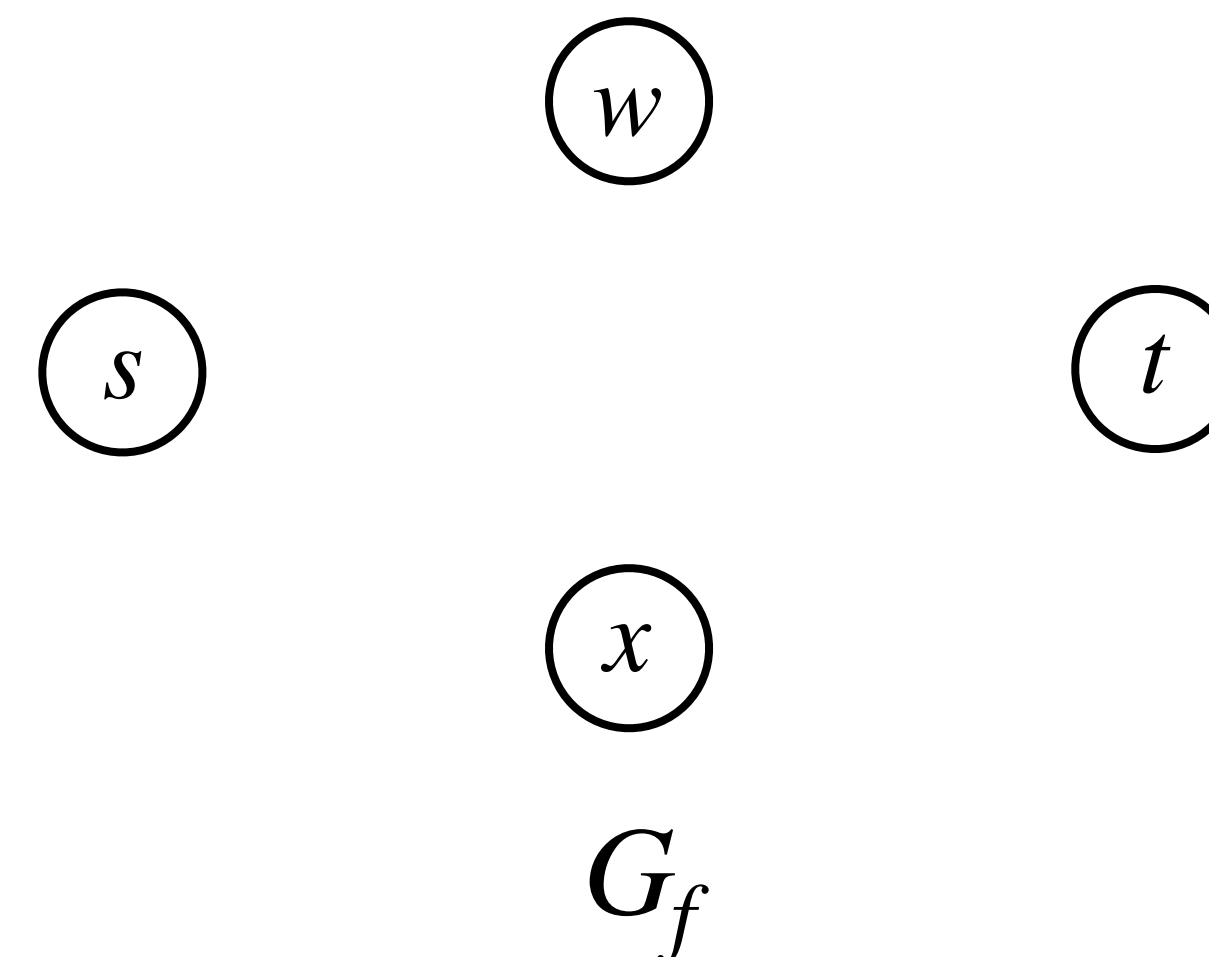
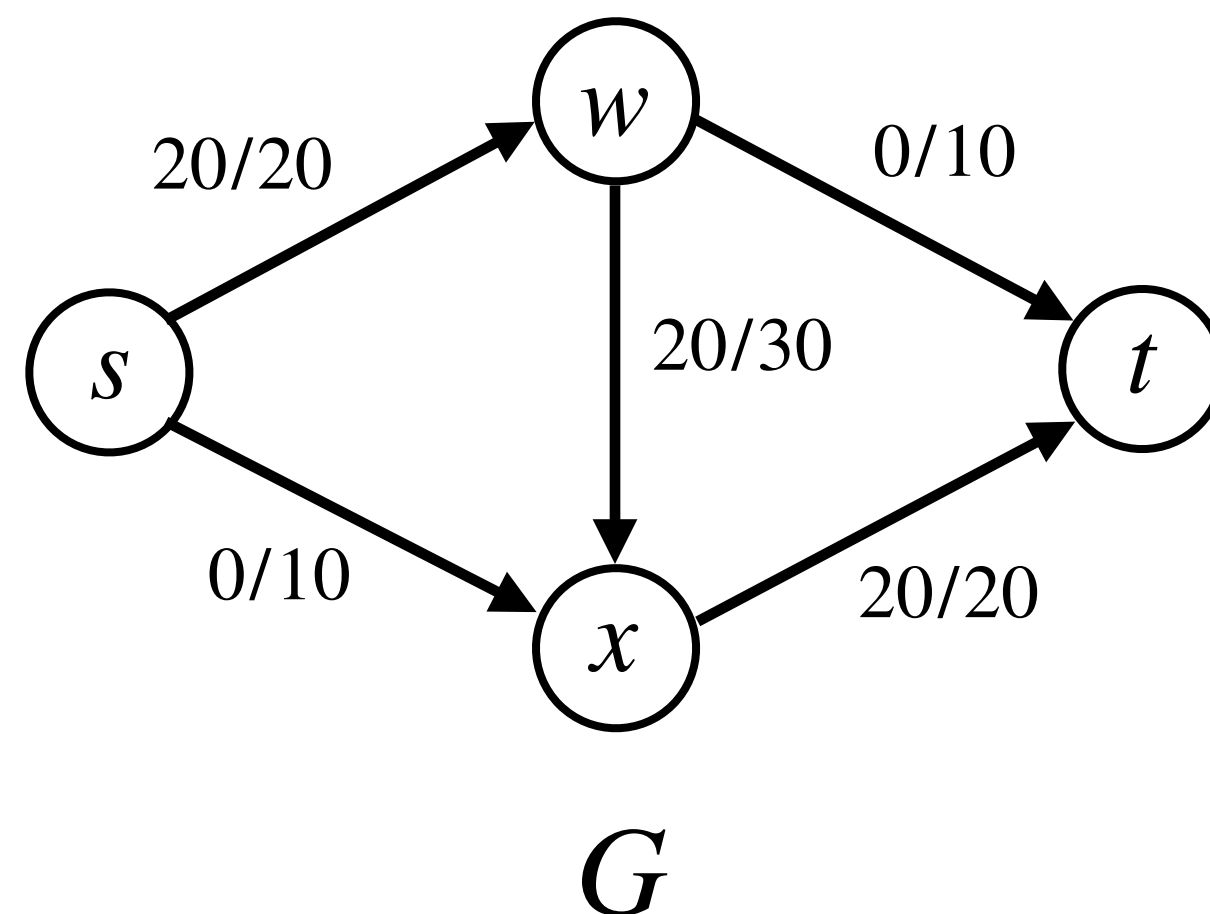


Residual Networks

Defn: For a given flow network $G = (V, E)$ and flow f , residual network $G_f = (V, E_f)$ is a flow network, where E_f contains:

- **Forward edges:** Edge (u, v) with capacity $c_f(u, v) = c(u, v) - f(u, v) > 0$
- **Backward edges:** Edge (v, u) with capacity $c_f(v, u) = f(u, v) > 0$

Example:

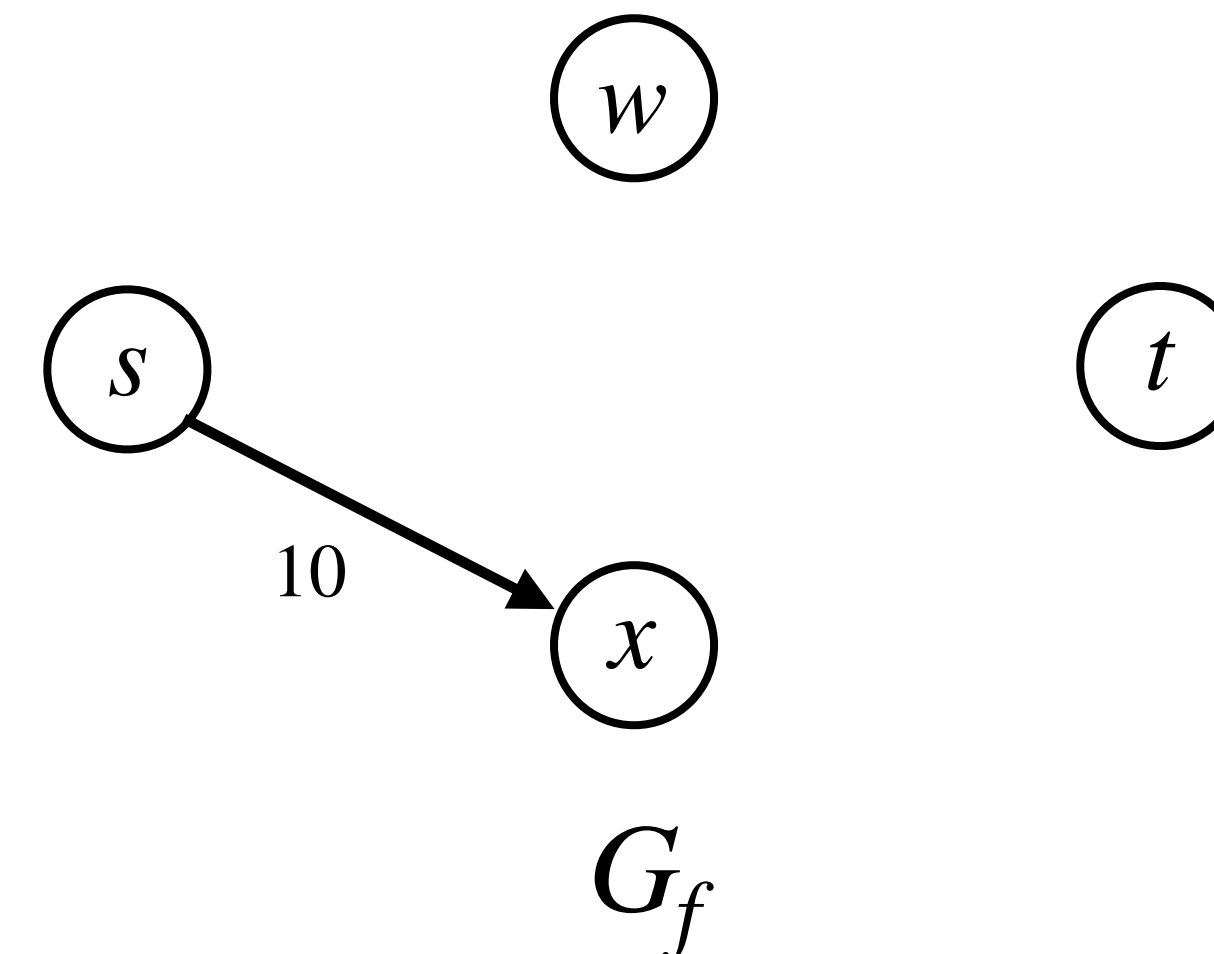
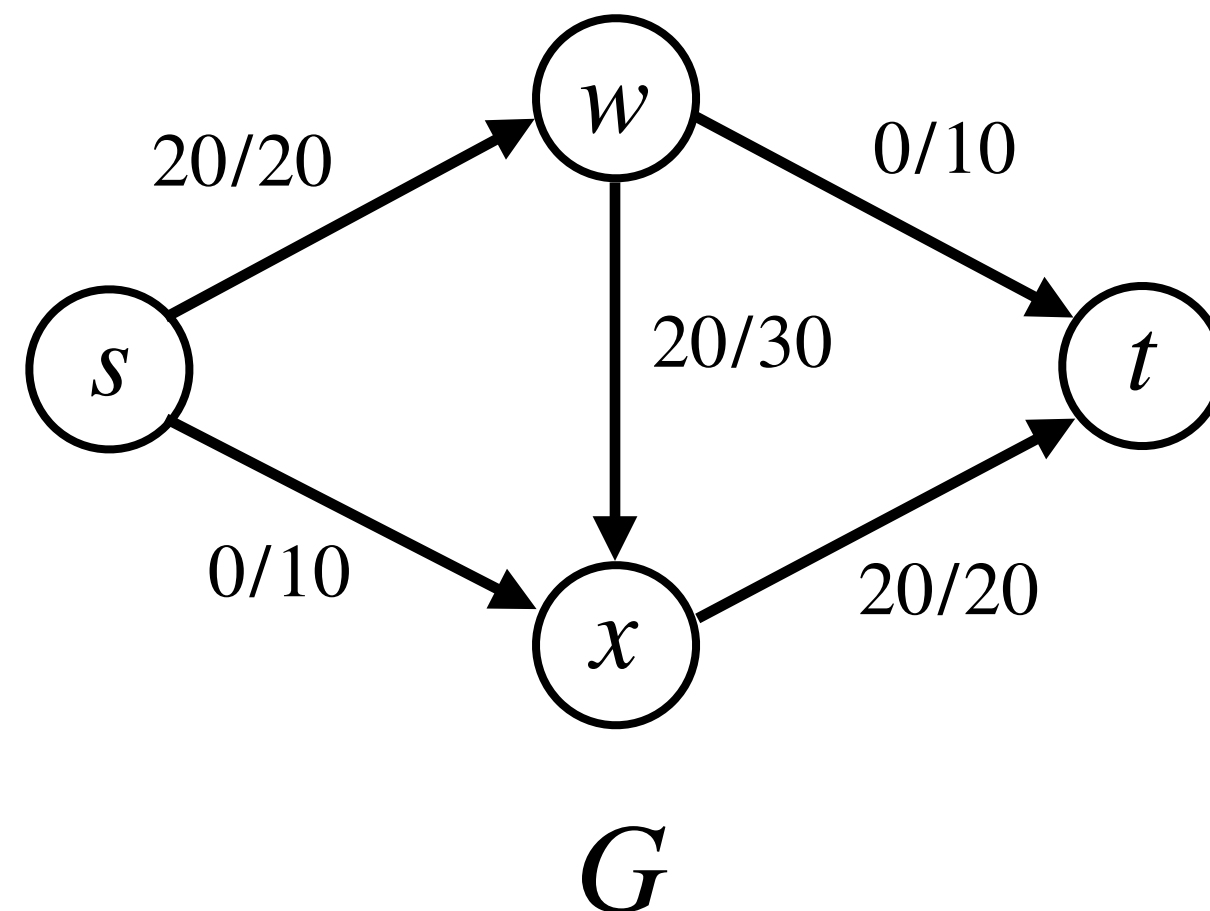


Residual Networks

Defn: For a given flow network $G = (V, E)$ and flow f , residual network $G_f = (V, E_f)$ is a flow network, where E_f contains:

- **Forward edges:** Edge (u, v) with capacity $c_f(u, v) = c(u, v) - f(u, v) > 0$
- **Backward edges:** Edge (v, u) with capacity $c_f(v, u) = f(u, v) > 0$

Example:

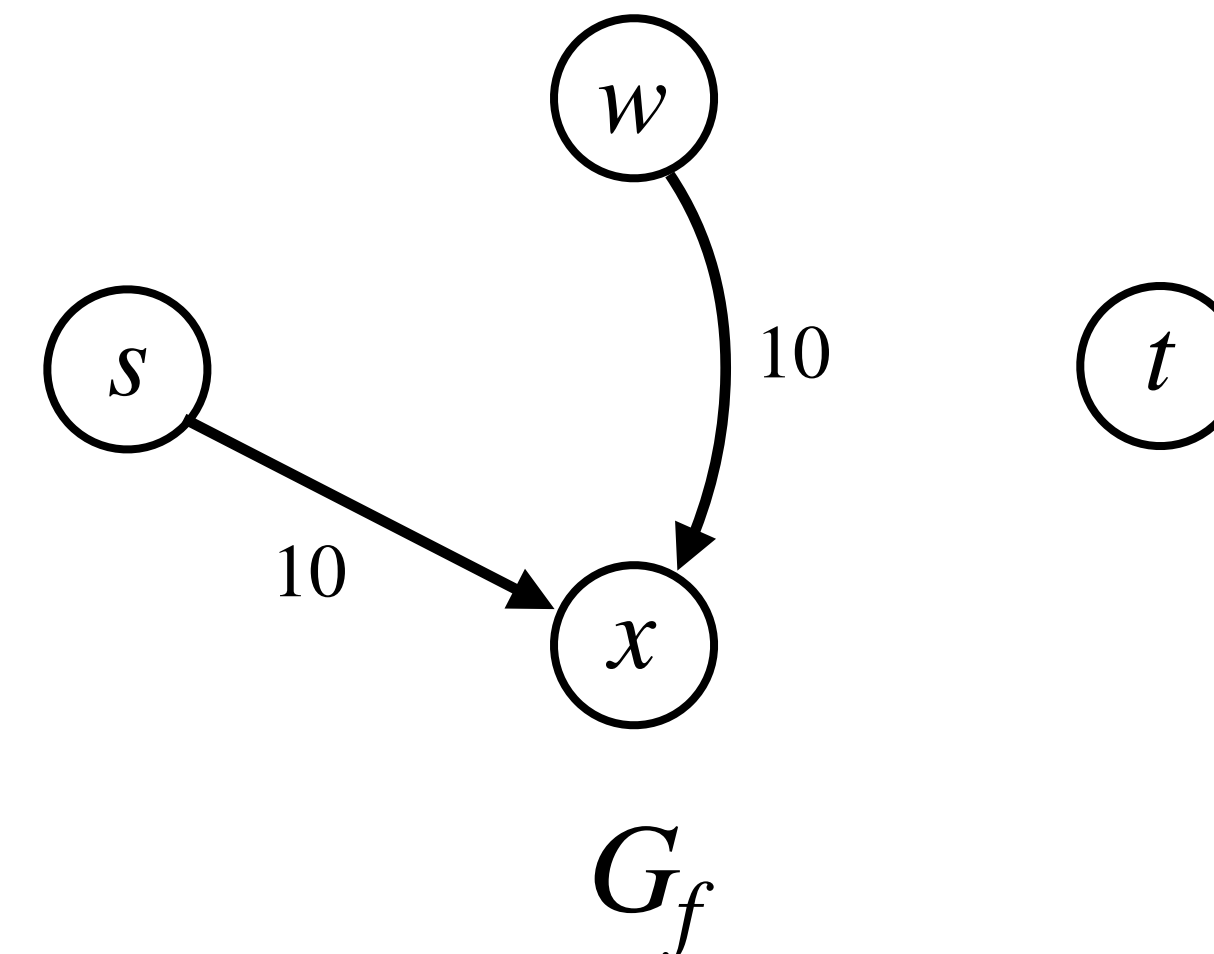
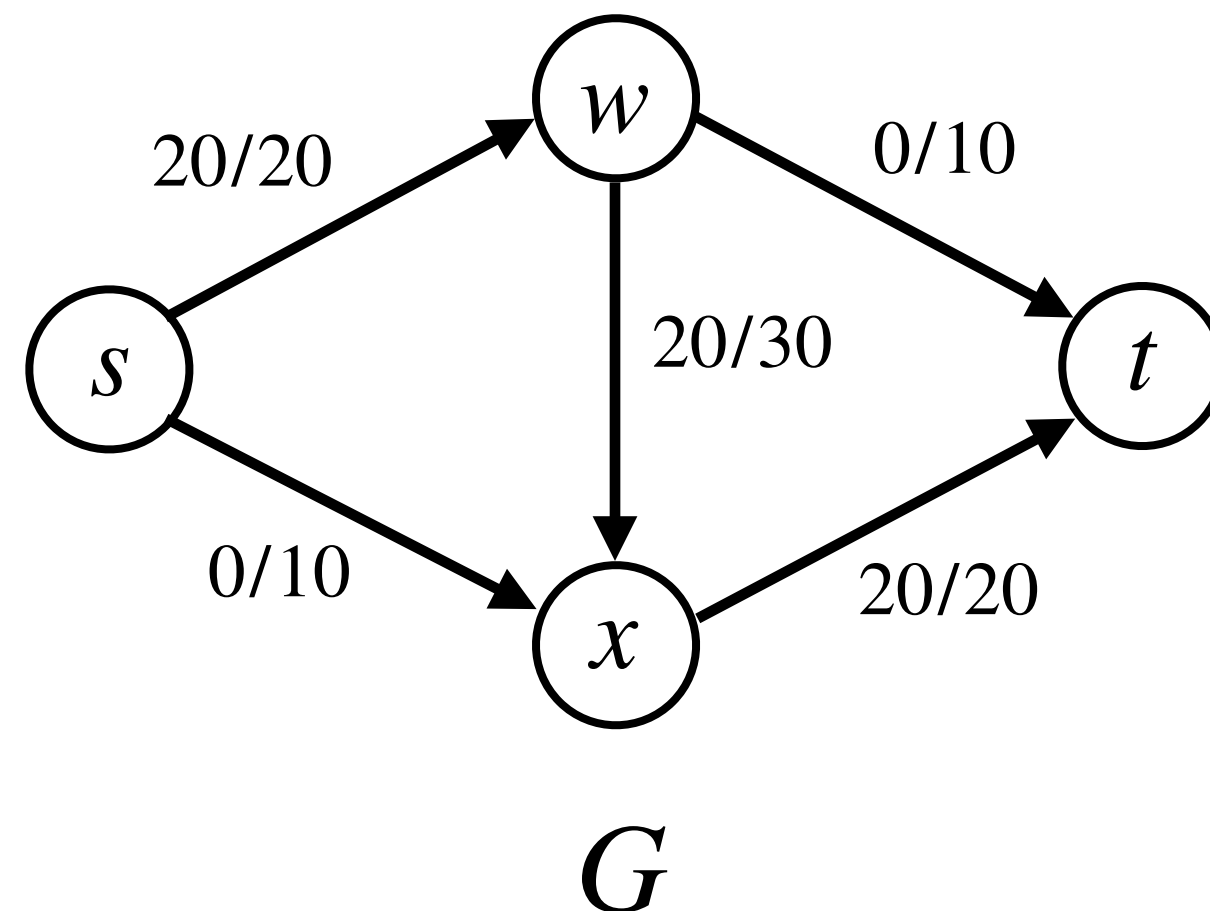


Residual Networks

Defn: For a given flow network $G = (V, E)$ and flow f , residual network $G_f = (V, E_f)$ is a flow network, where E_f contains:

- **Forward edges:** Edge (u, v) with capacity $c_f(u, v) = c(u, v) - f(u, v) > 0$
- **Backward edges:** Edge (v, u) with capacity $c_f(v, u) = f(u, v) > 0$

Example:

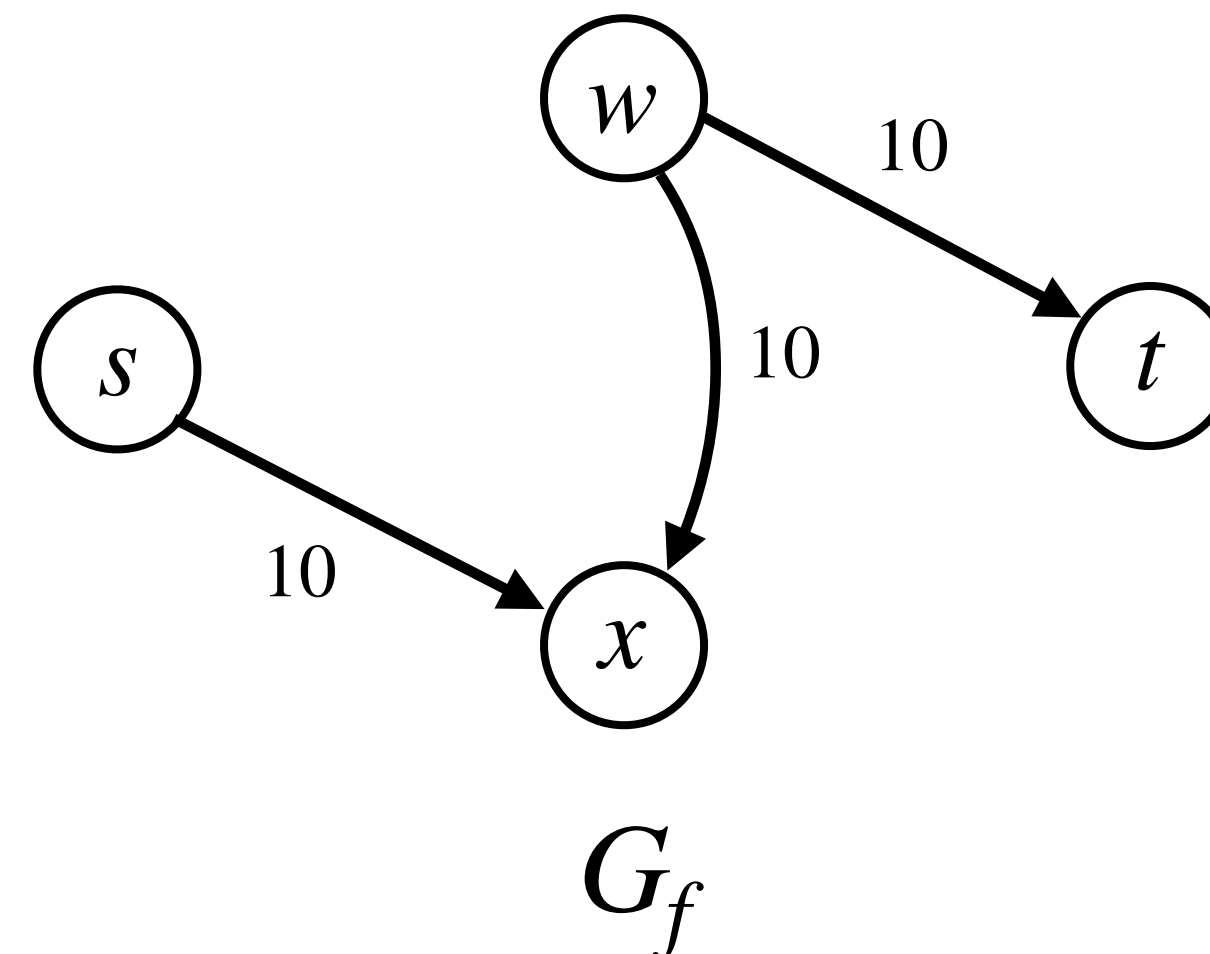
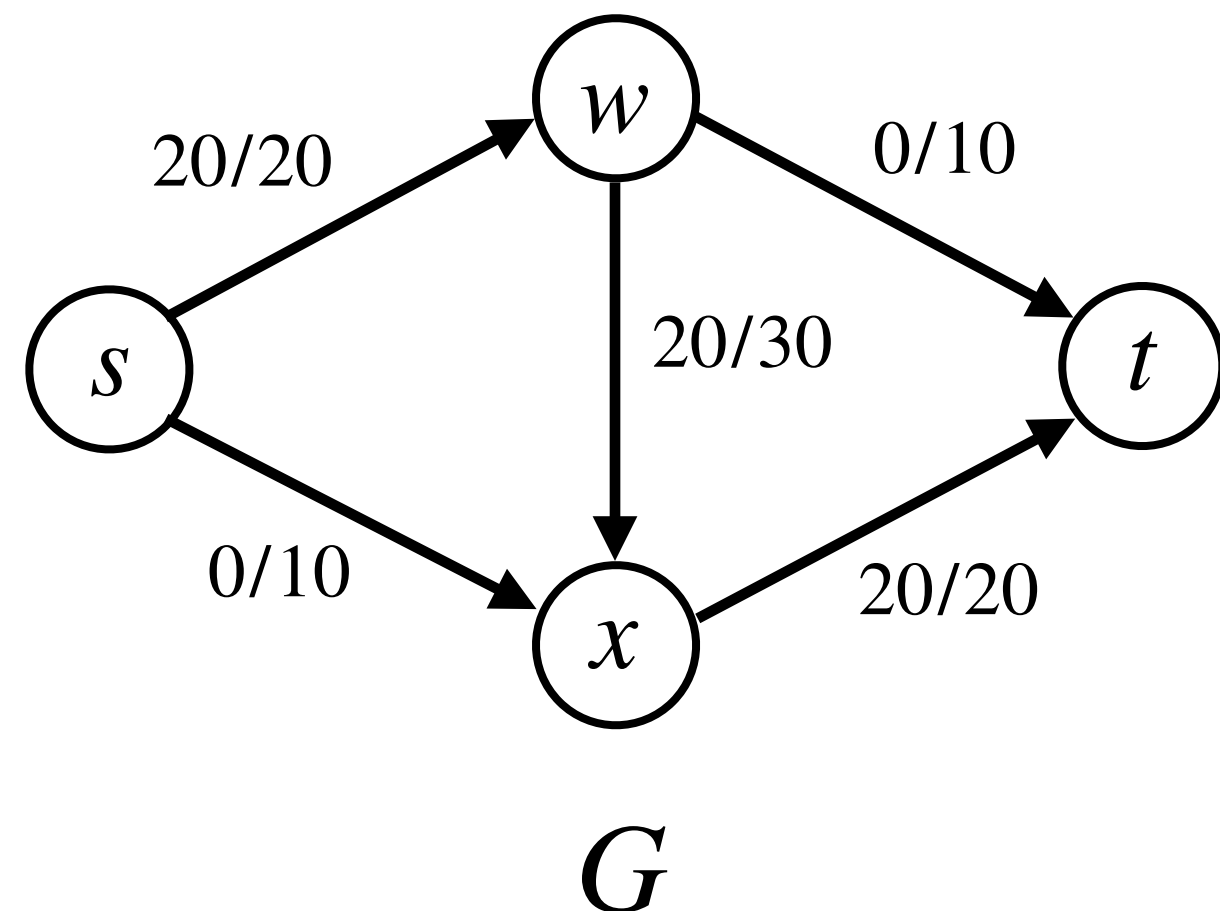


Residual Networks

Defn: For a given flow network $G = (V, E)$ and flow f , residual network $G_f = (V, E_f)$ is a flow network, where E_f contains:

- **Forward edges:** Edge (u, v) with capacity $c_f(u, v) = c(u, v) - f(u, v) > 0$
- **Backward edges:** Edge (v, u) with capacity $c_f(v, u) = f(u, v) > 0$

Example:

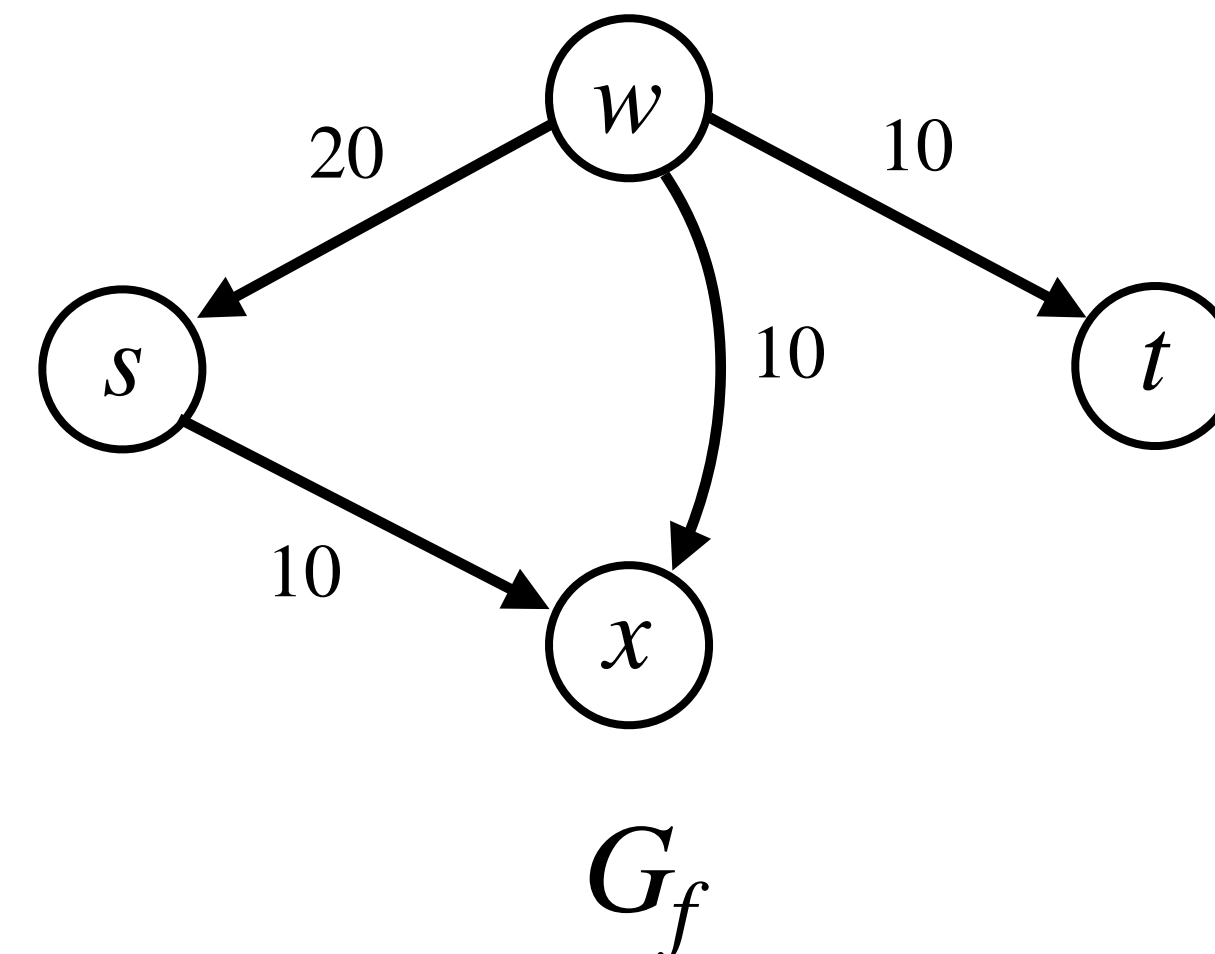
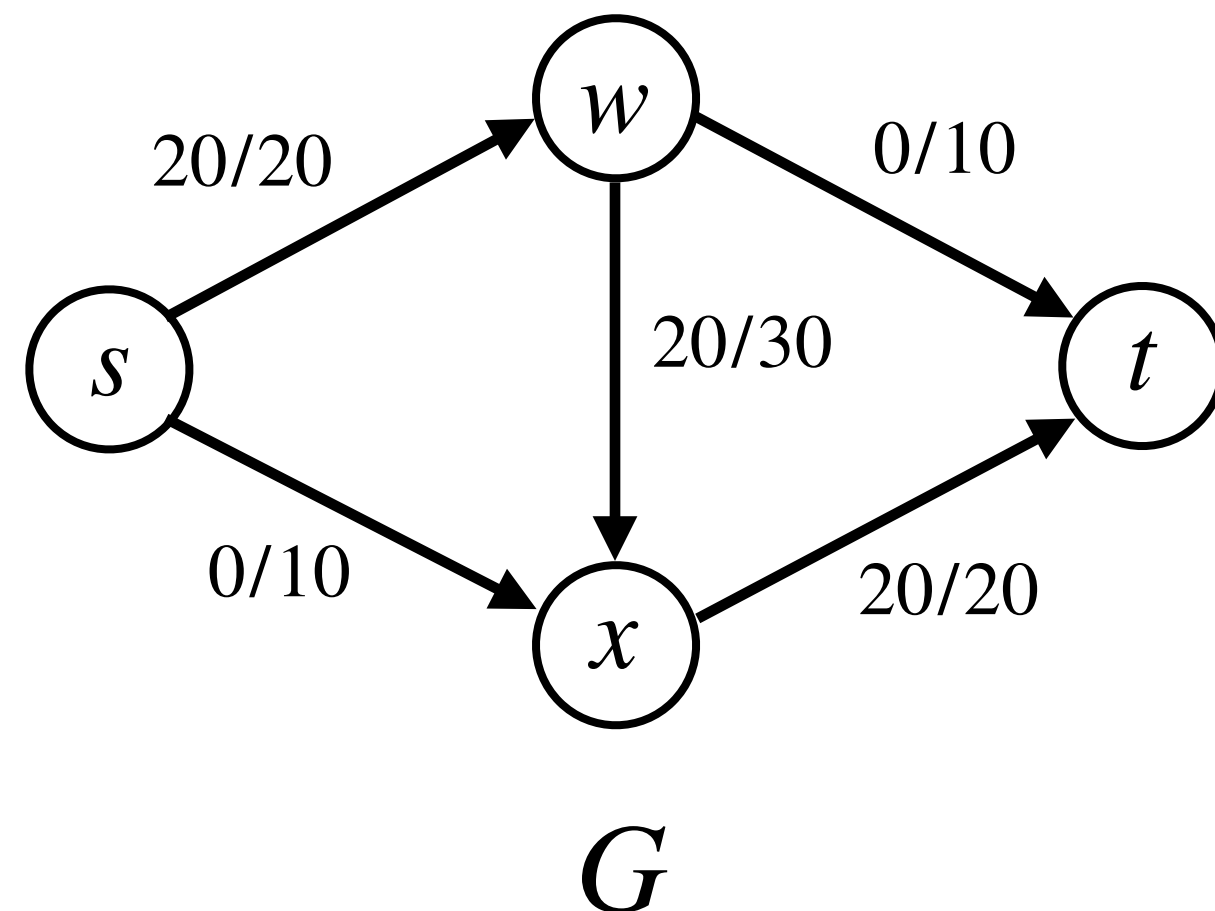


Residual Networks

Defn: For a given flow network $G = (V, E)$ and flow f , residual network $G_f = (V, E_f)$ is a flow network, where E_f contains:

- **Forward edges:** Edge (u, v) with capacity $c_f(u, v) = c(u, v) - f(u, v) > 0$
- **Backward edges:** Edge (v, u) with capacity $c_f(v, u) = f(u, v) > 0$

Example:

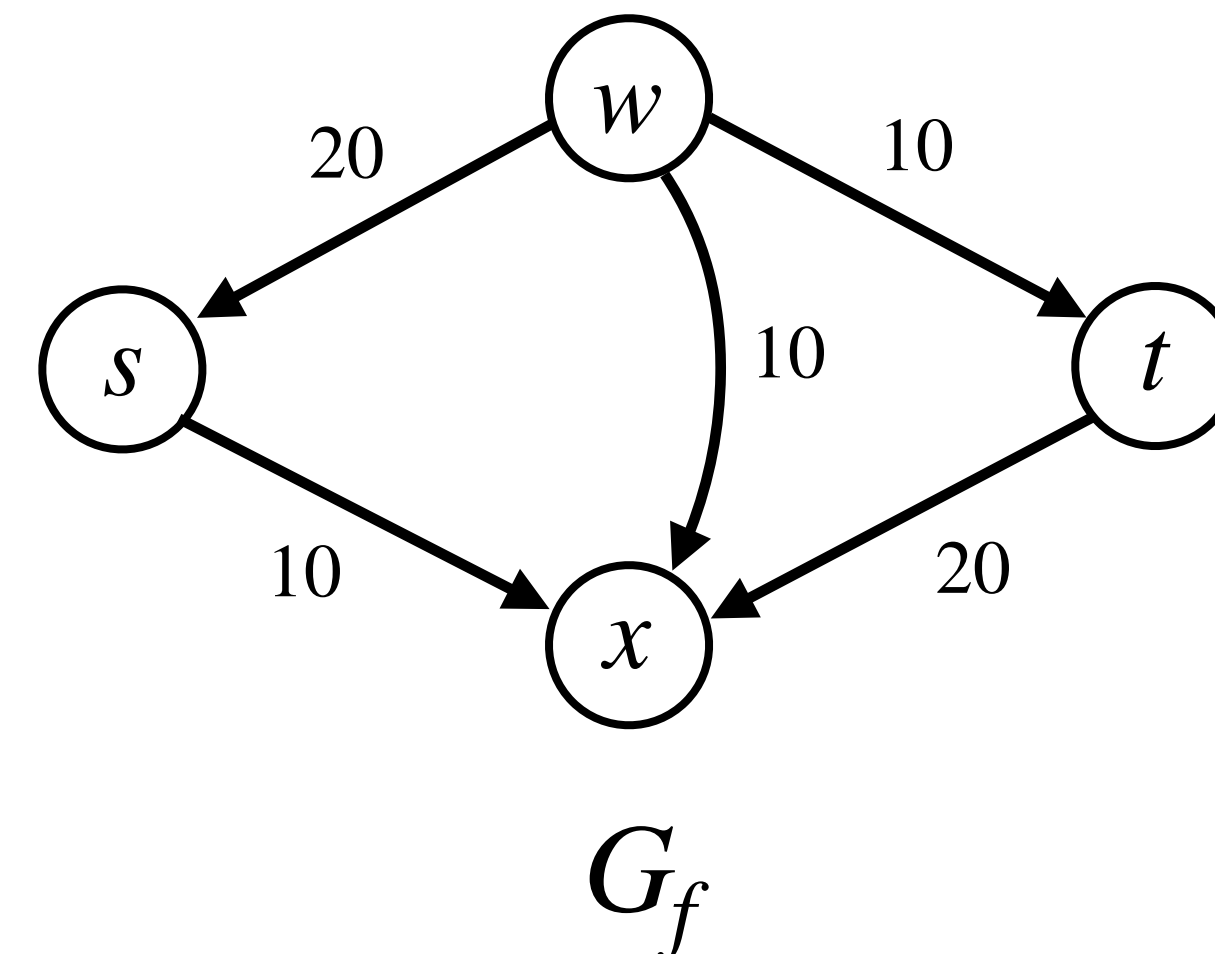
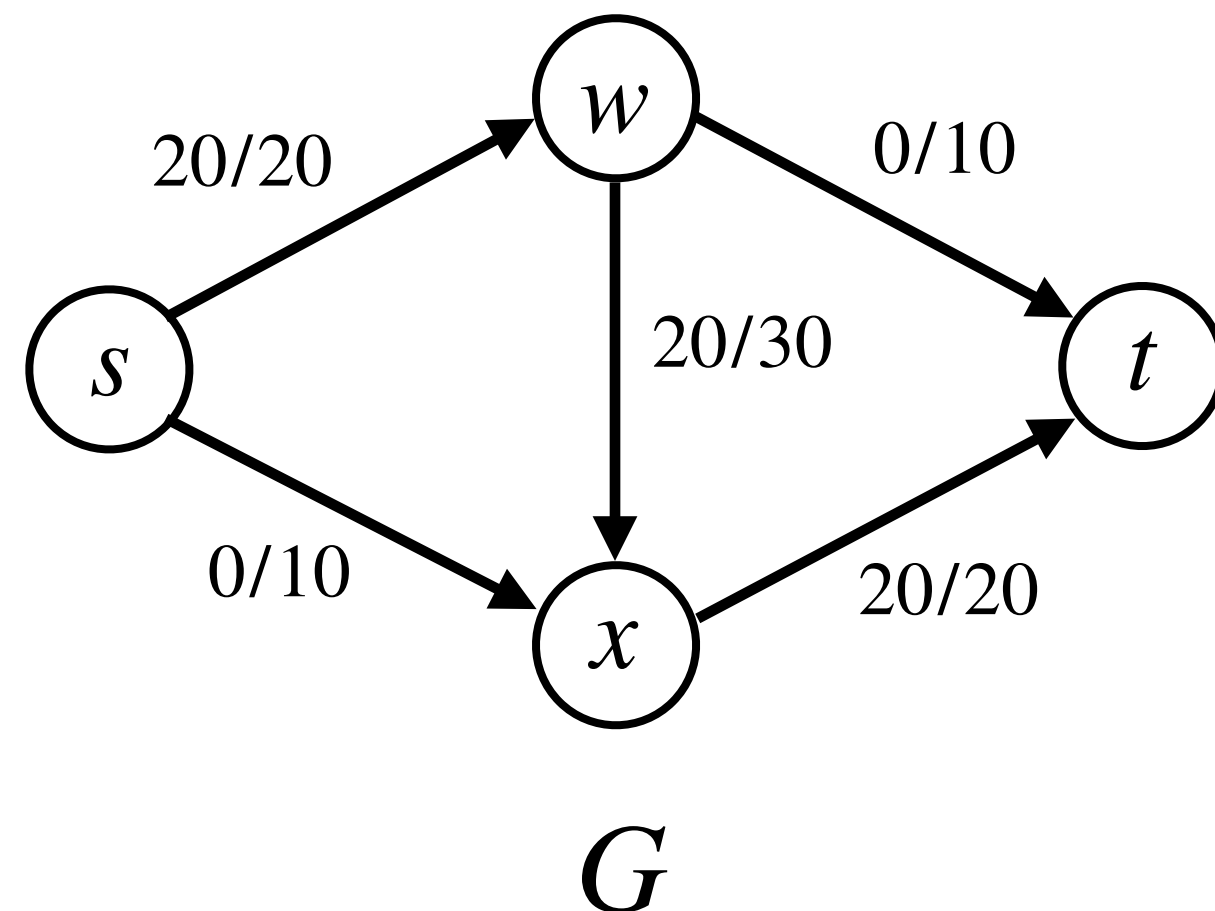


Residual Networks

Defn: For a given flow network $G = (V, E)$ and flow f , residual network $G_f = (V, E_f)$ is a flow network, where E_f contains:

- **Forward edges:** Edge (u, v) with capacity $c_f(u, v) = c(u, v) - f(u, v) > 0$
- **Backward edges:** Edge (v, u) with capacity $c_f(v, u) = f(u, v) > 0$

Example:

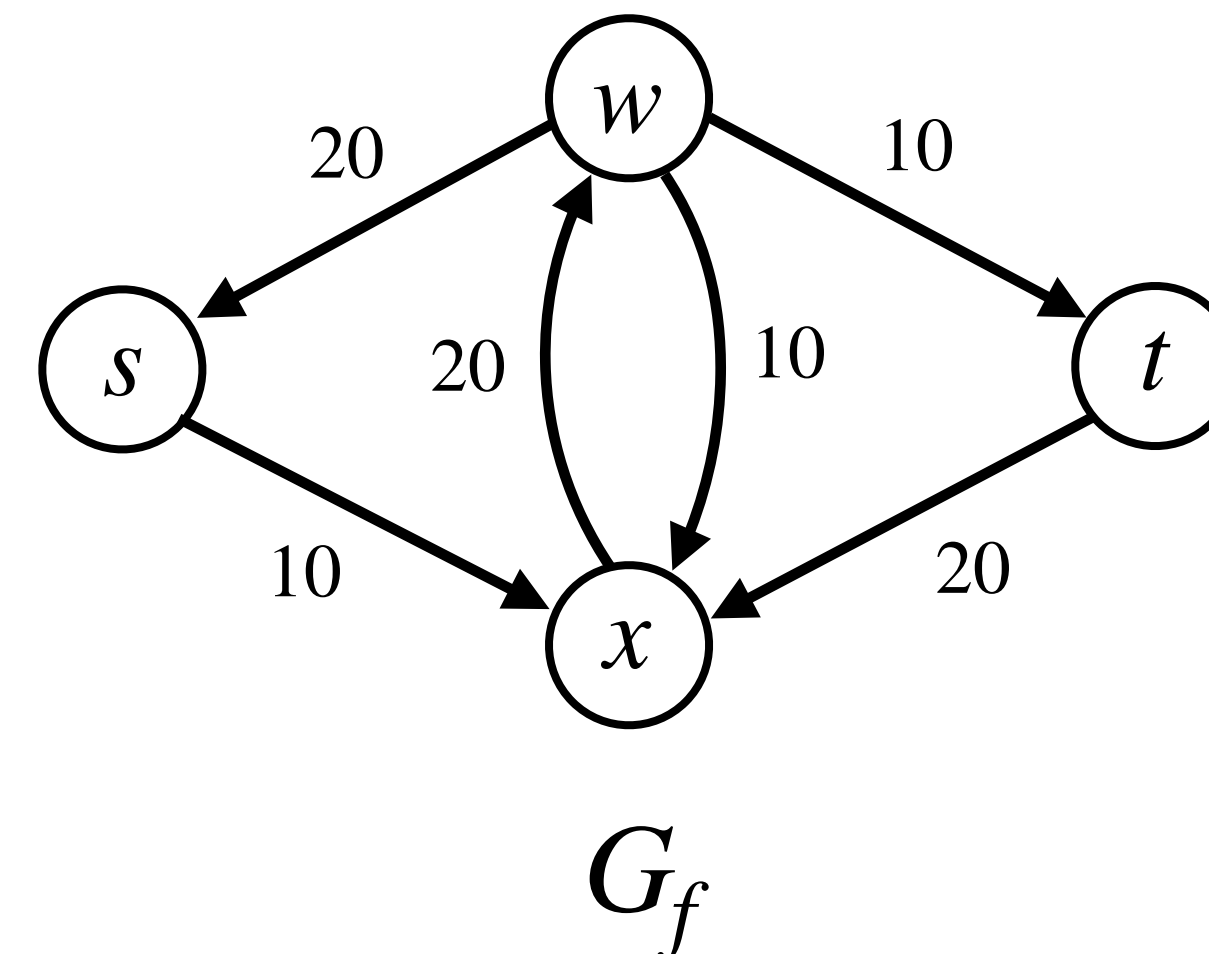
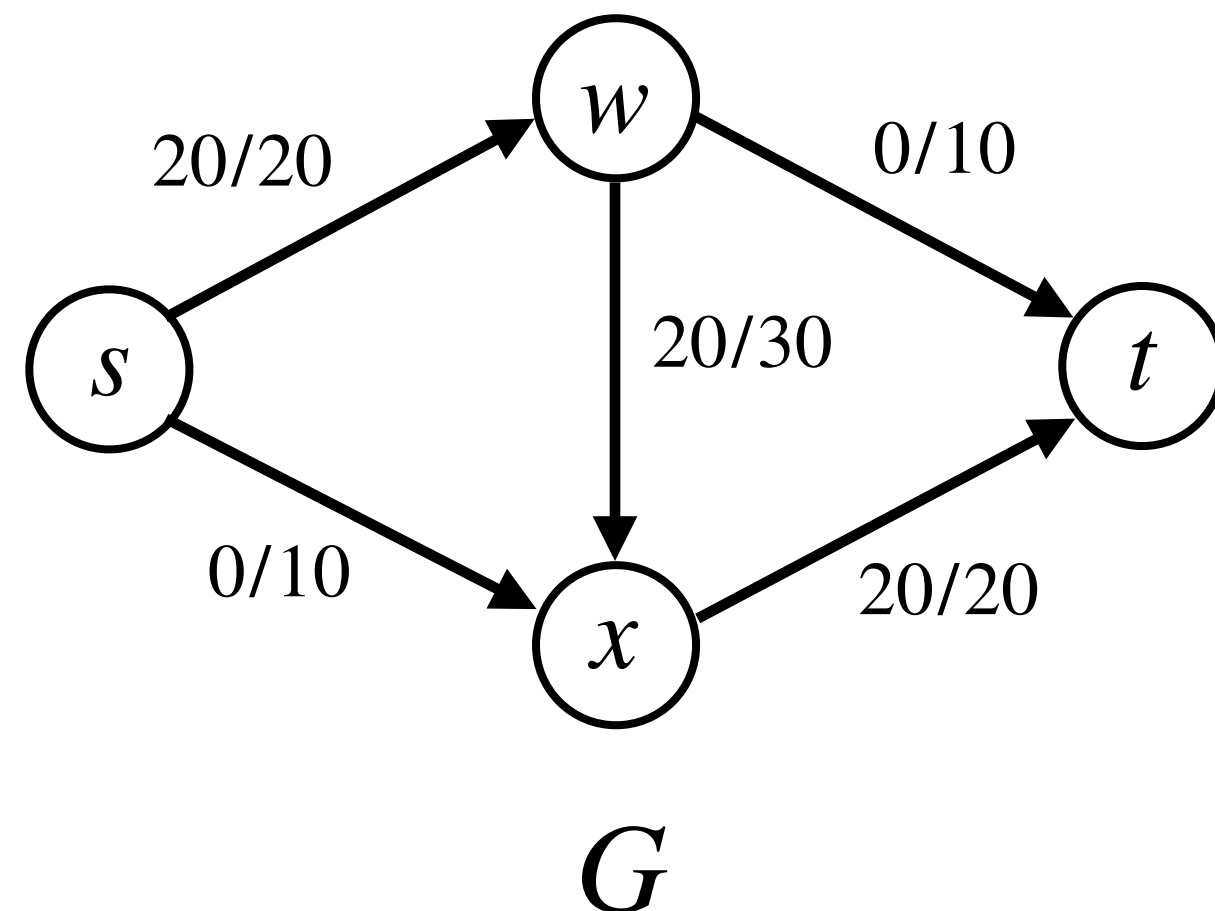


Residual Networks

Defn: For a given flow network $G = (V, E)$ and flow f , residual network $G_f = (V, E_f)$ is a flow network, where E_f contains:

- **Forward edges:** Edge (u, v) with capacity $c_f(u, v) = c(u, v) - f(u, v) > 0$
- **Backward edges:** Edge (v, u) with capacity $c_f(v, u) = f(u, v) > 0$

Example:

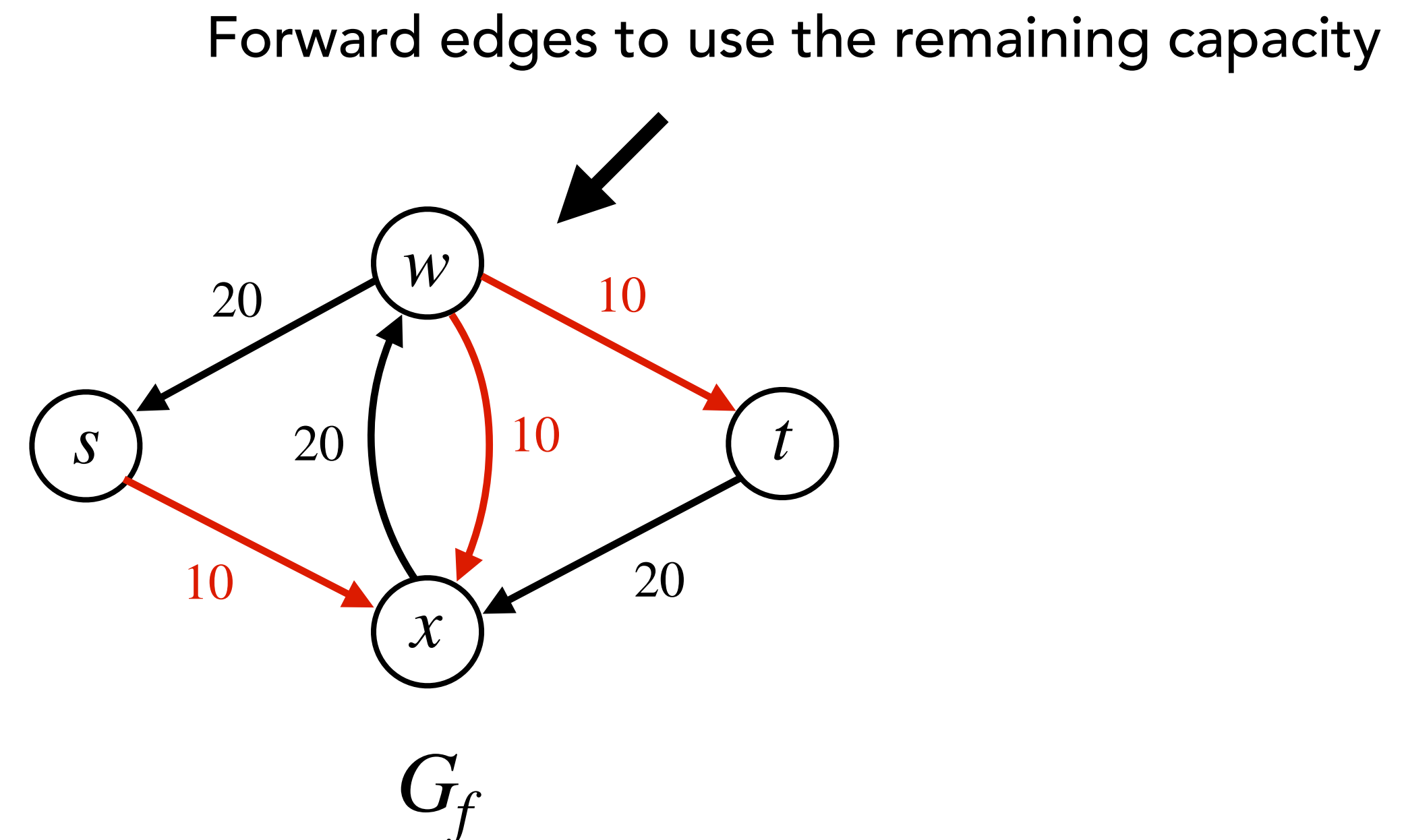
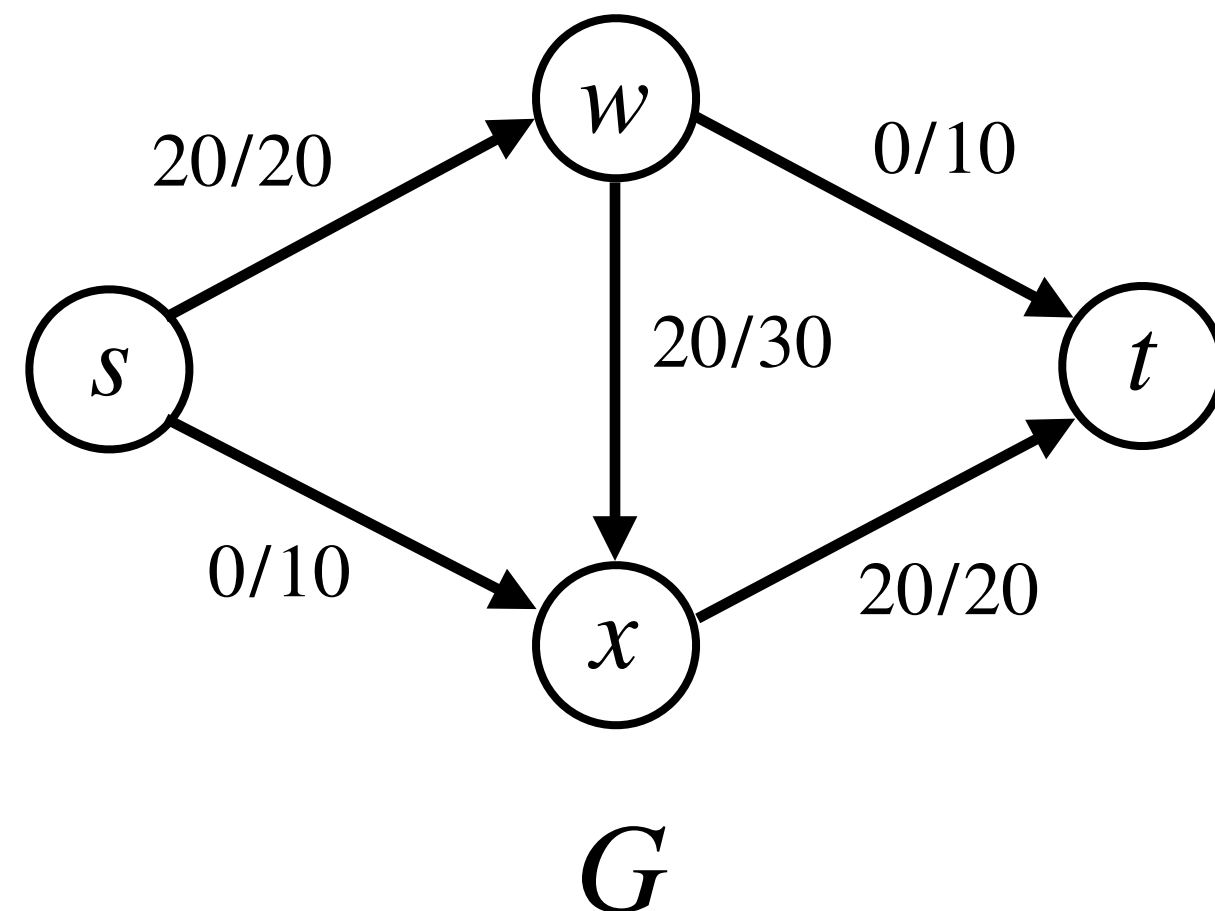


Residual Networks

Defn: For a given flow network $G = (V, E)$ and flow f , residual network $G_f = (V, E_f)$ is a flow network, where E_f contains:

- **Forward edges:** Edge (u, v) with capacity $c_f(u, v) = c(u, v) - f(u, v) > 0$
- **Backward edges:** Edge (v, u) with capacity $c_f(v, u) = f(u, v) > 0$

Example:

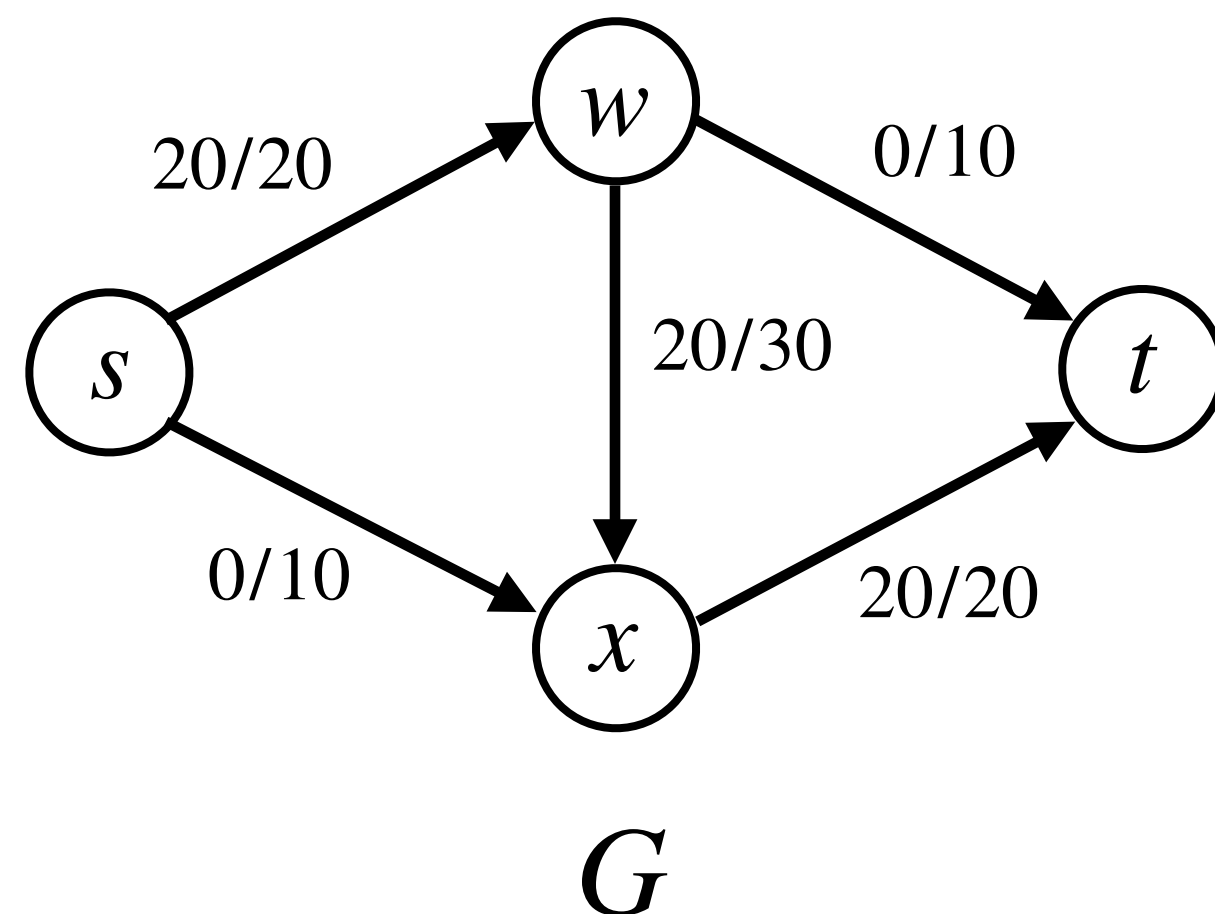


Residual Networks

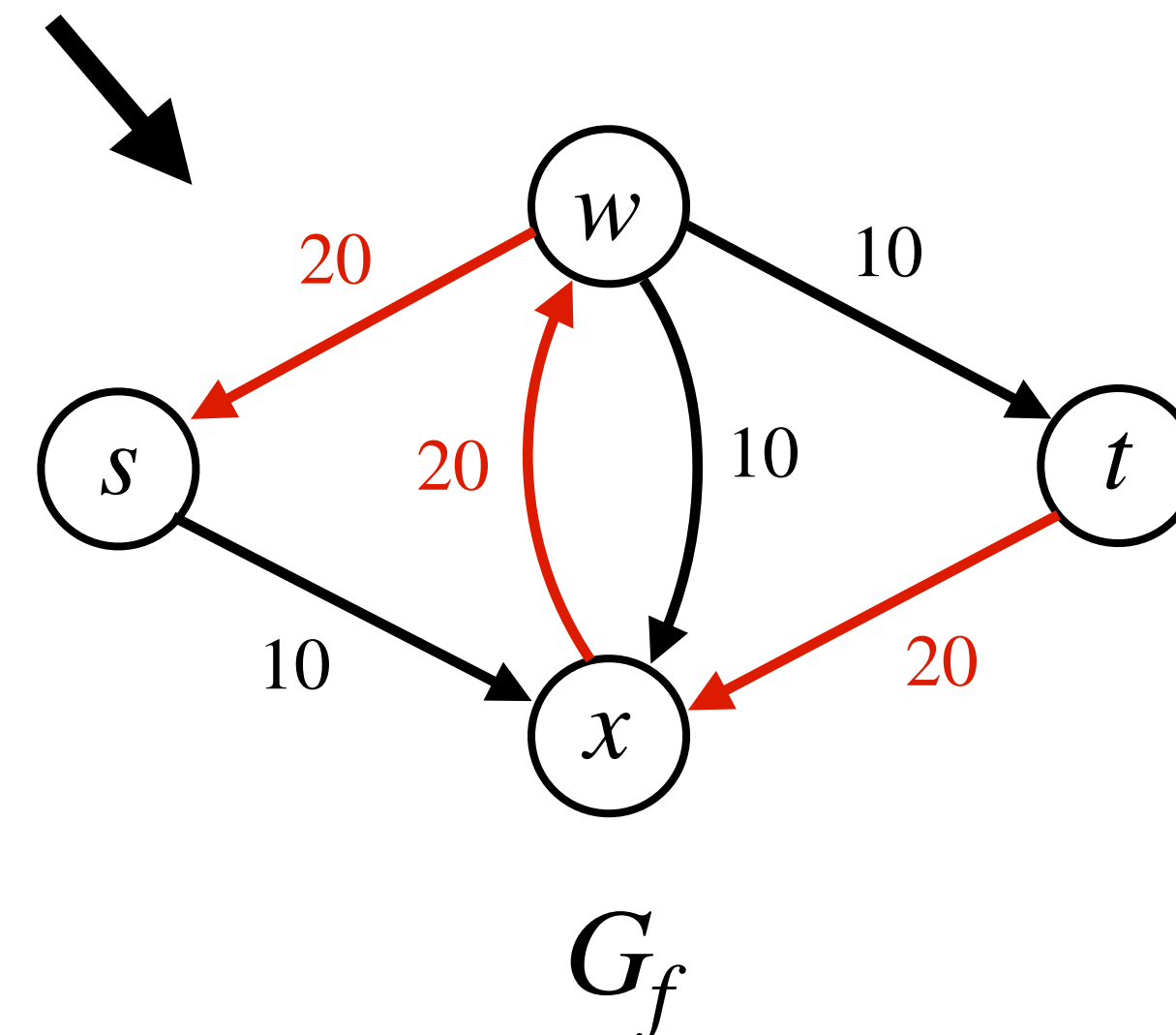
Defn: For a given flow network $G = (V, E)$ and flow f , residual network $G_f = (V, E_f)$ is a flow network, where E_f contains:

- **Forward edges:** Edge (u, v) with capacity $c_f(u, v) = c(u, v) - f(u, v) > 0$
- **Backward edges:** Edge (v, u) with capacity $c_f(v, u) = f(u, v) > 0$

Example:

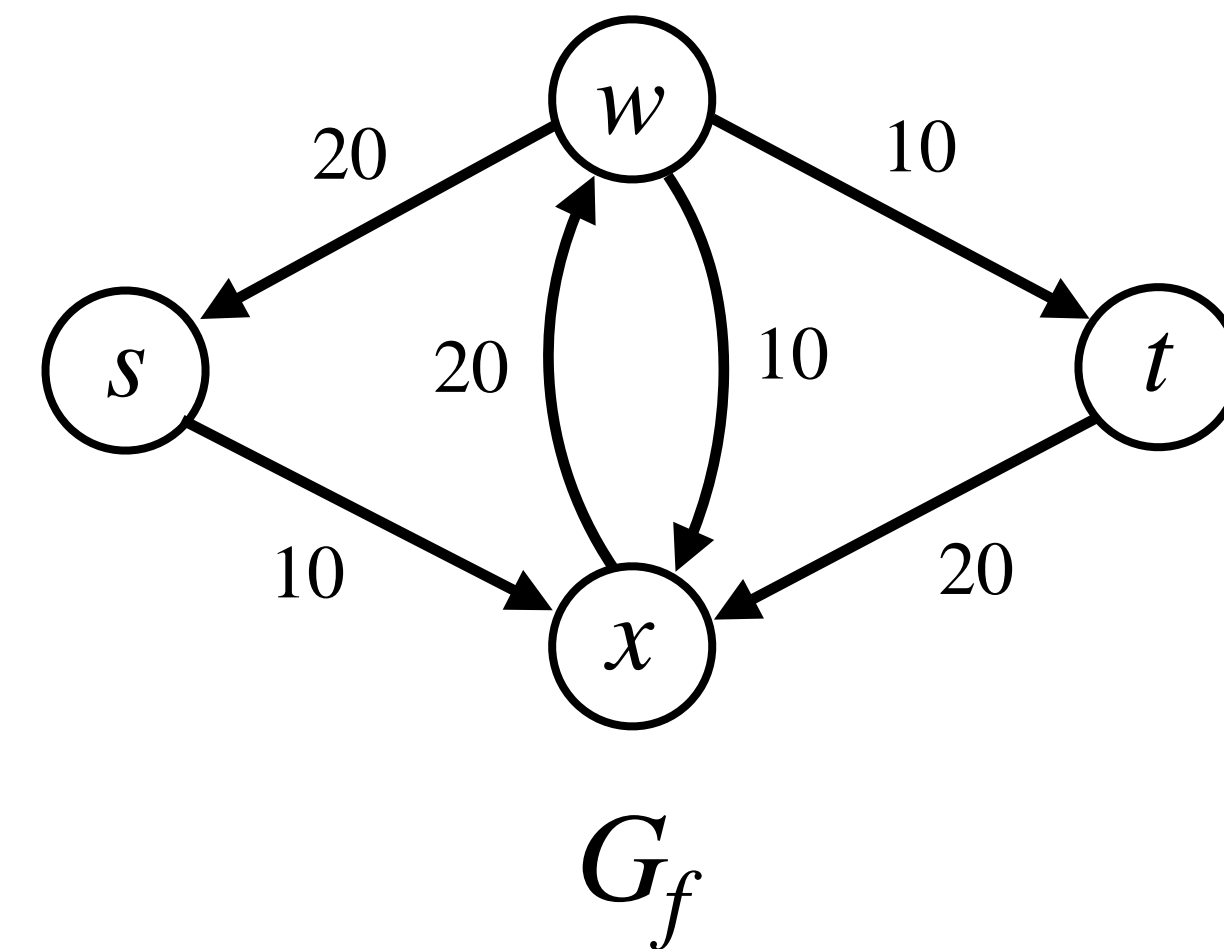
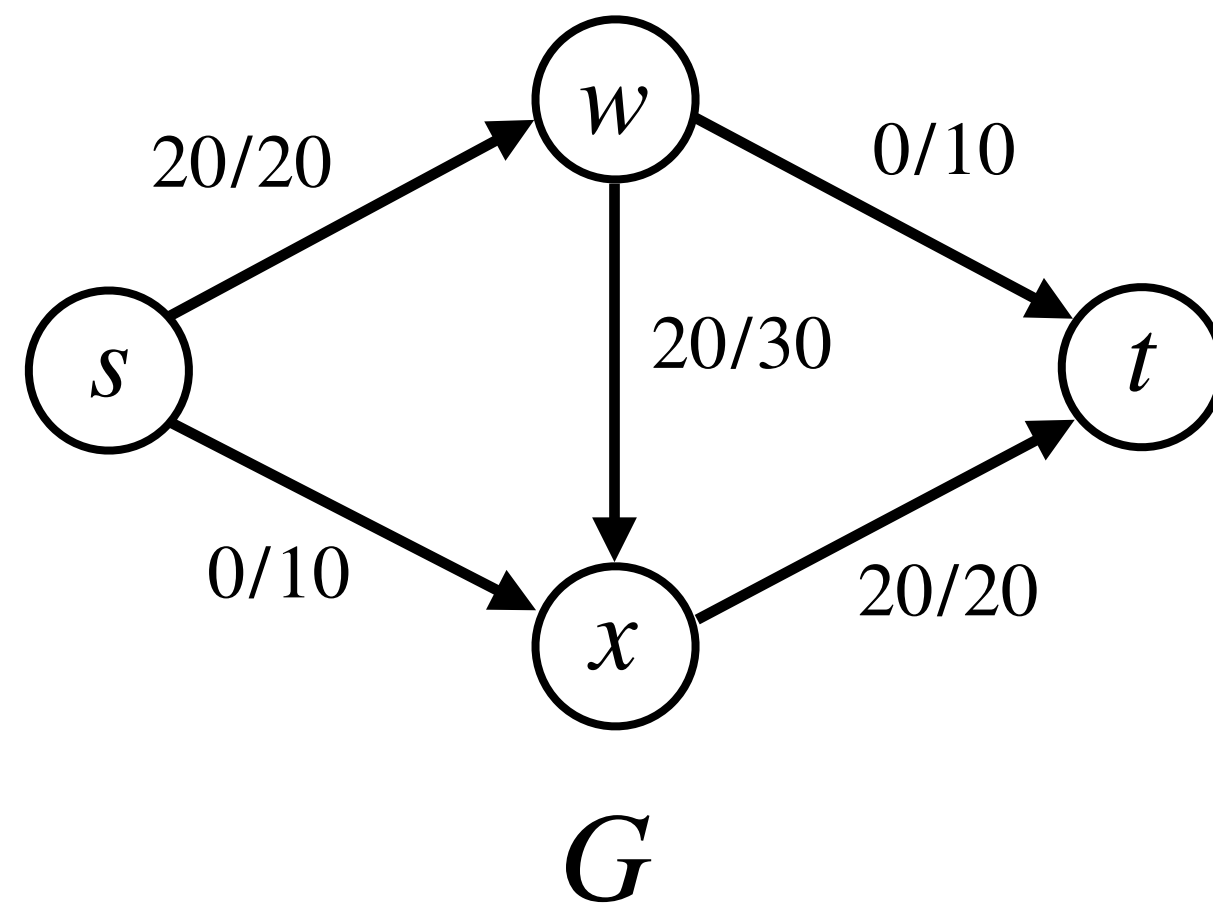


Backward edges to decrease the flow on some edges



Augmenting Flows via Residual Networks

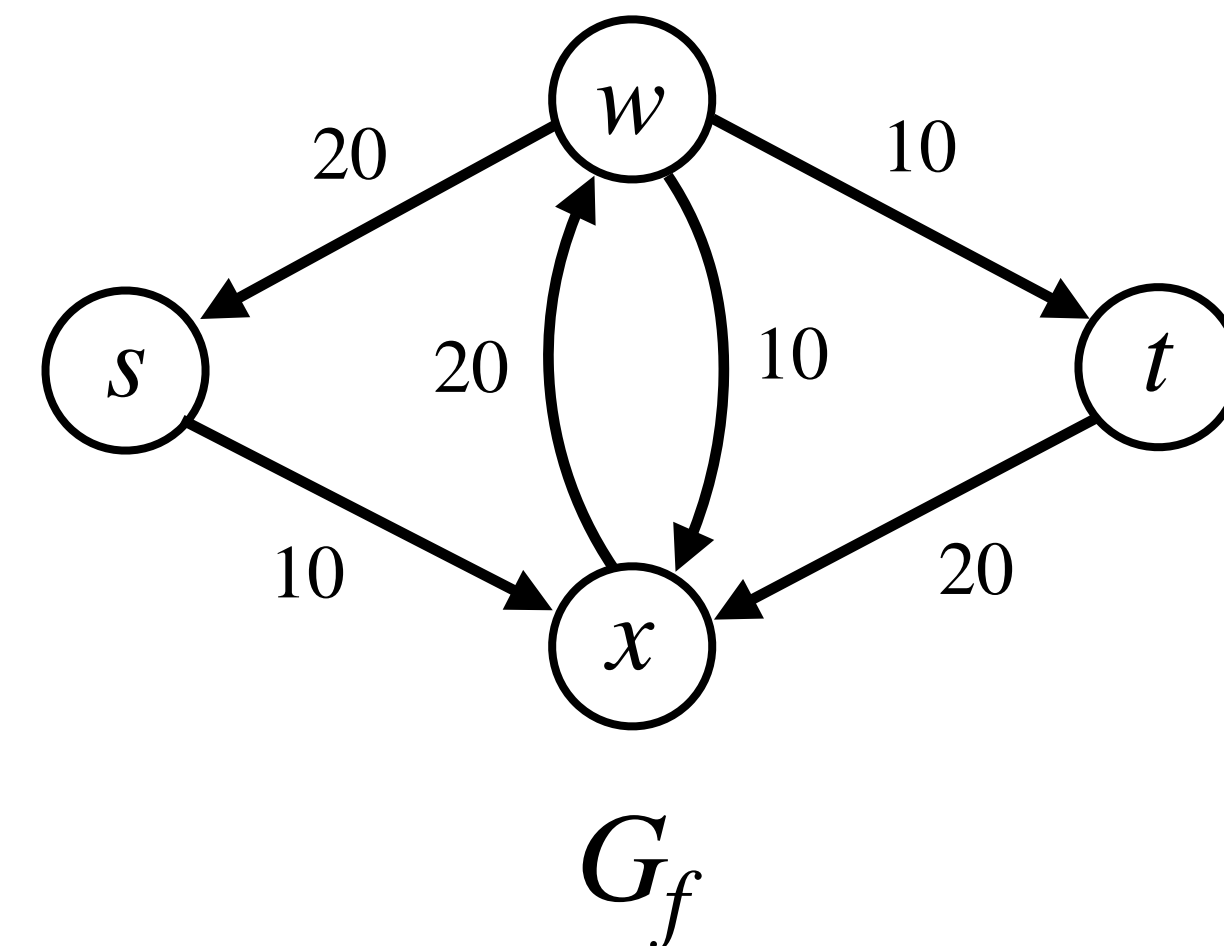
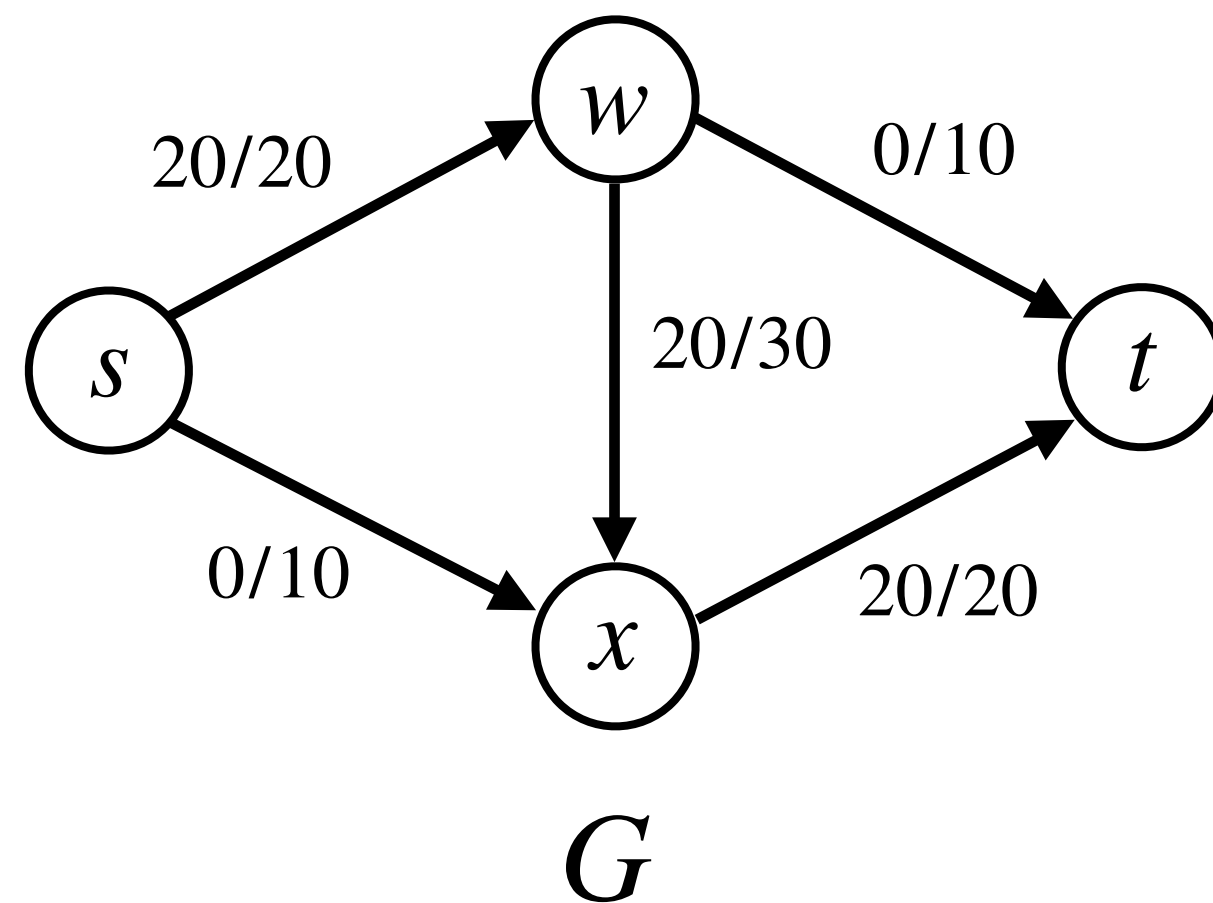
Example:



Augmenting Flows via Residual Networks

- Find $s \rightsquigarrow t$ path P in the residual network G_f and its bottleneck capacity δ .

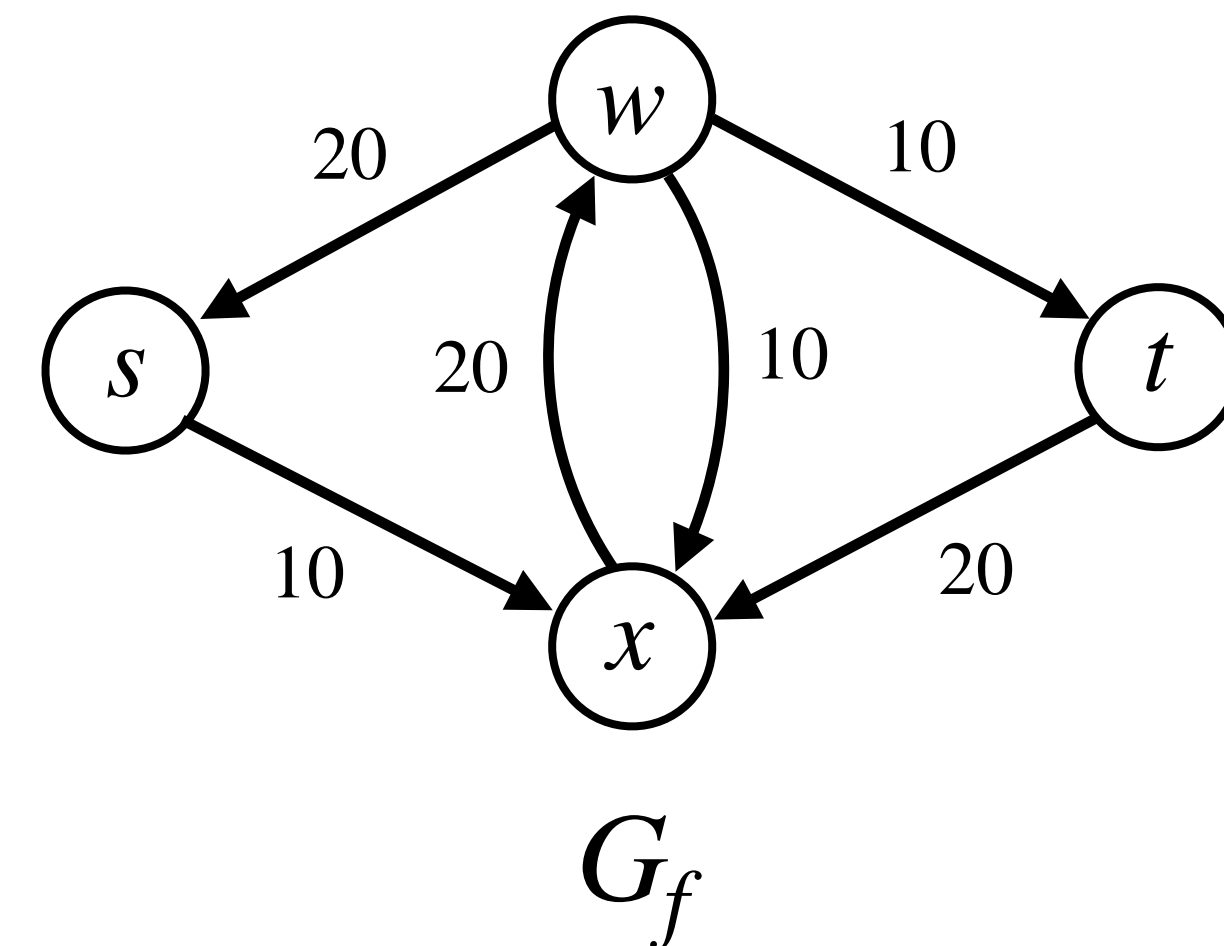
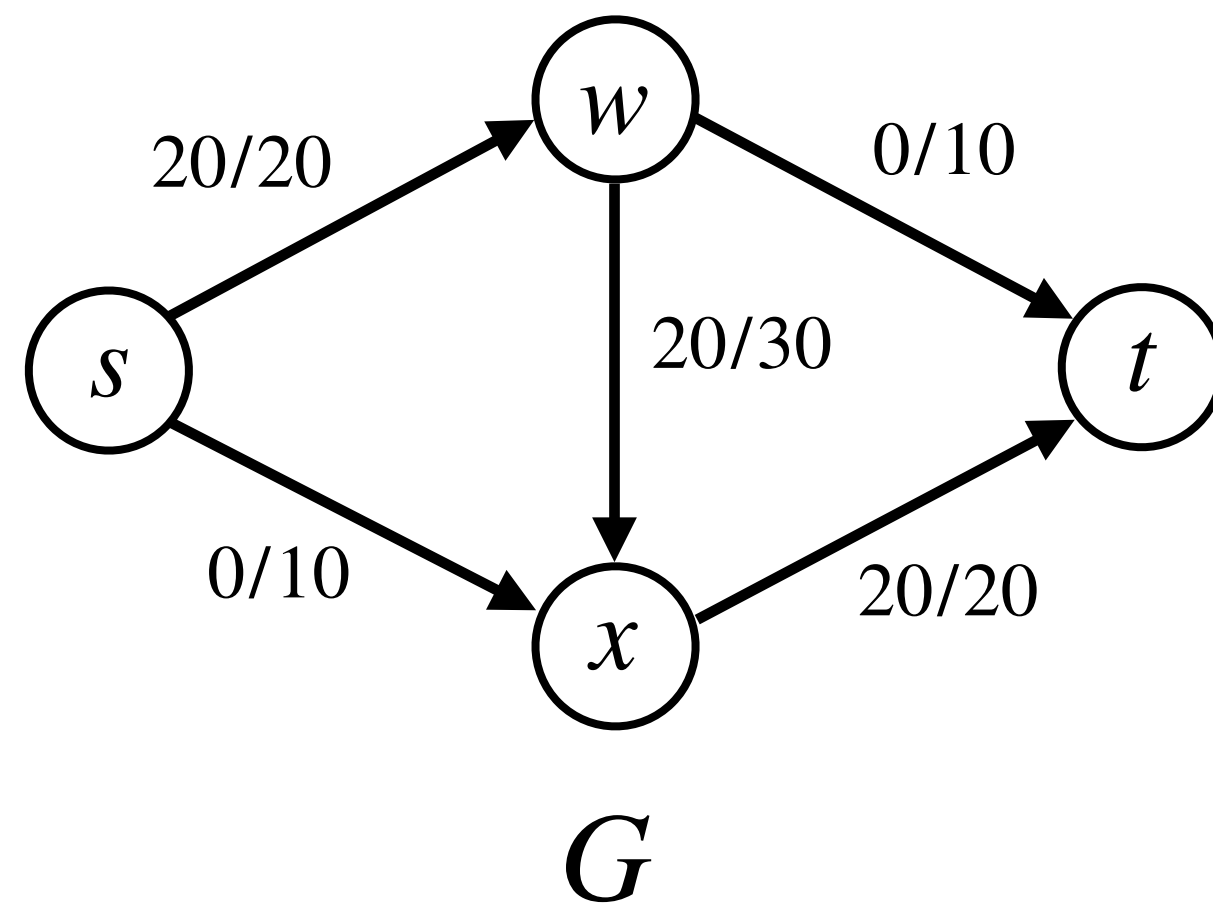
Example:



Augmenting Flows via Residual Networks

- Find $s \rightsquigarrow t$ path P in the residual network G_f and its **bottleneck capacity** δ .
- For every $(u, v) \in P$:

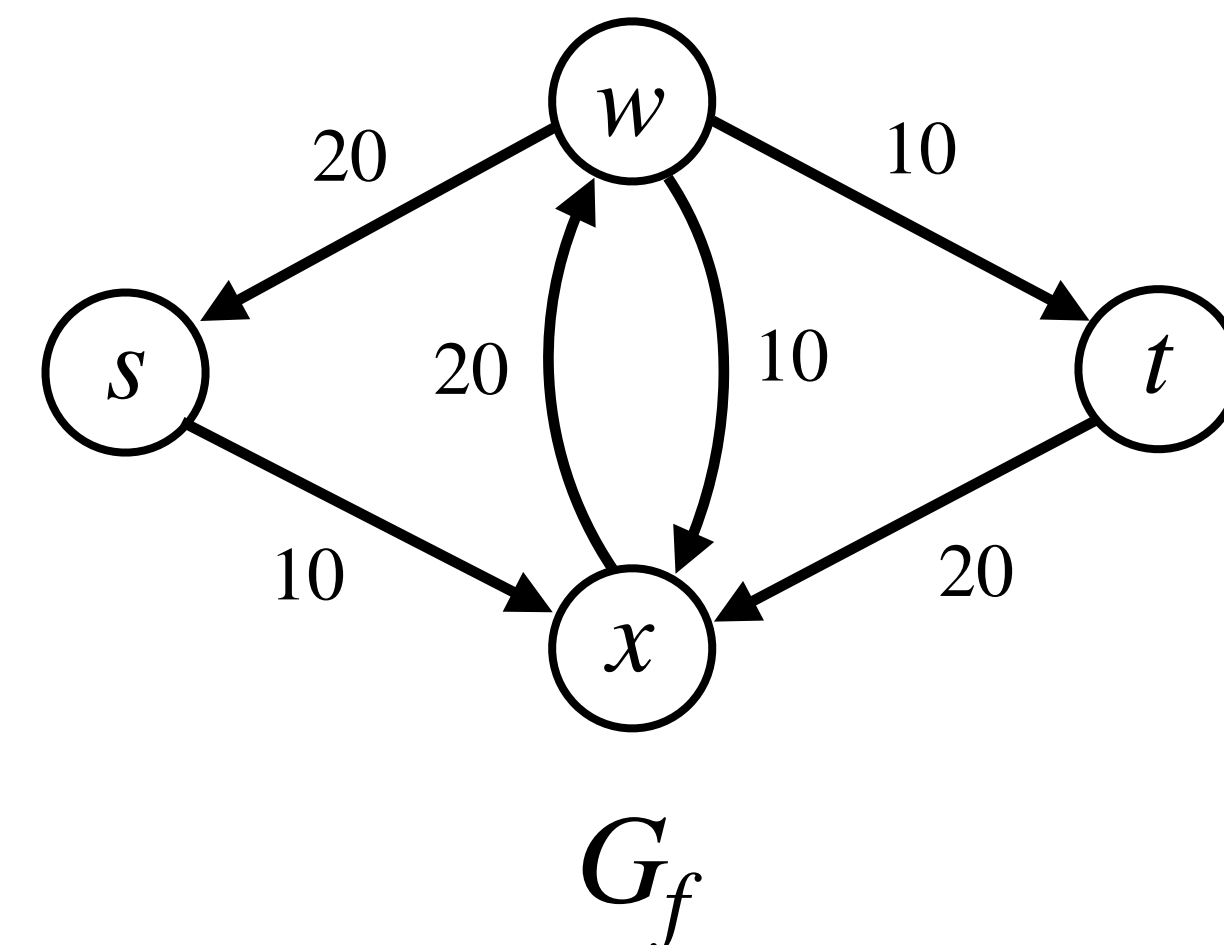
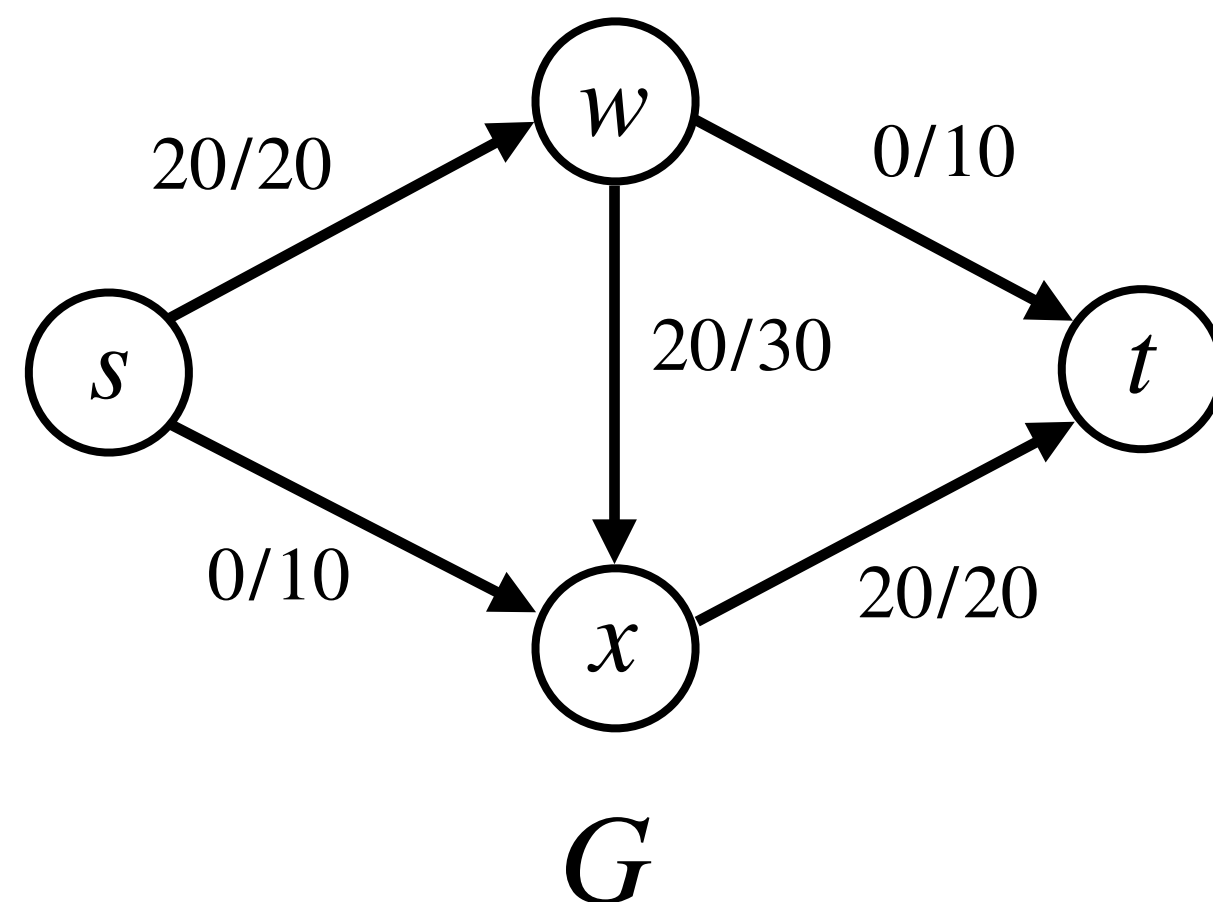
Example:



Augmenting Flows via Residual Networks

- Find $s \rightsquigarrow t$ path P in the residual network G_f and its bottleneck capacity δ .
- For every $(u, v) \in P$:
 - If $(u, v) \in E(G)$, add δ flow to (u, v) in f .

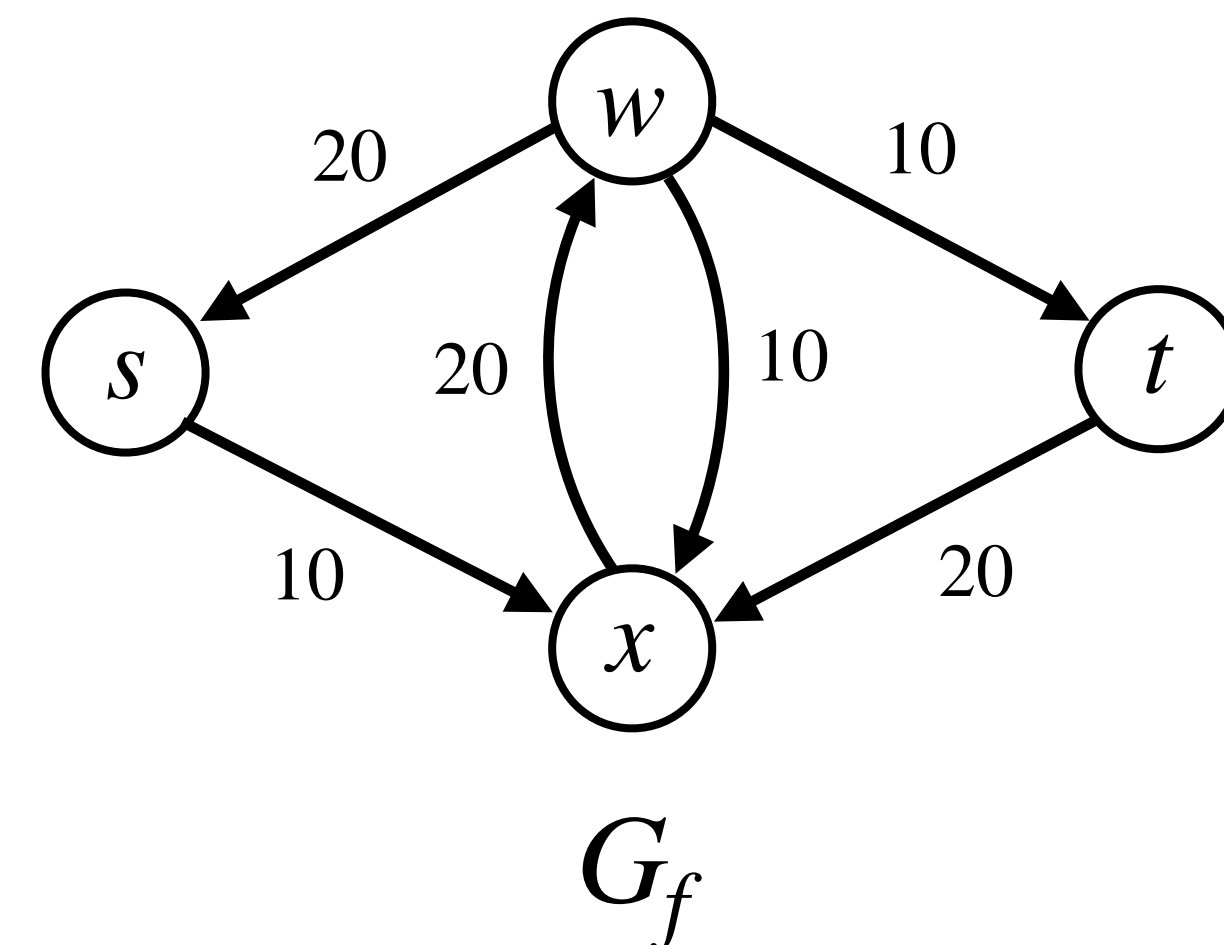
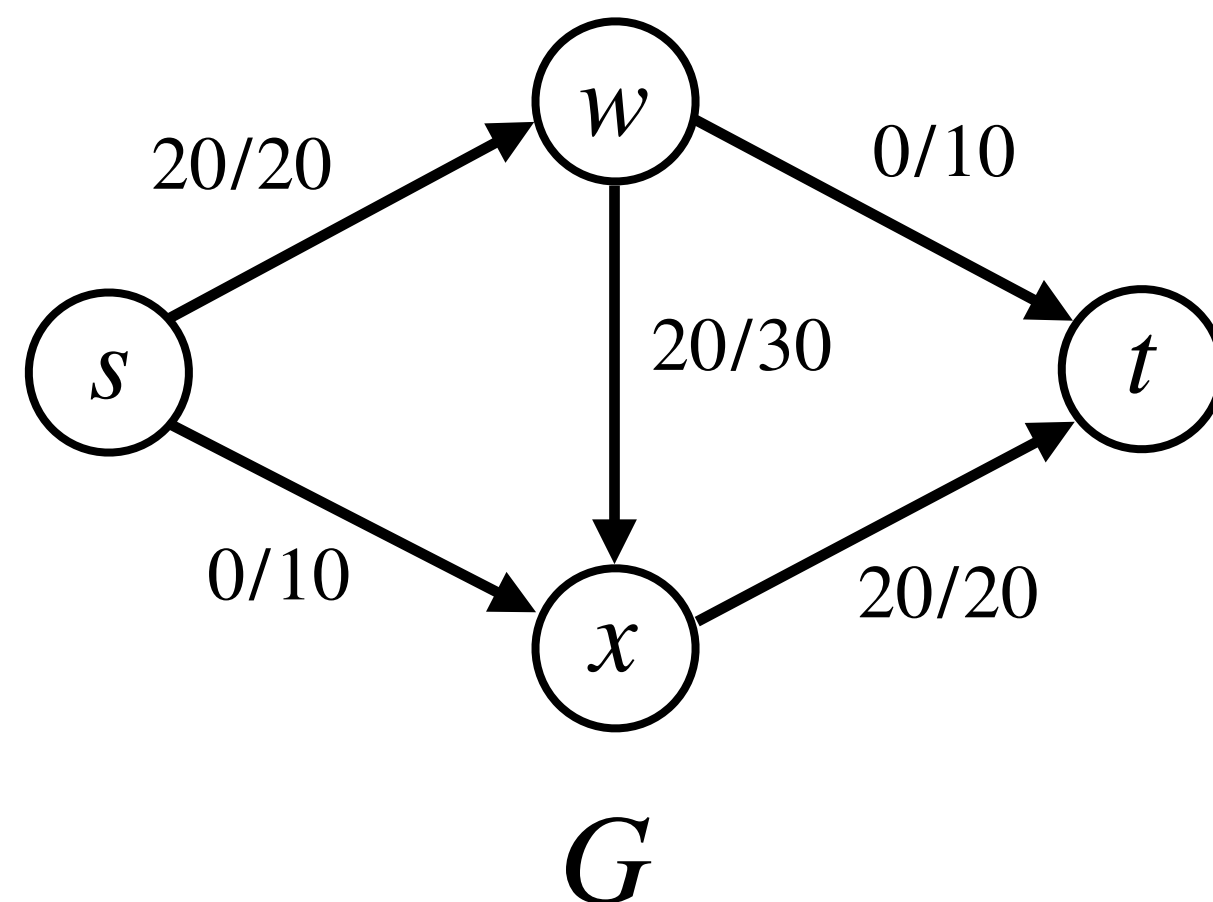
Example:



Augmenting Flows via Residual Networks

- Find $s \rightsquigarrow t$ path P in the residual network G_f and its bottleneck capacity δ .
- For every $(u, v) \in P$:
 - If $(u, v) \in E(G)$, add δ flow to (u, v) in f .
 - If $(v, u) \in E(G)$, subtract δ flow from (v, u) in f .

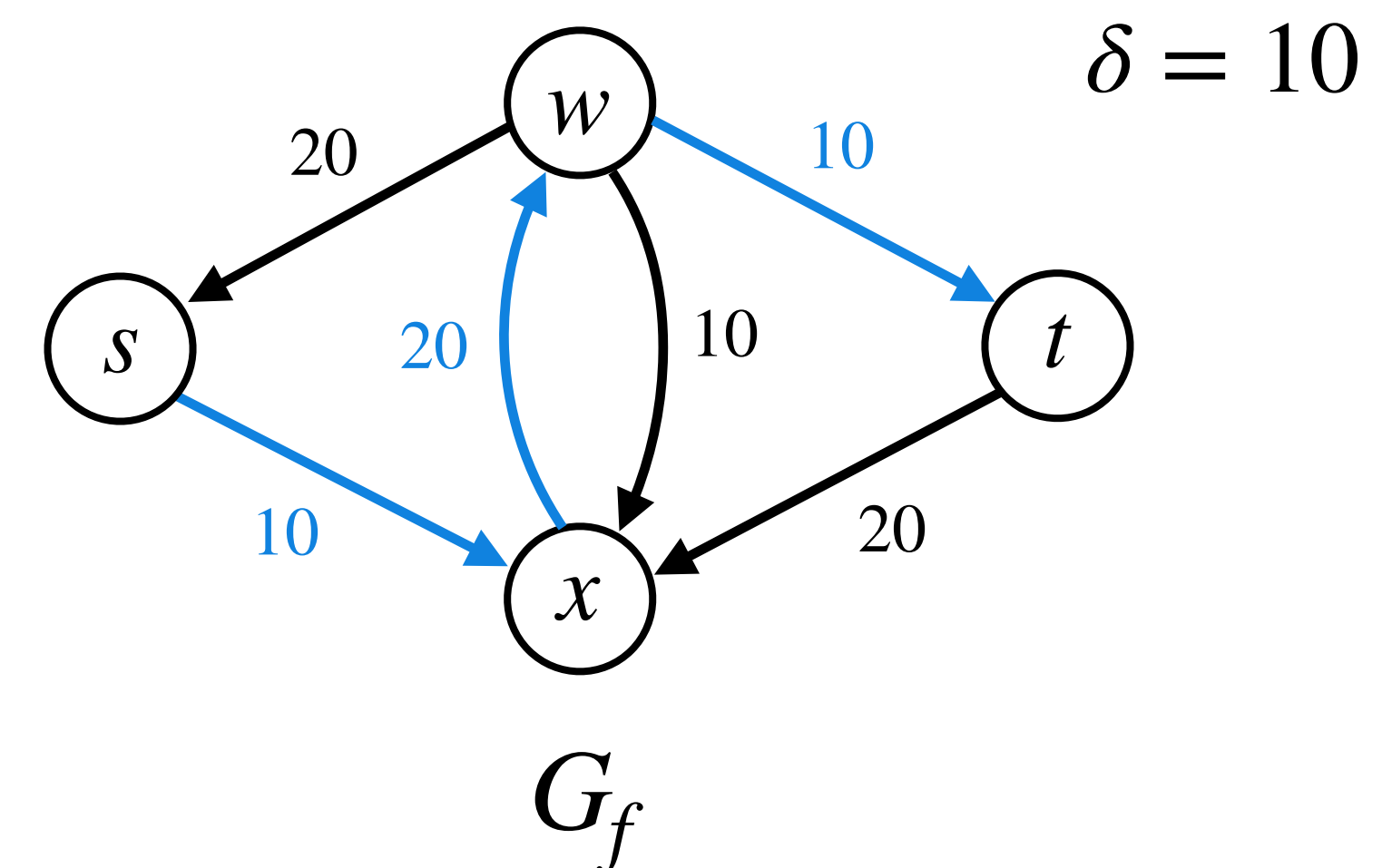
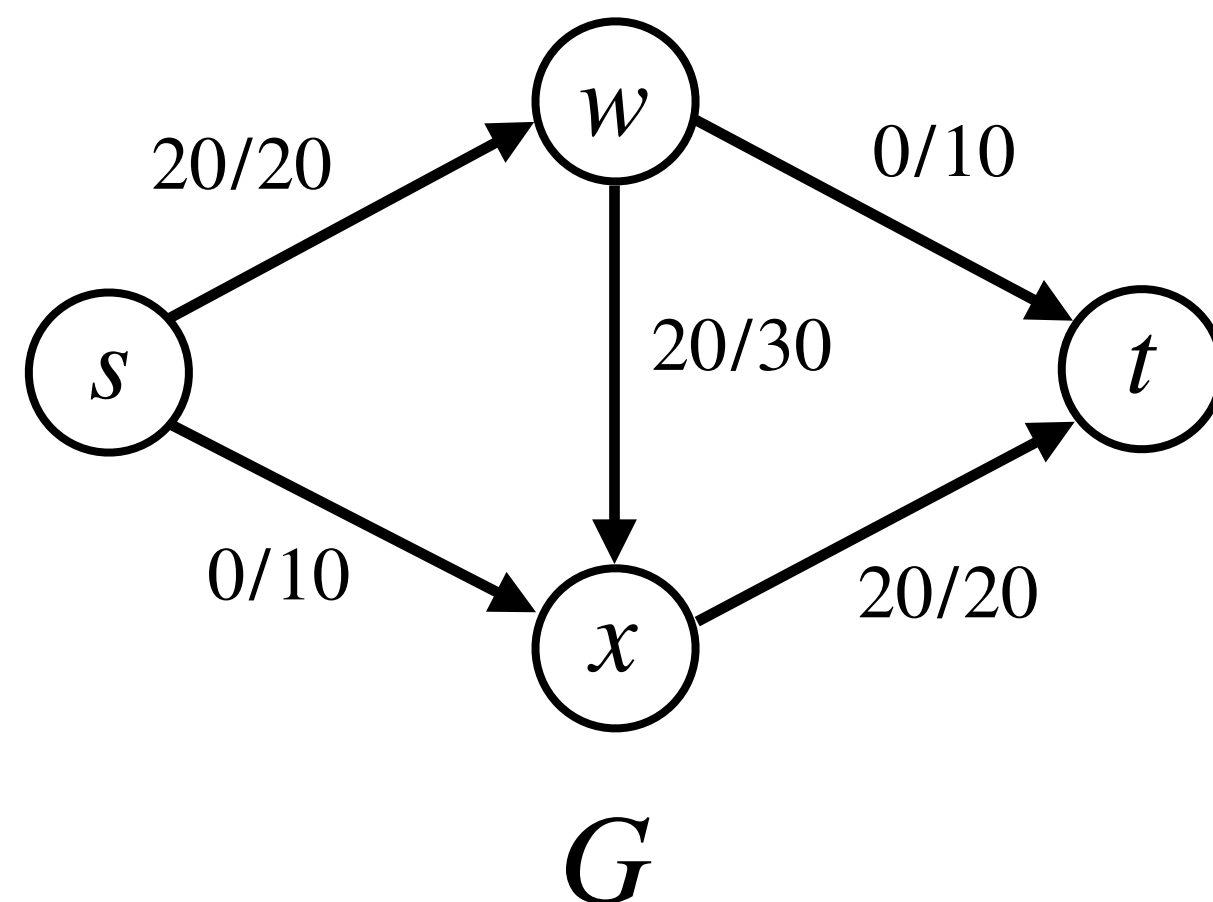
Example:



Augmenting Flows via Residual Networks

- Find $s \rightsquigarrow t$ path P in the residual network G_f and its bottleneck capacity δ .
- For every $(u, v) \in P$:
 - If $(u, v) \in E(G)$, add δ flow to (u, v) in f .
 - If $(v, u) \in E(G)$, subtract δ flow from (v, u) in f .

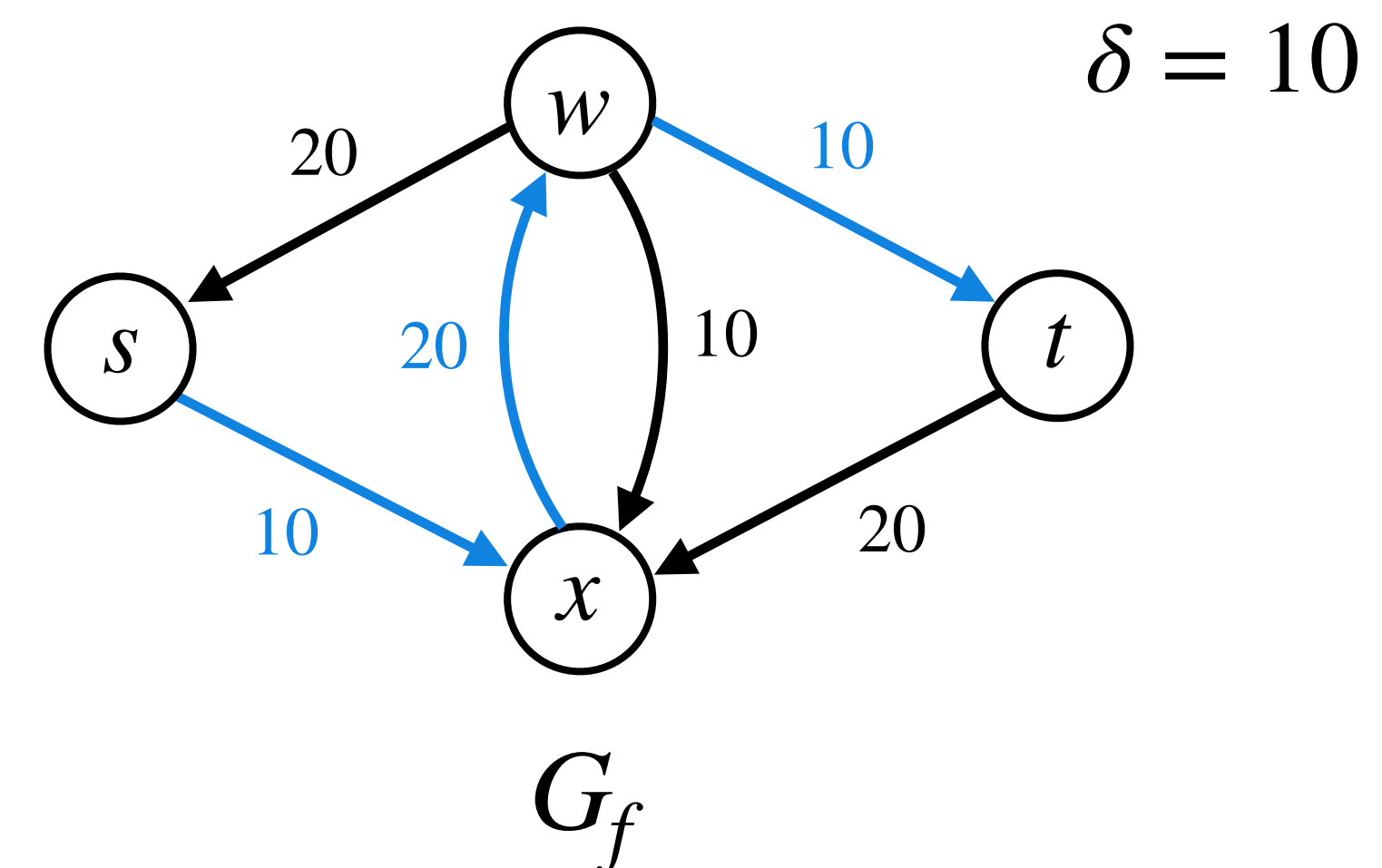
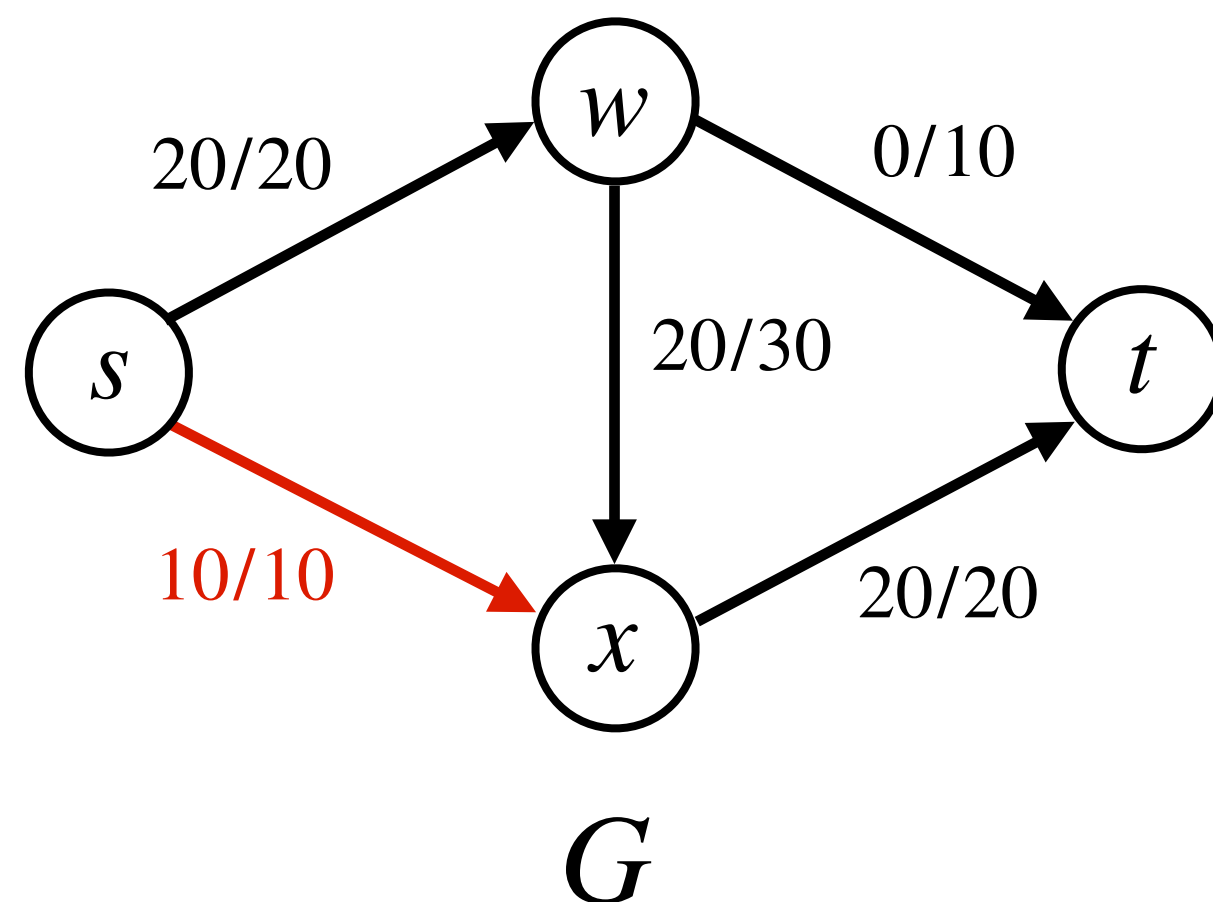
Example:



Augmenting Flows via Residual Networks

- Find $s \rightsquigarrow t$ path P in the residual network G_f and its **bottleneck capacity** δ .
- For every $(u, v) \in P$:
 - If $(u, v) \in E(G)$, add δ flow to (u, v) in f .
 - If $(v, u) \in E(G)$, subtract δ flow from (v, u) in f .

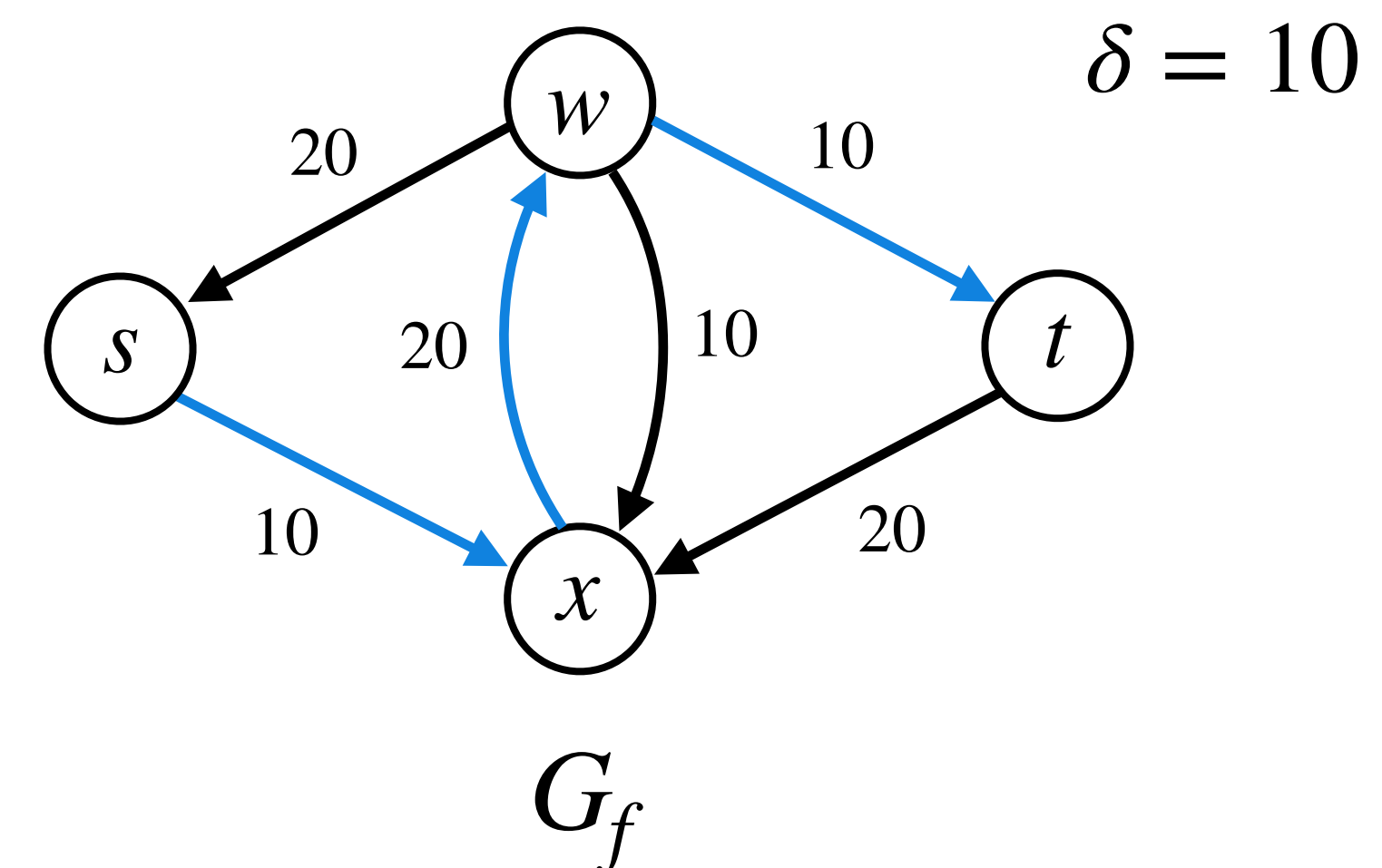
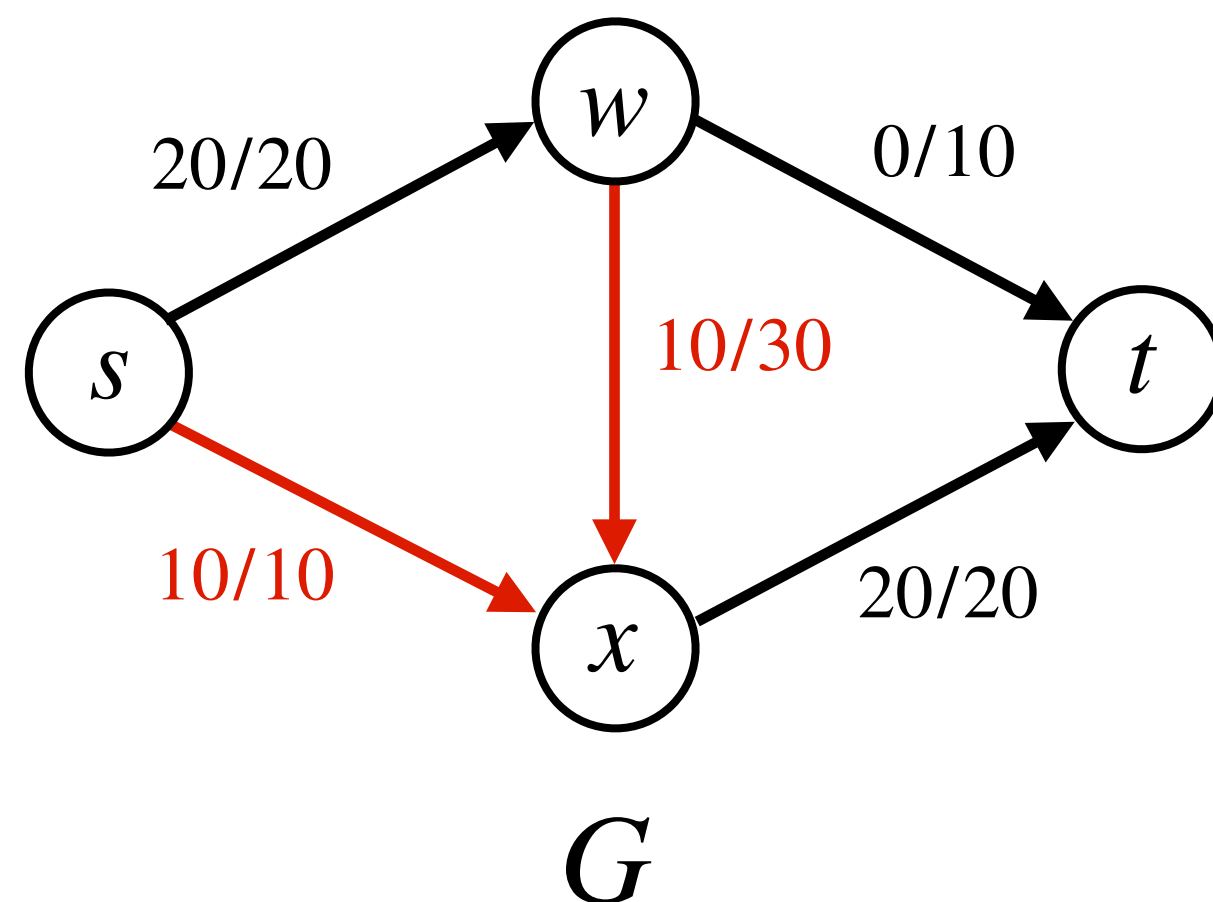
Example:



Augmenting Flows via Residual Networks

- Find $s \rightsquigarrow t$ path P in the residual network G_f and its **bottleneck capacity** δ .
- For every $(u, v) \in P$:
 - If $(u, v) \in E(G)$, add δ flow to (u, v) in f .
 - If $(v, u) \in E(G)$, subtract δ flow from (v, u) in f .

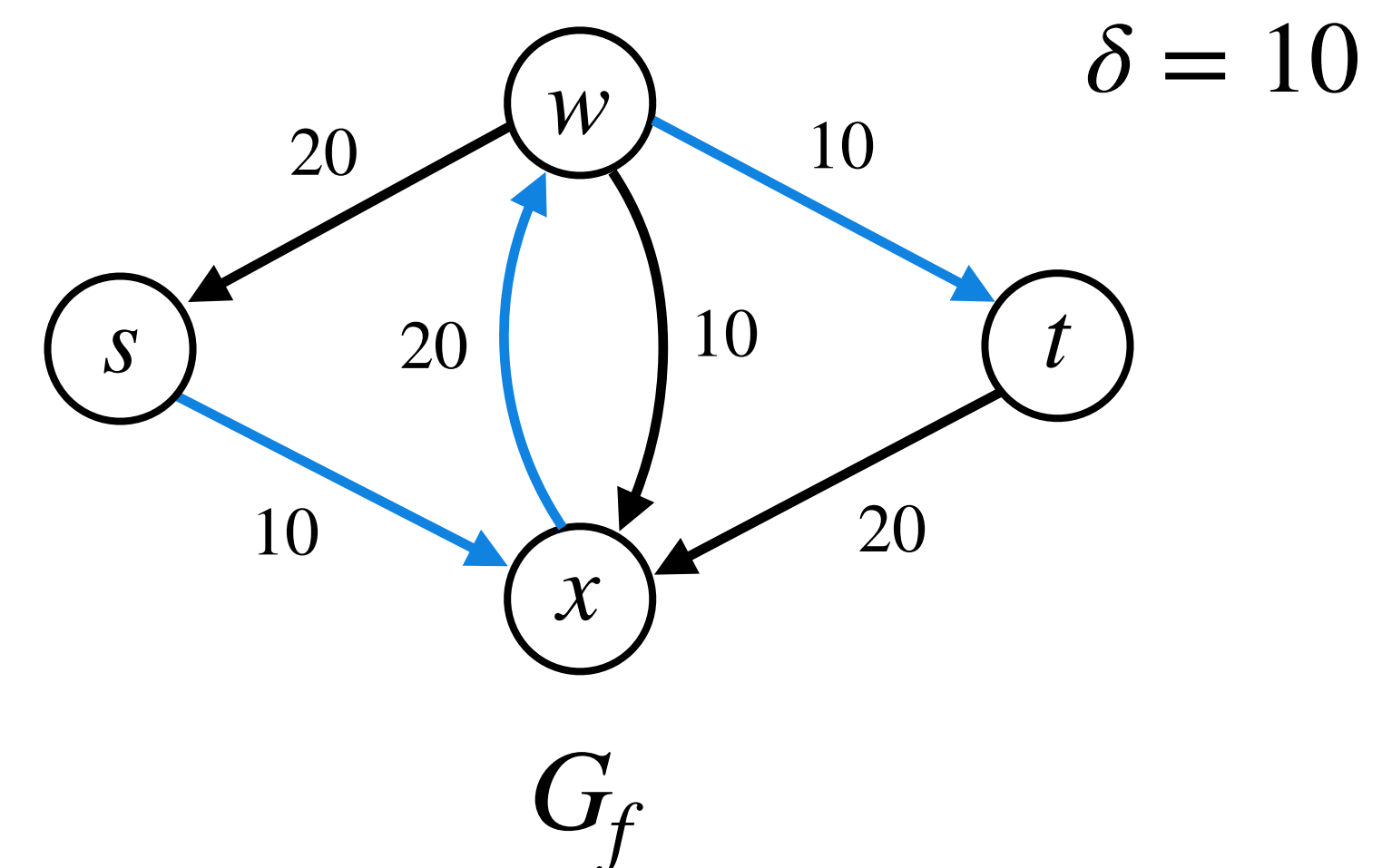
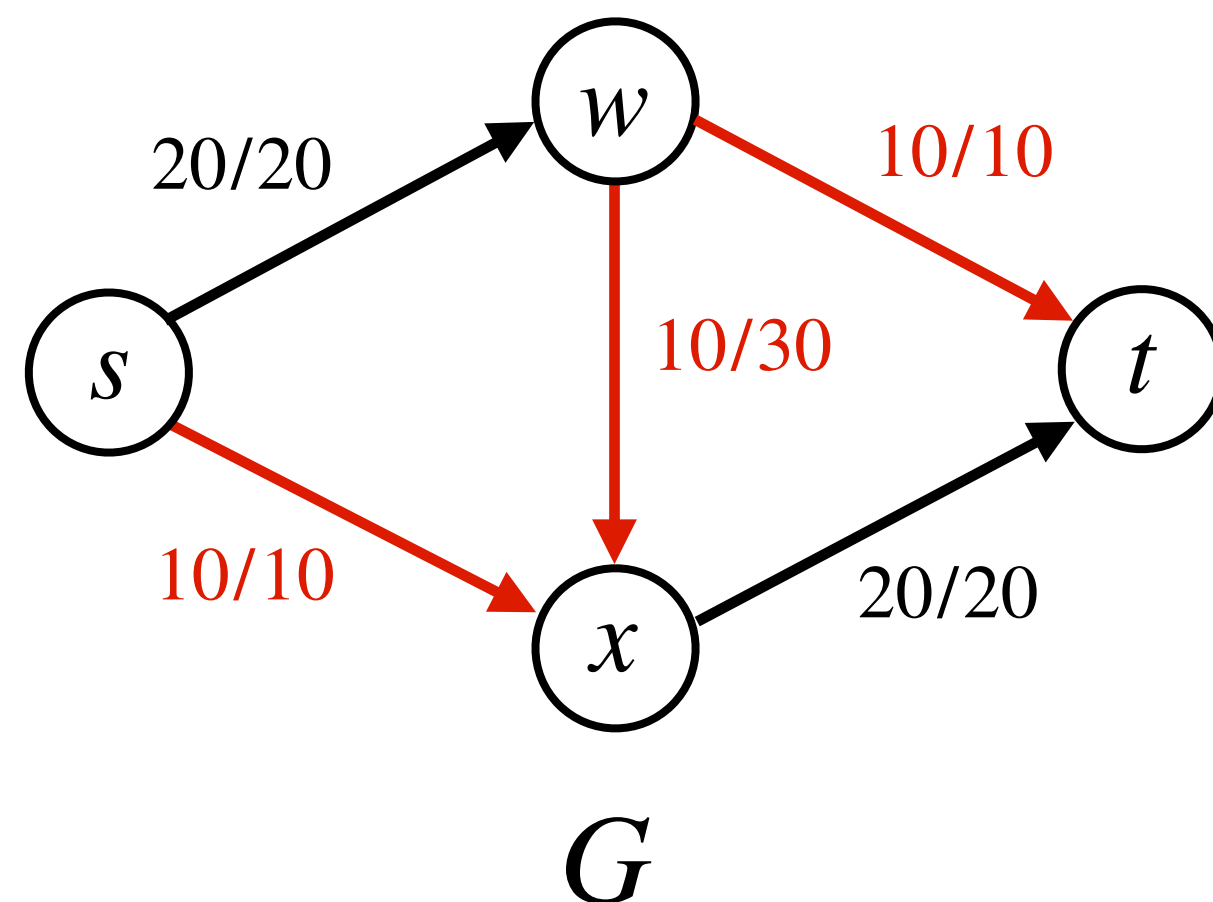
Example:



Augmenting Flows via Residual Networks

- Find $s \rightsquigarrow t$ path P in the residual network G_f and its **bottleneck capacity** δ .
- For every $(u, v) \in P$:
 - If $(u, v) \in E(G)$, add δ flow to (u, v) in f .
 - If $(v, u) \in E(G)$, subtract δ flow from (v, u) in f .

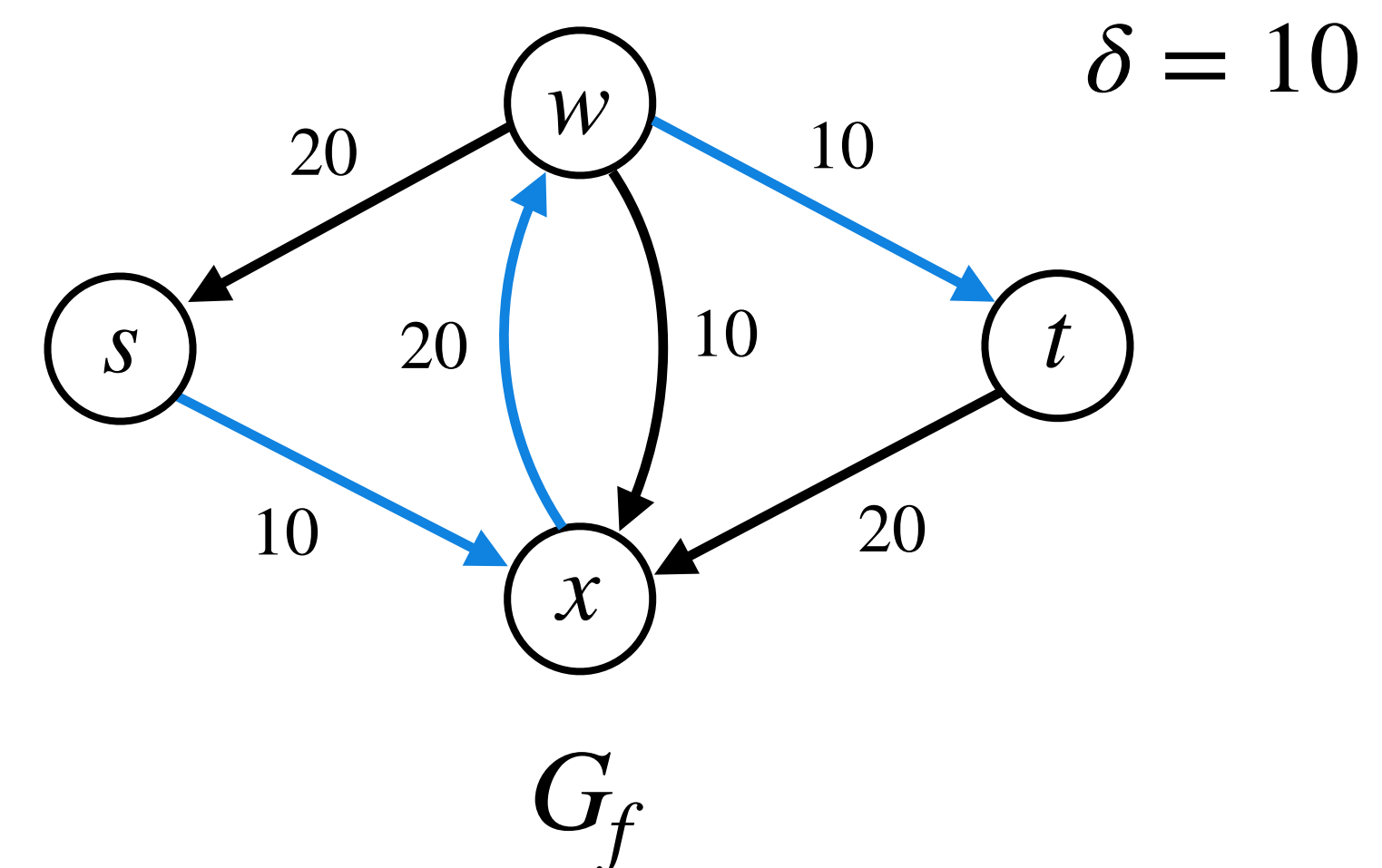
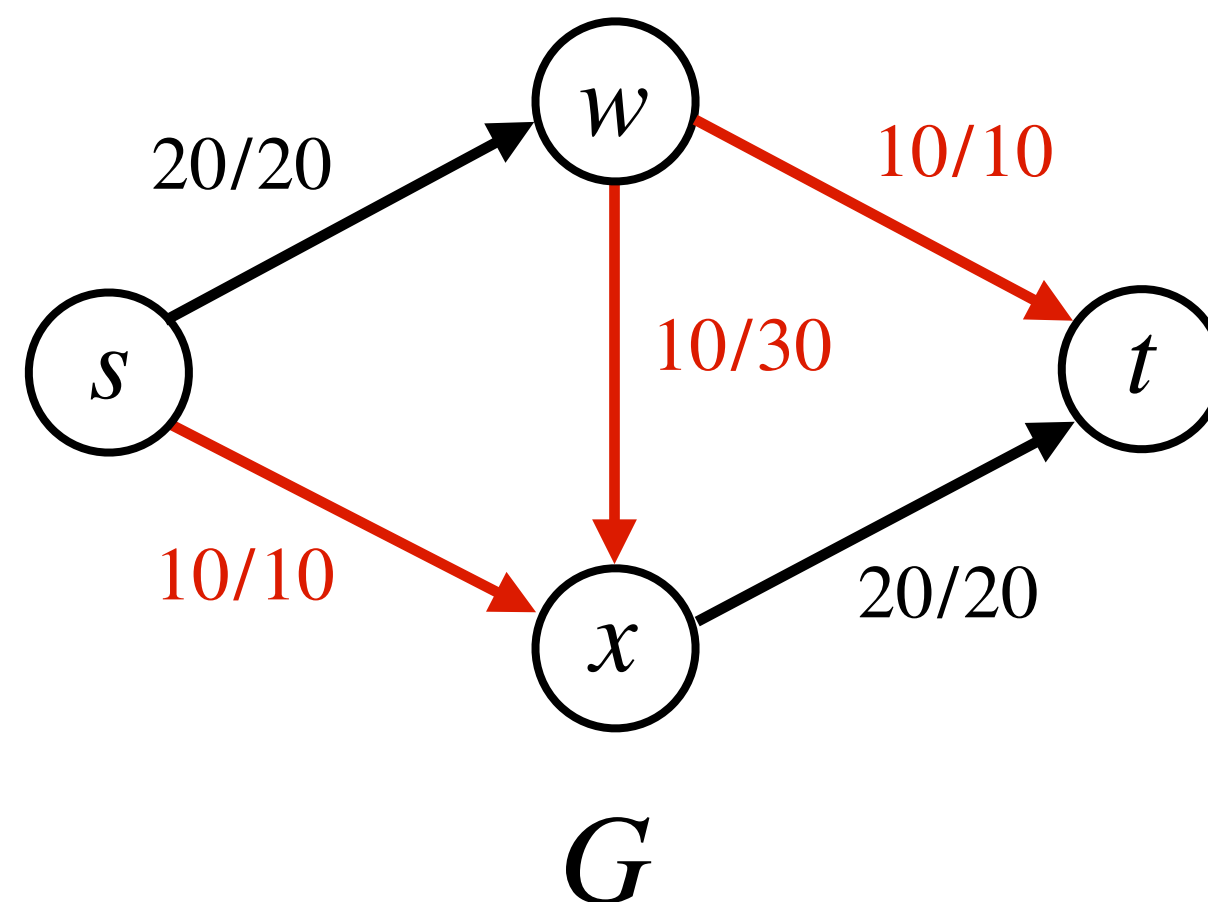
Example:



Augmenting Flows via Residual Networks

- Find $s \rightsquigarrow t$ path P in the residual network G_f and its **bottleneck capacity** δ .
 - For every $(u, v) \in P$:
 - If $(u, v) \in E(G)$, add δ flow to (u, v) in f .
 - If $(v, u) \in E(G)$, subtract δ flow from (v, u) in f .
- What about capacity, conservation constraints?

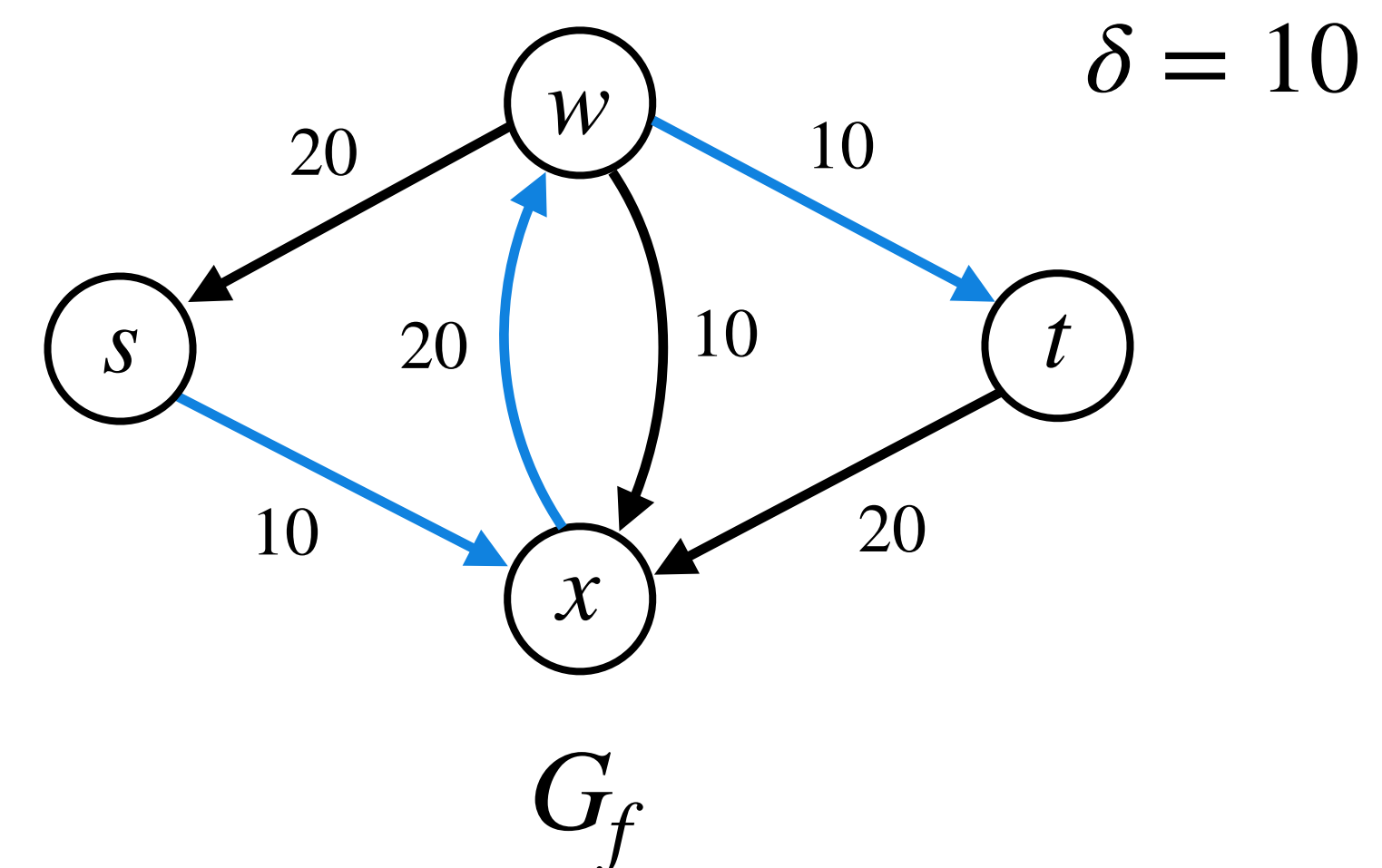
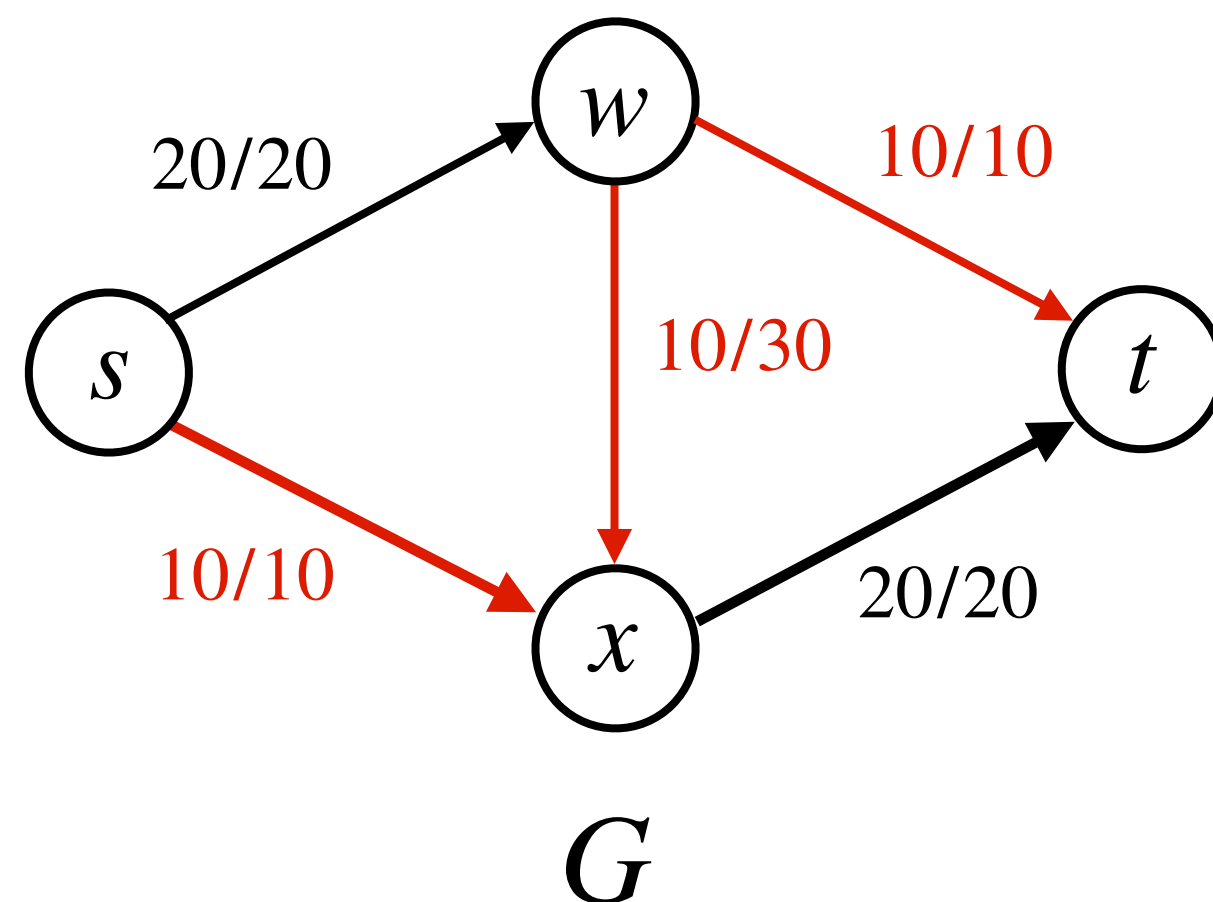
Example:



Augmenting Flows via Residual Networks

- Find $s \rightsquigarrow t$ path P in the residual network G_f and its **bottleneck capacity** δ .
- For every $(u, v) \in P$:
 - If $(u, v) \in E(G)$, add δ flow to (u, v) in f .
 - If $(v, u) \in E(G)$, subtract δ flow from (v, u) in f .

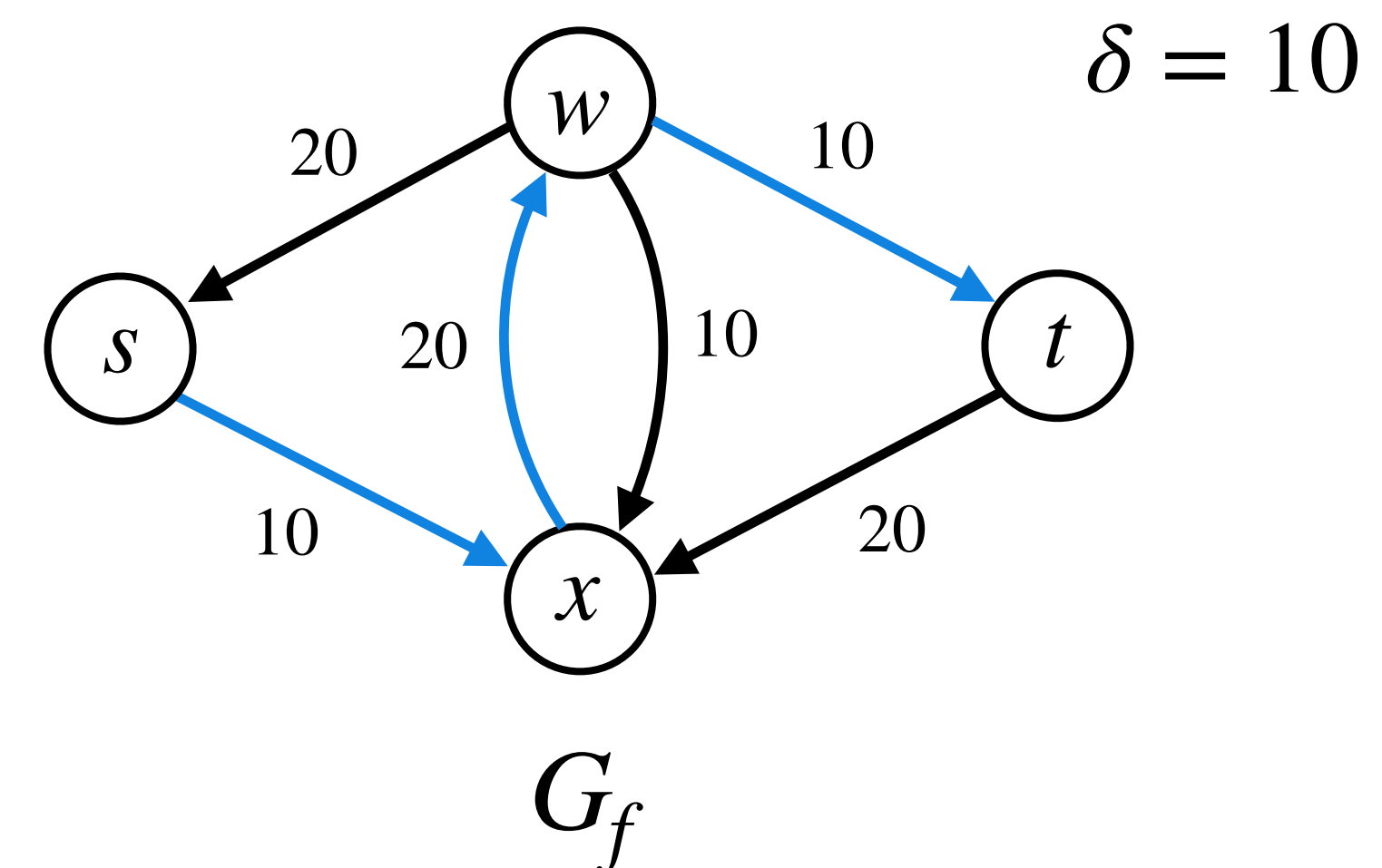
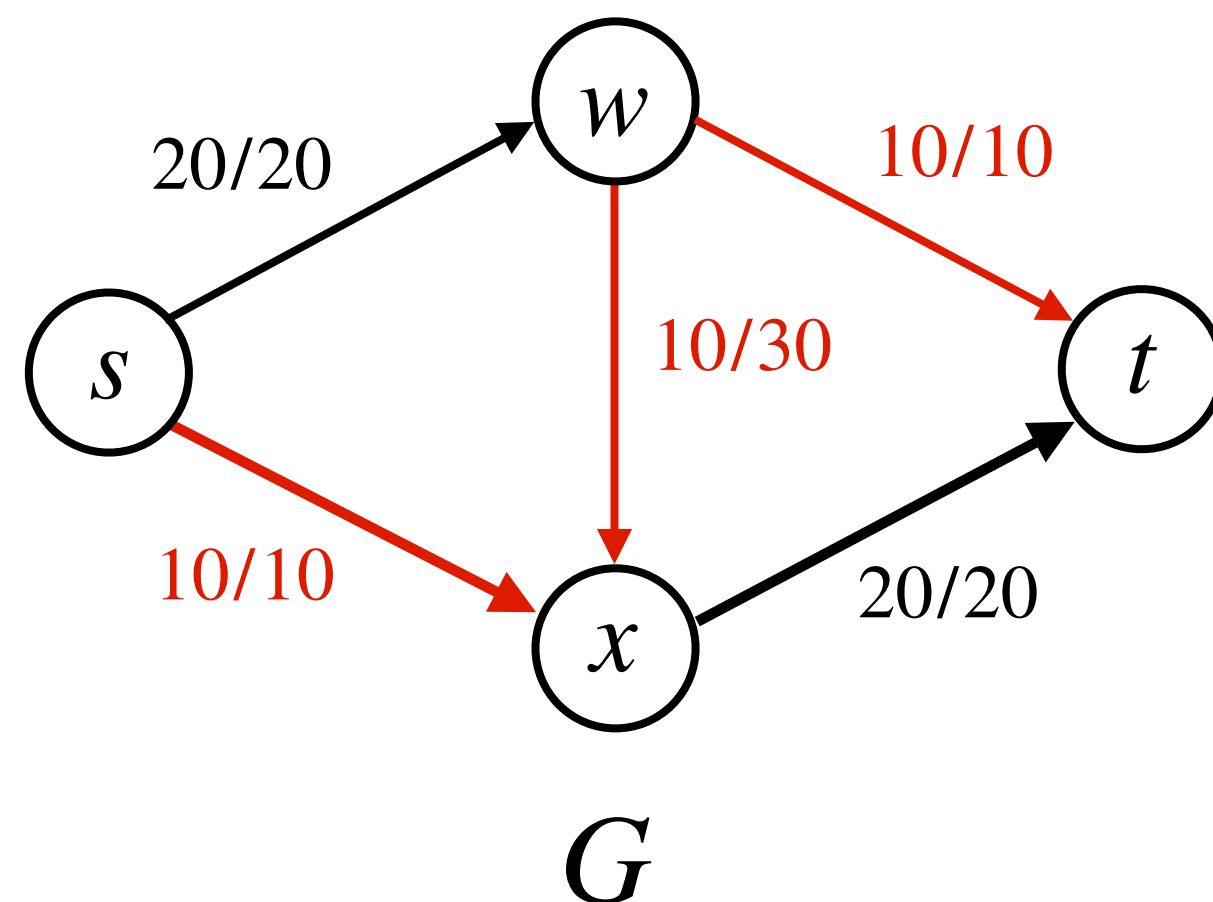
Example:



Augmenting Flows via Residual Networks

- Find $s \rightsquigarrow t$ path P in the residual network G_f and its **bottleneck capacity** δ .
 - For every $(u, v) \in P$:
 - If $(u, v) \in E(G)$, add δ flow to (u, v) in f .
 - If $(v, u) \in E(G)$, subtract δ flow from (v, u) in f .
- Will flow value increase?

Example:



Augmenting Paths

Augmenting Paths

Defn: Given a flow network $G = (V, E)$ and a flow f , an **augmenting path** is path from s to t

Augmenting Paths

Defn: Given a flow network $G = (V, E)$ and a flow f , an **augmenting path** is path from s to t in the residual network G_f .

Augmenting Paths

Defn: Given a flow network $G = (V, E)$ and a flow f , an **augmenting path** is path from s to t in the residual network G_f .

Defn: The **bottleneck capacity** of an augmenting path P is the least capacity of any edge in P .

Augmenting Paths

Defn: Given a flow network $G = (V, E)$ and a flow f , an **augmenting path** is path from s to t in the residual network G_f .

Defn: The **bottleneck capacity** of an augmenting path P is the least capacity of any edge in P .

Augmenting flow f : For every $(u, v) \in P$:

Augmenting Paths

Defn: Given a flow network $G = (V, E)$ and a flow f , an **augmenting path** is path from s to t in the residual network G_f .

Defn: The **bottleneck capacity** of an augmenting path P is the least capacity of any edge in P .

Augmenting flow f : For every $(u, v) \in P$:

- If $(u, v) \in E(G)$, add δ flow to (u, v) in f .

Augmenting Paths

Defn: Given a flow network $G = (V, E)$ and a flow f , an **augmenting path** is path from s to t in the residual network G_f .

Defn: The **bottleneck capacity** of an augmenting path P is the least capacity of any edge in P .

Augmenting flow f : For every $(u, v) \in P$:

- If $(u, v) \in E(G)$, add δ flow to (u, v) in f .
- If $(v, u) \in E(G)$, subtract δ flow from (v, u) in f .

Ford-Fulkerson Method

Ford-Fulkerson Method

Ford-Fulkerson(G, s, t):

Ford-Fulkerson Method

Ford-Fulkerson(G, s, t):

1. for each edge $(u, v) \in E(G)$

Ford-Fulkerson Method

Ford-Fulkerson(G, s, t):

1. **for** each edge $(u, v) \in E(G)$
2. $f(u, v) = 0$

Ford-Fulkerson Method

Ford-Fulkerson(G, s, t):

1. **for** each edge $(u, v) \in E(G)$
2. $f(u, v) = 0$
3. **while** there exists an $s \rightsquigarrow t$ path P in the residual network G_f

Ford-Fulkerson Method

Ford-Fulkerson(G, s, t):

1. **for** each edge $(u, v) \in E(G)$
2. $f(u, v) = 0$
3. **while** there exists an $s \rightsquigarrow t$ path P in the residual network G_f
4. $\delta = \text{Min}(c_f(u, v) : (u, v) \text{ in } P)$

Ford-Fulkerson Method

Ford-Fulkerson(G, s, t):

1. **for** each edge $(u, v) \in E(G)$
2. $f(u, v) = 0$
3. **while** there exists an $s \rightsquigarrow t$ path P in the residual network G_f
4. $\delta = \text{Min}(c_f(u, v) : (u, v) \text{ in } P)$
5. **for** each edge (u, v) in P

Ford-Fulkerson Method

Ford-Fulkerson(G, s, t):

1. **for** each edge $(u, v) \in E(G)$
2. $f(u, v) = 0$
3. **while** there exists an $s \rightsquigarrow t$ path P in the residual network G_f
4. $\delta = \text{Min}(c_f(u, v) : (u, v) \text{ in } P)$
5. **for** each edge (u, v) in P
6. **if** $(u, v) \in E$

Ford-Fulkerson Method

Ford-Fulkerson(G, s, t):

1. **for** each edge $(u, v) \in E(G)$
2. $f(u, v) = 0$
3. **while** there exists an $s \rightsquigarrow t$ path P in the residual network G_f
4. $\delta = \text{Min}(c_f(u, v) : (u, v) \text{ in } P)$
5. **for** each edge (u, v) in P
6. **if** $(u, v) \in E$
7. $f(u, v) = f(u, v) + \delta$

Ford-Fulkerson Method

Ford-Fulkerson(G, s, t):

1. **for** each edge $(u, v) \in E(G)$
2. $f(u, v) = 0$
3. **while** there exists an $s \rightsquigarrow t$ path P in the residual network G_f
4. $\delta = \text{Min}(c_f(u, v) : (u, v) \text{ in } P)$
5. **for** each edge (u, v) in P
6. **if** $(u, v) \in E$
7. $f(u, v) = f(u, v) + \delta$
8. **else**

Ford-Fulkerson Method

Ford-Fulkerson(G, s, t):

1. **for** each edge $(u, v) \in E(G)$
2. $f(u, v) = 0$
3. **while** there exists an $s \rightsquigarrow t$ path P in the residual network G_f
4. $\delta = \text{Min}(c_f(u, v) : (u, v) \text{ in } P)$
5. **for** each edge (u, v) in P
6. **if** $(u, v) \in E$
7. $f(u, v) = f(u, v) + \delta$
8. **else**
9. $f(v, u) = f(v, u) - \delta$

Ford-Fulkerson Method

Ford-Fulkerson(G, s, t):

1. **for** each edge $(u, v) \in E(G)$
2. $f(u, v) = 0$
3. **while** there exists an $s \rightsquigarrow t$ path P in the residual network G_f
4. $\delta = \text{Min}(c_f(u, v) : (u, v) \text{ in } P)$
5. **for** each edge (u, v) in P
6. **if** $(u, v) \in E$
7. $f(u, v) = f(u, v) + \delta$
8. **else**
9. $f(v, u) = f(v, u) - \delta$
10. **return** f

Ford-Fulkerson Method

Ford-Fulkerson(G, s, t):

1. **for** each edge $(u, v) \in E(G)$
2. $f(u, v) = 0$
3. **while** there exists an $s \rightsquigarrow t$ path P in the residual network G_f
4. $\delta = \text{Min}(c_f(u, v) : (u, v) \text{ in } P)$
5. **for** each edge (u, v) in P
6. **if** $(u, v) \in E$
7. $f(u, v) = f(u, v) + \delta$
8. **else**
9. $f(v, u) = f(v, u) - \delta$
10. **return** f

Why f will be maximum
when loop breaks?



Ford-Fulkerson Method: Correctness

Ford-Fulkerson Method: Correctness

Theorem: If there is no augmenting path in the residual network G_f , then f is a maximum flow.

Ford-Fulkerson Method: Correctness

Theorem: If there is no augmenting path in the residual network G_f , then f is a maximum flow.

Proof: Proof involves studying **cuts** and **max-flow, min-cut theorem**.

Ford-Fulkerson Method: Analysis

Ford-Fulkerson(G, s, t):

1. **for** each edge $(u, v) \in E(G)$
2. $f(u, v) = 0$
3. **while** there exists an $s \rightsquigarrow t$ path P in the residual network G_f
4. $\delta = \text{Min}(c_f(u, v) : (u, v) \text{ in } P)$
5. **for** each edge (u, v) in P
6. **if** $(u, v) \in E$
7. $f(u, v) = f(u, v) + \delta$
8. **else**
9. $f(v, u) = f(v, u) + \delta$
10. **return** f

Time Complexity: $O(|E| \cdot |f^*|)$

Ford-Fulkerson Method: Analysis

Ford-Fulkerson(G, s, t):

1. **for** each edge $(u, v) \in E(G)$  $O(|E|)$
2. $f(u, v) = 0$
3. **while** there exists an $s \rightsquigarrow t$ path P in the residual network G_f
4. $\delta = \text{Min}(c_f(u, v) : (u, v) \text{ in } P)$
5. **for** each edge (u, v) in P
6. **if** $(u, v) \in E$
7. $f(u, v) = f(u, v) + \delta$
8. **else**
9. $f(v, u) = f(v, u) + \delta$
10. **return** f

Time Complexity: $O(|E| \cdot |f^*|)$

Ford-Fulkerson Method: Analysis

Ford-Fulkerson(G, s, t):

1. **for** each edge $(u, v) \in E(G)$  $O(|E|)$
2. $f(u, v) = 0$
3. **while** there exists an $s \rightsquigarrow t$ path P in the residual network G_f 
4. $\delta = \text{Min}(c_f(u, v) : (u, v) \text{ in } P)$
5. **for** each edge (u, v) in P
6. **if** $(u, v) \in E$
7. $f(u, v) = f(u, v) + \delta$
8. **else**
9. $f(v, u) = f(v, u) + \delta$
10. **return** f

Loop may run for $|f^*|$ time,
where f^* is a max-flow,
as flow may increase by one
with every iteration

Time Complexity: $O(|E| \cdot |f^*|)$

Ford-Fulkerson Method: Analysis

Ford-Fulkerson(G, s, t):

1. for each edge $(u, v) \in E(G)$
2. $f(u, v) = 0$
3. while there exists an $s \rightsquigarrow t$ path P in the residual network G_f
4. $\delta = \text{Min}(c_f(u, v) : (u, v) \text{ in } P)$
5. for each edge (u, v) in P
6. if $(u, v) \in E$
7. $f(u, v) = f(u, v) + \delta$
8. else
9. $f(v, u) = f(v, u) + \delta$
10. return f

$O(|E|)$

Loop may run for $|f^*|$ time,
where f^* is a max-flow,
as flow may increase by one
with every iteration

Time Complexity: $O(|E| \cdot |f^*|)$

Ford-Fulkerson Method: Analysis

Ford-Fulkerson(G, s, t):

1. **for** each edge $(u, v) \in E(G)$
2. $f(u, v) = 0$
3. **while** there exists an $s \rightsquigarrow t$ path P in the residual network G_f
4. $\delta = \text{Min}(c_f(u, v) : (u, v) \text{ in } P)$
5. **for** each edge (u, v) in P
6. **if** $(u, v) \in E$
7. $f(u, v) = f(u, v) + \delta$
8. **else**
9. $f(v, v) = f(v, u) - \delta$
10. **return** f

$O(|E|)$

$O(|E|)$

Loop may run for $|f^*|$ time,
where f^* is a max-flow,
as flow may increase by one
with every iteration

Time Complexity: $O(|E| \cdot |f^*|)$